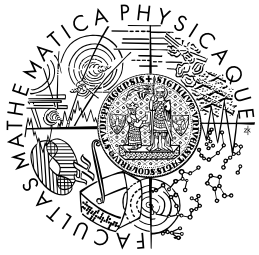


FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

**BACHELOR THESIS**

Seyed Danial Mohebbi Nowzar

# Nature Inspired Algorithms for Image Segmentation

Department of Theoretical Computer Science and Mathematical Logic

Supervisor of the bachelor thesis: Martin Pilát

Study programme: Computer Science

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date .....  .....

Author's signature

This thesis is dedicated to my family, my friends — especially Aziz, Carlos, and Yoshiki — and my professors Martin Pilat and Martin Koutecký, who supported me throughout my academic journey. Your support means a lot to me.

Computational resources were provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic.

Title: Nature Inspired Algorithms for Image Segmentation

Author: Seyed Danial Mohebbi Nowzar

Department: Department of Theoretical Computer Science and Mathematical Logic

Supervisor: Martin Pilát, Department of Theoretical Computer Science and Mathematical Logic

Abstract: Multilevel image thresholding is a classical image segmentation approach whose computational cost grows exponentially with the number of thresholds, making exhaustive search intractable and motivating the use of nature-inspired methods. Unlike deep learning methods, this approach requires no training data. This thesis compares Seven metaheuristic algorithms, including a proposed hybrid (WOA-ZOA) of the Whale Optimization Algorithm and the Zebra Optimization Algorithm, on a dataset mostly drawn from the BSDS500 dataset. All algorithms were run for a fixed number of iterations, and the results were analyzed using Friedman and Wilcoxon tests. The experiments show that the choice of fitness function has a greater impact than the choice of nature-inspired method: Kapur-SSIM performs best, followed by Kapur, while Kapur-Spatial performs worst. The proposed WOA-ZOA is highly competitive under Kapur and Kapur-SSIM.

Keywords: Nature-inspired methods, Image segmentation, Metaheuristic Algorithms

# Contents

# Introduction

Image segmentation sits at the core of computer vision: partitioning an image into coherent regions is a prerequisite for higher-level tasks such as object recognition and scene understanding. Because so much analysis depends on it, the problem has been studied across a wide range of applied domains, from autonomous vehicles [1] to intelligent medical technology [2, 3]. Deep learning has transformed computer vision [4] and can even rival human experts on segmentation tasks, but it comes with its own caveats: it depends on large quantities of manually annotated training data, and the models it produces are opaque, offering little insight into how a particular segmentation was arrived at. Classical thresholding methods such as Otsu's method avoid both problems, but exhaustive search over all threshold combinations grows exponentially in cost. These constraints motivate a different direction. Rather than learning from labeled examples, this thesis turns to nature-inspired optimization techniques, such as swarm intelligence [5], genetic algorithms [6], evolutionary strategies [7], and a novel hybrid of two swarm intelligence algorithms.

Image segmentation is the task of dividing an image into multiple regions, or so-called regions of interest (ROIs), that correspond to different objects in the image. These regions, according to human visual perception, are meaningful and non-overlapping. Image segmentation techniques can be broadly categorized into several approaches:

- Thresholding-based segmentation separates an image into regions by selecting one or more intensity thresholds. Pixels are classified based on whether their intensity values fall above or below the chosen threshold [8].
- Clustering algorithms group pixels into clusters based on similarity in intensity, color, or texture. Algorithms such as k-means [9, 10] partition the image into regions where pixels within the same cluster share similar characteristics.
- Edge-based segmentation identifies object boundaries by detecting discontinuities in image intensity. It relies on edge detection techniques to locate significant changes in pixel values, which correspond to the borders of different regions [11].
- Region-based segmentation groups pixels into regions by growing outward from seed points based on similarity criteria such as intensity or texture, merging or splitting regions until a homogeneity condition is satisfied [12].

The main contributions of this thesis are as follows:

- Proposing a novel hybrid optimization algorithm that combines the Whale Optimization Algorithm (WOA) [13] and the Zebra Optimization Algorithm (ZOA) [14].
- Applying the proposed method, alongside eight other classical and metaheuristic algorithms, to the problem of multilevel image segmentation.

- Proposing two novel objective functions – one incorporating spatial information, the other reconstruction quality – alongside standard Kapur entropy.
- Evaluating, within a single experimental framework, how much of the variation in segmentation quality is attributable to the choice of algorithm, to the choice of objective function, and to their interaction.

The remainder of this thesis is organized as follows:

- Chapter 1 provides the background and formal definitions of the methods used in the experiments and reviews related work in metaheuristic image thresholding and identifies the gaps this thesis addresses.
- Chapter 2 describes the proposed hybrid WOA-ZOA algorithm and the objective functions used in the experiments.
- Chapter 3 describes the experimental setup and the results of the conducted experiments.

# 1 Background and Related Work

This chapter covers the pre-existing nature-inspired algorithms and classical methods used in the experiments and provides intuition about how these algorithms differ.

## 1.1 Multilevel Thresholding as an Optimization Problem

An optimization problem is the task of finding the best solution from a set of possible solutions (the so-called search space) by maximizing or minimizing a given objective function. Thus, an optimization problem can be defined as:

$$\text{Find } X_{opt} \in \Omega \text{ Such that } F(X_{opt}) = \min_{X \in \Omega} F(X)$$

Where $X_{opt}$ denotes the optimal solution, $X$ denotes a candidate solution, $\Omega$ denotes the search space, and $F(X)$ is the objective function. The image segmentation problem is recontextualized as an optimization problem in the following subsections. Due to its non-linear search space, classical optimization methods may struggle to find the global optima and may get stuck in local optima. Thus, metaheuristic algorithms are used to explore the search space and obtain near optimal solutions.

In an optimization problem, finding a good solution is only half the challenge; the computational cost of obtaining it determines whether a method is practical at all. Exhaustive search over all threshold combinations, as in the classical multilevel extension of Otsu's method, is guaranteed to find the optimal thresholds under a given criterion, but its cost grows exponentially with the number of thresholds. Variations of Otsu's method have been proposed to overcome this limitation, though each comes with its own caveats: recursively splitting the image into two classes, for instance, reduces the cost dramatically, but each split is optimal only with respect to its local region rather than the global threshold configuration. Metaheuristic algorithms offer an alternative: heuristic-based optimization methods that search the solution space intelligently, trading guaranties of optimality for substantially lower computational costs.

Multilevel image segmentation, also known as $k$-level thresholding, is the process of dividing an image into $k$ distinct regions of interest based on the pixels' intensity values by finding $k-1$ thresholds. The objective is to find a set of optimal threshold values that divide the image into meaningful and disjoint regions.

### 1.1.1 Grayscale Image Segmentation

Let $I(x, y) \in \{0, 1, \ldots, L-1\}$ denote the intensity value of a pixel in a grayscale image, where $L$ denotes the number of possible intensity values, and $x, y$ denote the horizontal and vertical positions of a pixel, respectively. In $k$-level segmentation for grayscale images, we find a set of $k-1$ thresholds

$$t_1, t_2, \ldots, t_{k-1}.$$

Using these thresholds, we obtain regions

$$R_1, R_2, \ldots, R_k$$

such that

$$R_1 = [0, t_1], \quad R_2 = (t_1, t_2], \quad \ldots, \quad R_k = (t_{k-1}, L-1].$$

The thresholds must satisfy

$$0 < t_1 < t_2 < \cdots < t_{k-1} < L - 1.$$

The segmented image can then be represented as

$$S(x, y) = j \quad \text{if} \quad I(x, y) \in R_j, \quad j = 1, \ldots, k.$$

The main challenge is finding threshold values such that the segmented image does not lose important information due to incorrect thresholding, under-segmentation, or over-segmentation; the search space can be large.

### 1.1.2 Color Image Segmentation

Multilevel segmentation extends from grayscale to color images by treating a pixel's intensity as a higher-dimensional vector:

$$I(x, y) = [R(x, y), G(x, y), B(x, y)],$$

where $R(x, y)$, $G(x, y)$, and $B(x, y)$ are the red, green, and blue channel values, respectively. One approach is to split the image into three channels and compute threshold values independently for each channel before merging the results. Although the objective remains to partition the image into $k$ distinct regions, the increased dimensionality of pixel intensities raises the computational cost. This approach is used in the experiments in this thesis, applying independent threshold sets to each of the $R$, $G$, and $B$ channels.

## 1.2 Classical Segmentation Techniques

Now that we have defined the segmentation problem, we turn to classical image segmentation techniques, which aim to partition an image into meaningful and disjoint regions of interest based on intensity, color, and other factors. These methods do not require training and are usually efficient, requiring relatively few iterations; however, they may yield insufficient results.

### 1.2.1 Otsu's Method

The Otsu method [15] assumes that there are two classes: a background and an object. Suppose that the image is divided into two classes $C_1$ and $C_2$ based on a threshold $t$. The optimal threshold $t$ is found by maximizing the inter-class variance $\sigma_b^2(t)$:

$$\sigma_b^2(t) = P_1(t)P_2(t)\left[\mu_1(t) - \mu_2(t)\right]^2$$

Equivalently, minimizing the intra-class (within-class) variance $\sigma_w^2$

$$\sigma_w^2(t) = P_1(t)\sigma_1^2(t) + P_2(t)\sigma_2^2(t)$$

Where $P_1$ and $P_2$ are the probabilities of the two classes constructed from applying threshold $t$ to the image– $P_1 = \sum_{i=0}^{t} p_i \in [0, 1]$ and $P_2 = 1 - P_1$ and $p_i = \frac{n_i}{N}$ is the number of pixels at gray level $i$ over the total pixel counts $N$, and $\mu_1$ and $\mu_2$ are their respective mean intensities. These two are equivalent due to the total variance $\sigma_I^2$ being constant and equal to the sum of $\sigma_b^2$ and $\sigma_w^2$.

For K-level segmentation, there exists a recursive generalization of Otsu's criterion [16] instead of the multi-class generalization of Otsu's criterion [17] which serves to avoid exhaustive search, where the multi-class Otsu's criterion is:

$$\sigma_b^2(t_1, \ldots, t_{k-1}) = \sum_{j=1}^{k} P_j \left(\mu_j - \mu_T\right)^2$$

Here, $P_j$ is the probability of class $j$, and $\mu_j$ and $\mu_T$ are the mean intensities of class $j$ and the global intensity mean, respectively. A drawback of this approach is that the variance can be maximized locally at each recursive step. In this thesis, we iteratively select the largest region and apply binary Otsu to it until we get $k - 1$ thresholds.

### 1.2.2 K-means Clustering

The K-means clustering algorithm is a clustering-based segmentation that partitions the image into $k$ distinct clusters based on pixel intensities, where $k$ denotes the number of disjoint regions we desire. Thus, each pixel is assigned to the cluster that is nearest to its centroid, outputting regions of similar intensity. Let $\{x_1, x_2, \ldots, x_n\}$ denote the set of pixel values in the image. The K-means algorithm aims to minimize the objective function:

$$J(C) = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

where $C = \{C_1, C_2, \ldots, C_k\}$ is the partition of pixels into $k$ clusters, $C_i$ is the $i$-th cluster, and $\mu_i$ is the centroid of that cluster.
The algorithm operates iteratively through the following steps:

1. Initialize the centroids of the $k$ clusters.
2. Assign each pixel to the nearest cluster using a distance function (for example, the Euclidean distance).
3. Recompute the centroids as the mean of the pixels in their corresponding clusters.
4. Repeat steps 2-3 until convergence.

It may be used to segment grayscale images, and in the context of colored images, it treats a pixel's intensity as a vector of its red, green, and blue channel values. Although K-means clustering is simple, it is very sensitive to the initialization of centroids. It is always guaranteed to converge, but not necessarily to the optimal solution.

## 1.3 Metaheuristic Optimization

Unlike classical segmentation algorithms, the methods described in the remainder of this chapter approach segmentation as a global optimization problem solved via iterative search using nature-inspired optimization algorithms. The following principles form the foundation of nature-inspired optimization.

### 1.3.1 Exploration vs Exploitation

Metaheuristic optimization algorithms rely on two main mechanisms: exploration and exploitation. These two work together to effectively traverse the search space for a near-optimal solution [18].

On the one hand, exploration is the ability of an algorithm to investigate different regions of the search space. It helps find new and potentially better solutions while preventing the algorithm from getting stuck in a local optimum.

On the other hand, exploitation focuses on refining and improving the best solutions found so far by intensifying the search around promising areas in order to converge toward an optimal solution.

A crucial part of a well-performing metaheuristic algorithm is balancing exploitation and exploration. Excessive exploration may lead to slow convergence, while excessive exploitation may lead to premature convergence to suboptimal solutions.

In the context of image segmentation, exploration enables the algorithm to search for diverse threshold combinations, while exploitation refines these thresholds to achieve optimal segmentation performance.

### 1.3.2 The Role of the Fitness Function

While the previous section discussed how metaheuristic algorithms emphasize the importance of exploration and exploitation to traverse a search space, the search itself is only as meaningful as the objective function guiding it. The fitness function defines what "better" means for a given problem, and a poorly chosen fitness function can cause even a well designed algorithm to converge toward solutions that are not actually useful for the task at hand.

The design of an effective fitness function is widely recognized as problem-dependent rather than universal. Across different application domains, the criteria that constitute a "good" fitness function vary considerably, and different classes of problems depend on different considerations when formulating one [19]. This problem dependence extends beyond the fitness function itself: algorithmic suitability, more broadly, has been shown to be conditional on the specific characteristics of the problem being solved, rather than holding universally across all optimization tasks [20].

## 1.4 Nature-Inspired Algorithms

Nature-inspired optimization algorithms translate simple biological or physical metaphors into update rules that can efficiently search high-dimensional landscapes. One such subfamily of algorithms is the Swarm intelligence algorithms: Swarm

intelligence algorithms are population based meta-heuristic algorithms that consist of a group of search agents, updating their positions concerning one another and mimicking the social behavior of animals in nature.

### 1.4.1 Particle Swarm Optimization

Particle Swarm Optimization (PSO) [21, 22] is an optimization algorithm inspired by the behavior of flocks of birds and groups of fish. It is an algorithm designed for continuous optimization that utilizes a population of particles, where each particle is represented as two vectors in $\mathbf{R}^n$—one of which is its position $\vec{x}$, while the other is its velocity $\vec{v}$. Furthermore, each particle keeps track of the position in space where it achieved its best fitness value, denoted by $p_{best}$, and the position where the best fitness value was achieved for all entire population, denoted by $g_{best}$.

The entire PSO algorithm is thus very simple. It is based on the fact that each particle moves in the search space and is attracted to the places where it has found its best solution $p_{best}$ and where the best global solution found so far $g_{best}$ is. Initially, particle positions are initialized randomly in the search space, and then the algorithm iteratively updates the velocities and positions of all particles according to the following equations:

$$\vec{v} = w\vec{v} + \alpha_p r_p(p_{best} - x) + \alpha_b r_b(g_{best} - x)$$

$$\vec{x} = \vec{x} + \vec{v}$$

Where $r_p$,$r_b$ denote random numbers between 0 and 1, and $w$,$\alpha_p$,$\alpha_b$ are parameters that determine inertia and the extent to which an individual is attracted to its own and the globally best found solution so far. Thus, a particle exploits by moving towards the global best $g_{best}$ while exploring by getting pulled toward the local best $p_{best}$.

### 1.4.2 Gray Wolf Optimizer

Gray Wolf Optimizer (GWO) [23] is an optimization algorithm that mimics the hunting and social behavior of gray wolf packs in nature. It is used for continuous optimization problems and maintains a population of solutions commonly referred to as wolves. The population is divided into $\alpha$, $\beta$, $\delta$, and $\omega$, where $\alpha$, $\beta$, and $\delta$ denote the best, the second best, and the third best solutions found so far, while the rest of the wolves are $\omega$.

During the hunt, the positions of $(\alpha, \beta, \delta)$ are used to update the positions of the rest of the population by first computing the following:

$$\vec{D}_\alpha = |\vec{C}_1 \cdot \vec{X}^t_\alpha - \vec{X}|, \quad \vec{X}_1 = \vec{X}^t_\alpha - \vec{A}_1 \cdot \vec{D}_\alpha$$

$$\vec{D}_\beta = |\vec{C}_2 \cdot \vec{X}^t_\beta - \vec{X}|, \quad \vec{X}_2 = \vec{X}^t_\beta - \vec{A}_2 \cdot \vec{D}_\beta$$

$$\vec{D}_\delta = |\vec{C}_3 \cdot \vec{X}^t_\delta - \vec{X}|, \quad \vec{X}_3 = \vec{X}^t_\delta - \vec{A}_3 \cdot \vec{D}_\delta$$

The population's position is then updated using the following equation:

$$\vec{X}^{t+1} = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3}$$

Where $\vec{D}_\alpha, \vec{D}_\beta, \vec{D}_\delta$ denotes the distance of $\alpha, \beta$, and $\delta$ from the prey position—the optimal solution position, respectively; $\vec{X}_\alpha^t, \vec{X}_\beta^t, \vec{X}_\delta^t$ are the positions of $\alpha, \beta$, and $\delta$ at time $t$, respectively; $\vec{X}_1, \vec{X}_2, \vec{X}_3$ are the updated positions derived from $\alpha, \beta$, and $\delta$, respectively; and $\vec{A}, \vec{C}$ are coefficient vectors used to balance exploration and exploitation, updated via:

$$\vec{A} = 2a \cdot \vec{r}_1 - a$$

$$\vec{C} = 2 \cdot \vec{r}_2$$

where

$$a = 2 - 2 \cdot \frac{t}{T}$$

decreases linearly from 2 to 0 over the course of the search, as a means to balance exploration and exploitation, and $\vec{r}_1, \vec{r}_2$ are random vectors with components drawn uniformly from $[0, 1]$. $\vec{C}$ additionally introduces a stochastic component to the search. $T$ is the total number of epochs. This algorithm is very effective in exploiting a solution due to the way it updates the population as the sum of the updated position of $(\alpha, \beta, \delta)$.

### 1.4.3 Zebra Optimization Algorithm

Zebras have a number of social behaviors in nature, two of which are foraging and defense strategies against predators. The Zebra Optimization Algorithm (ZOA) [14] is based on these two mechanisms. Each candidate solution is represented by a zebra, whose position is updated based on its herd's leader decisions and its interactions with predators. These two behaviors construct two phases of the algorithm, respectively:

#### Phase 1: Foraging behavior of the algorithm

A specific kind of zebra is the pioneer zebra, which feeds on higher and less nutritious grass to create an environment with shorter grass for other species. Therefore, other zebras in the herd move in the environment under the guidance of this pioneer zebra. In the context of optimization problems, the best individual within the population is called the pioneer zebra, which guides the rest of the individuals towards its location in the search space. This is done via the following:

$$x_{i,j}^{(t+1)} = x_{i,j}^t + r.(PZ_j - I.x_{i,j}^t)$$

$$I = \text{round}(1 + \text{rand})$$

$$X_i^{(t+1)} = \begin{cases} X_i^{(t+1)}, & \text{if } F_i^{(t+1)} < F_i^t \\ X_i^t, & \text{otherwise.} \end{cases}$$

Where $X_i^{(t+1)}$ is the new state of the $i$-th zebra after the first phase, $x_{i,j}^{(t+1)}$ is the value of the $j$-th dimension for the $i$-th zebra, $r$ and rand are random numbers in the interval $[0, 1]$, $PZ_j$ is the $j$-th dimension of the pioneer zebra(best solution), and $I \in \{1, 2\}$. If $I = 2$, this signals larger changes in the population's movement.

**Phase 2: Defending against predators**

Zebras have many predators: lions, cheetahs, leopards, wild dogs, and more. When the zebras are attacked or threatened by predators, their defensive strategy differs based on the predator. When attacked by lions or big predators, the zebra escapes in a zigzag pattern and makes random movements, while zebras are much more aggressive if smaller predators are attacking. In the former ($S_1$), the zebra selects an escape route in response to a predator's attack. In the latter strategy ($S_2$), if a smaller predator is attacking, they will instead scare it away. The movements of zebras in the search space during this phase can be described by the following formulas:

$$x_{i,j}^{(t+1)} = \begin{cases} S_1 : x_{i,j}^t + R.(2r-1)(1-\frac{t}{t_{max}}) & \text{if } P_s \leq 0.5, \\ S_2 : x_{i,j}^t + r.(AZ_j - I.x_{i,j}^t) & \text{otherwise.} \end{cases}$$

$$X_i^{t+1} = \begin{cases} X_i^{t+1} & \text{if } F_i^{(t+1)} < F_i^t \\ X_i^t & \text{otherwise,} \end{cases}$$

Where $X_i^{(t+1)}$ is the new state of the $i$-th zebra after the second phase, $R$ is a constant value equal to 0.01, $P_s$ is the probability of selecting one of these two strategies, $(S_1, S_2)$ and $r$ which is randomly generated from the interval $[0, 1]$, and $AZ_j$ is the $j$-th dimension of the attacked zebra. $t_{max}$ is the maximum number of epochs.

Phase 1 introduces the exploitation part of the algorithm, while phase 2 introduces $S_1$, which is focused on exploration and $S_2$, which focuses on local exploitation. Additionally, $I, r$, rand, and $P_s$ add stochastic components to the algorithm, which will help avoid local optima.

### 1.4.4 Whale Optimization Algorithm

Whales typically live in groups. A distinctive behavior observed in humpback whales is bubble-net hunting, which involves two commonly reported maneuvers: (i) a dive to approximately 12 m followed by the creation of a spiral bubble curtain around the prey, and (ii) a three-stage sequence consisting of the coral loop, lobtail, and capture loop. Detailed descriptions of these behaviors can be found in [24]. The whale optimization algorithm (WOA) is a nature-inspired metaheuristic algorithm based on the bubble-net hunting strategy of humpback whales. It is designed for continuous optimization problems. WOA models prey encircling and bubble-net attacking through the following mechanism:

**Encircling prey (exploitation)**

Since we don't know the prey's true location (optimal solution), whales assume that the current best solution is the prey and update their positions relative to it via:

$$\vec{D} = |\vec{C}.\vec{X}_{opt} - \vec{X}|$$

$$\vec{X}^{t+1} = \vec{X}_{opt} - \vec{A}.\vec{D}$$

Where $\vec{X}_{opt}$ is the current best found solution, $\vec{X}$ is the current position, and $\vec{A}, \vec{C}$ are coefficient vectors. $\vec{A}, \vec{C}$ is defined as:

$$\vec{A} = 2a.\vec{r}_1 - a$$

$$\vec{C} = 2.\vec{r}_2$$

Where $a$ decreases linearly from 2 to 0, and $\vec{r}_1, \vec{r}_2$ are random vectors in the interval $[0, 1]$. Notice when $|\vec{A}| < 1$ whales move toward the best solution $\vec{X}_{opt}$, thus exploiting; otherwise, they explore.

**Bubble-net attacking method**

The spiral updating position is mathematically modeled as follows:

$$\vec{X}^{t+1} = \vec{D}.e^{bl}.cos(2\pi l) + \vec{X}_{opt}$$

Where $\vec{D}$ is the distance from the prey, $b$ is a constant defining the spiral shape, and $l$ is a random number between -1 and 1. Combining this with the shrinking encircling behavior described earlier, we get the following:

$$\vec{X}^{t+1} = \begin{cases} \vec{X}_{opt} - \vec{A}.\vec{D} & \text{if } p < 0.5 \\ \vec{D}.e^{bl}.cos(2\pi l) + \vec{X}_{opt} & \text{otherwise} \end{cases}$$

Where $p \in [0, 1]$ is the random probability of choosing one of these two updating position methods.

**Search for prey (exploration)**

When $|A| \geq 1$, whales update their position with respect to a random whale, as modeled below:

$$\vec{D} = |\vec{C}.\vec{X}_{rand} - \vec{X}|$$

$$\vec{X}^{t+1} = \vec{X}_{rand} - \vec{A}.\vec{D}$$

Thus, it promotes exploration and avoids premature convergence to a local optimum up to a degree.

### 1.4.5 Other nature-inspired algorithms

There are other categories under the nature-inspired optimization algorithms, such as Evolutionary Strategies and Evolutionary Algorithms, of which we will cover one each in this chapter.

### Genetic Algorithm

A classic evolutionary algorithm is the simple genetic algorithm. In this algorithm, individuals are coded in a specific way, such as sequences of bits of fixed length or sequences of real numbers. The fitness function always depends on the particular problem to be solved. At the start of the algorithm, individuals of the population are initialized randomly, and their fitness is calculated. The first population consists of these individuals. Subsequently, the parents to which the genetic operators will be applied are selected. The selection is done in such a way that the probability of selecting a given individual is proportional to its fitness, a method known as roulette-wheel selection. Selection is done with repetition, meaning an individual may be selected multiple times. Genetic operators, such as crossovers and mutations, are then applied to these individuals. Crossover combines two solutions to create a new solution. An example would be the single point crossover, where a point is selected randomly, and one part of the new solution is copied from one parent before this point and the part after this point from the other parent. Then, mutation is applied, which can vary, for example, by flipping a bit. The goal of mutation is to improve the diversity of the population, so it serves as a means of exploration. Then the newly selected offspring replace the original parents, and the algorithm continues to the next generation.

### Covariance Matrix Adaptation Evolution Strategy

The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [25] is a stochastic, derivative-free optimization algorithm for continuous, nonlinear, and nonconvex optimization problems.

A population of $\lambda$ candidate solutions is drawn at each generation as follows:

$$\vec{X}_k^t = \vec{m}^t + \sigma^t \cdot \mathcal{N}(0, C^t), \quad k = 1, \ldots, \lambda$$

where $\vec{m}^t$ is the mean of the current best estimate of the optimal solution, $\sigma^t$ being the step size, and $C^t$ denotes the covariance matrix. The exploration of the algorithm is led by a large $\sigma^t$ and the covariance matrix $C^t$.

After selecting the best $\mu$ individuals, they are utilized to update the mean, pulling the search toward richer regions of the search space:

$$\vec{m}^{t+1} = \sum_{i=1}^{\mu} w_i \vec{x}_i^t \quad (*)$$

where $w_i$ are positive weights, such that the better ranked solutions receive a higher influence on the updated mean.

Additionally, The covariance matrix is adapted:

$$C^{t+1} = (1 - c_c)C^t + c_c \cdot p_c p_c^T$$

followed by the step size being adjusted to balance the search:

$$\sigma^{t+1} = \sigma^t \cdot \exp\left(\frac{c_\sigma}{d_\sigma}\left(\frac{\|p_\sigma\|}{E\|\mathcal{N}(0, I)\|} - 1\right)\right)$$

where $c_c$ and $c_\sigma$ are learning rates that control the adaptation speeds of the covariance matrix and step size, respectively, and $p_c$ and $p_\sigma$ are evolution paths that carry information about the search trajectory across generations.

Exploration is thus driven by $\sigma^t$, the covariance matrix $C^t$, and random sampling from the Gaussian distribution, while the mean update in Equation (*) drives the search toward the best solutions found so far.

## 1.5 Related Work and Research Gaps

Several studies have applied metaheuristic algorithms to multilevel thresholding. In [26], The Multi-Verse Optimizer (MVO) [27] and the Ant Lion Optimizer (ALO) [28] were evaluated against other evolutionary algorithms using metrics such as FSIM, MSE, SSIM, and PSNR; notably, they employed Kapur entropy as the sole fitness function and restricted their experiments to low threshold counts ($k$ from 2 to 5). In [29], multiple fitness functions were compared across several evolutionary algorithms, though nearly all of the objectives considered were entropy-based.

Beyond applying individual metaheuristics, a further line of research hybridizes two algorithms in the hope of combining their complementary strengths. Evidence that such hybridization can outperform either parent alone is provided by [30], which combines the Gray Wolf Optimizer (GWO) with Sand Cat Swarm Optimization (SCSO) [31], using Otsu's criterion and Kapur entropy as fitness functions.

Several hybrids of WOA have been specifically proposed, each targeting its tendency to perform poorly during exploitation and stagnate at a local best solution. Combining WOA with the Gray Wolf Optimizer [32] addresses this directly, while embedding WOA within Simulated Annealing [33] pursues the same goal through a different mechanism. A further combination pairs WOA with PSO, using PSO to drive exploitation and WOA to drive exploration [34]. A common thread across these approaches is the use of a second algorithm, or a component of one, specifically to help the search escape local optima.

The Zebra Optimization Algorithm, being one of the more recently proposed metaheuristics, likewise has considerable room for further hybridization. A recent hybrid of GWO and ZOA [35] uses ZOA to strengthen GWO's exploration ability. Another recent hybrid, combining the Jellyfish Optimization Algorithm with ZOA [36], has been applied to virtual machine allocation and management in order to reduce the number of active server machines. To the best of the author's knowledge, however, no existing work hybridizes WOA and ZOA specifically; this thesis contributes to the exploration of ZOA-based hybrids by proposing exactly such a combination, aiming to encourage additional exploration through ZOA's second phase rather than through a separate anti-stagnation mechanism.

Two further gaps emerge from this body of work. First, the objective functions employed are almost exclusively entropy-based: neither reconstruction quality nor spatial information is incorporated into the search criterion itself. Spatial extensions of Kapur entropy do exist, but evaluating a candidate solution under these formulations is typically exponential in cost, requiring techniques such as dynamic programming to remain even partially viable [37]. Second, existing comparative studies typically fix a single objective function and vary only the search algorithm, leaving the relative contribution of objective function choice

versus algorithm choice largely unexamined.

This thesis addresses both gaps. It proposes a computationally lighter extension of Kapur entropy that incorporates spatial information, alongside a second extension that incorporates reconstruction quality directly into the objective. It then evaluates the following set of algorithms within a single experimental framework: four popular swarm intelligence algorithms (PSO, GWO, WOA, and ZOA), two evolutionary methods (GA and CMA-ES), and two classical baselines (Otsu's method and k-means clustering), alongside the proposed WOA-ZOA hybrid. This design permits three comparisons that prior work leaves unexamined: how population-based methods fare against one another under a fixed evaluation budget, whether the proposed hybrid improves upon its parent algorithms and the broader swarm intelligence family, and how much of the observed variation in segmentation quality is attributable to the choice of algorithm versus the choice of objective function. Furthermore, whereas the studies discussed above restrict their experiments to $k \leq 5$, the evaluation here extends to higher values of $k$ (6, 8, and 10), where the search space grows substantially harder.

# 2 Proposed Methodology

Now that we have discussed the pre-existing nature-inspired algorithms and classical methods used for image segmentation and the the formulation of image segmentation as an optimization problem. we introduce the objective functions and the hybridization of WOA and ZOA.

## 2.1 Objective Function

The most important component of any nature-inspired optimization algorithm is the fitness function, denoted by $F$, which is defined by the objective function. It allows us to evaluate candidate solutions and compare them.

### 2.1.1 Kapur Entropy

As the name implies, this objective function (and those that follow) uses entropy, which in information theory measures homogeneity and redundancy in data.

Kapur entropy [38] is a histogram-based criterion for multilevel thresholding that maximizes the sum of entropies across segmented regions. Unlike Otsu's method, which maximizes between-class variance, Kapur entropy treats each segmented region's histogram as its own probability distribution and rewards thresholds that maximize information.

Given a candidate solution $\vec{X} = \{t_1, \ldots, t_{k-1}\}$, we divide the image into $k$ disjoint regions $R_1, \ldots, R_k$ as defined previously. Let $p_j$ denote the proportion of pixels at gray level $j$ to total pixel counts $N$, with $\sum_j p_j = 1$. Let

$$\omega_i = \sum_{j \in R_i} p_j$$

denote the total probability that a uniformly drawn pixel belongs to segment $i$ – the fraction of image pixels that fall into segment $i$ . The entropy of segment $i$ is then

$$H_i = -\sum_{j \in R_i} \frac{p_j}{\omega_i} \ln \frac{p_j}{\omega_i}.$$

To avoid division by zero, we skip segments with $\omega_i \leq \delta$, where $\delta = 10^{-10}$. The total Kapur entropy is the sum of entropies over the $k$ regions:

$$H_{kapur}(\vec{X}) = \sum_{i=1}^{k} H_i.$$

For color images, Kapur entropy is applied independently to each color channel (red, green, and blue) and then summed to obtain a single fitness value:

$$H^{\text{color}}_{kapur}(\vec{X}) = \sum_{c \in \{r,g,b\}} H_{kapur}(\vec{X}_c),$$

where $\vec{X}_c$ are the thresholds belonging to channel $c$.

The optimal solution $\vec{X}_{opt}$ maximizes the following quantity:

$$\vec{X}_{opt} \leftarrow \begin{cases} \arg\max_{\vec{X}} H_{kapur}(\vec{X}) & \text{if the image is grayscale,} \\ \arg\max_{\vec{X}} H_{kapur}^{\text{color}}(\vec{X}) & \text{otherwise.} \end{cases}$$

In our experiments, each metaheuristic minimizes the negative of the corresponding objective (i.e., $-H_{kapur}(\vec{X})$ or $-H_{kapur}^{\text{color}}(\vec{X})$), since maximizing entropy is equivalent to minimizing its negation.
A known limitation of Kapur entropy is that it is computed purely from the image histogram, disregarding spatial information. Two images with identical histograms but different spatial structures may therefore receive the same Kapur entropy score.

### 2.1.2 Spatial Kapur Entropy

To circumvent this caveat of Kapur's entropy, we attempt to penalize the standard Kapur entropy according to the degree of spatial incoherence in the segmented image. Given a candidate solution $\vec{X}$ and the segmented image $S$, obtained by replacing each pixel with the mean intensity of its assigned segment, the spatial penalty is defined as the mean absolute difference between adjacent pixels in $S$:

$$M(\vec{X}) = \frac{1}{2}\left(\begin{array}{l} \dfrac{1}{N_1}\displaystyle\sum_{x=1}^{W-1}\sum_{y=1}^{H}|S(x,y)-S(x+1,y)| \\ +\dfrac{1}{N_2}\displaystyle\sum_{x=1}^{W}\sum_{y=1}^{H-1}|S(x,y)-S(x,y+1)| \end{array}\right)$$

where $W$ and $H$ denote the image width and height, respectively, and $N_1 = (W-1)\times H$ and $N_2 = W \times (H-1)$ represent the number of horizontal and vertical pixel pairs.

A spatially coherent segmentation is one in which each region forms a contiguous collection of pixels. Consequently, $M(\vec{X})$ produces mostly zero-valued differences except at region boundaries, resulting in a low value of $M(\vec{X})$ for coherent segmentations. The proposed objective function is therefore defined as

$$H_{SK}(\vec{X}) = \frac{H_{kapur}(\vec{X})}{1 + M(\vec{X})}.$$

Where $H_{kapur}(\vec{X})$ is the Kapur entropy defined in the previous section. The denominator penalizes threshold choices even if the entropy is high. The constant 1 is to avoid division by 0, and $M(\vec{X})$, when low, increases $H_{SK}(\vec{X})$.
For color images, $H_{SK}$ is applied to each color channel: red, green, and blue, independently and calculated as follows:

$$H_{SK}^{color}(\vec{X}) = \sum_{c\in\{r,g,b\}} H_{SK}(\vec{X}_c)$$

where $\vec{X}_c$ denotes the thresholds that belong to channel c. As a result, the total color image fitness value is defined as

$$H_{SK}^{color}(\vec{X}) = \frac{H_{kapur}^{color}(\vec{X})}{1 + M^{color}(\vec{X})}.$$

The optimal solution $\vec{X}_{opt}$ will be one that maximizes the following quantity:

$$\vec{X}_{opt} = \begin{cases} \arg\max_{\vec{X}} H_{SK}(\vec{X}) & \text{if image is gray} \\ \arg\max_{\vec{X}} H_{SK}^{color}(\vec{X}) & \text{otherwise} \end{cases}$$

In our experiments, each metaheuristic minimizes the negative of the corresponding objective (i.e., $-H_{SK}(\vec{X})$ for grayscale images and $-H_{SK}^{color}(\vec{X})$ for color images), since maximizing the objective is equivalent to minimizing its negation.

### 2.1.3 Kapur-SSIM Mixed Objective

Both of the previous objective functions optimize criteria derived from the segmented image without explicitly measuring how faithfully they reconstruct the original image. To address this, a third objective function is introduced that directly incorporates a reconstruction quality metric called SSIM [39] which is further described in section 3.1
For a candidate solution $\vec{X}$, the mixed objective is defined as a weighted combination of normalized Kapur entropy and SSIM:

$$H_{mixed}(\vec{X}) = (1-\lambda)\, H_{kapur}^{n}(\vec{X}) + \lambda\, SSIM(I, S(\vec{X}))$$

Where $S(\vec{X})$ denotes the segmented image obtained using $\vec{X}$, and $\lambda \in [0, 1]$ is a weighting parameter.

Combining two quantities into a single weighted sum is only meaningful if the weights actually reflect the intended balance between them. SSIM is bounded by construction to $[-1, 1]$, but Kapur entropy $H_{kapur}$ has no such fixed range – its scale depends on $k$ and on the image's own histogram, and is typically much larger in magnitude than SSIM. Without normalization, a fixed weight such as $\lambda = 0.5$ would not actually divide influence evenly between the two terms: the raw magnitude of $H_{kapur}$ would dominate the sum almost entirely, regardless of the value chosen for $\lambda$, since $\lambda$ only controls the proportion of two terms that are not on comparable scales to begin with. To make $\lambda$ meaningful, both terms must first be brought onto a comparable numerical range. $H_{kapur}^{n}$ is a normalized version of $H_{kapur}$, scaled to be compatible with SSIM:

$$H_{\text{kapur}}^{n} = \frac{H_{\text{kapur}}}{k \ln(L)}$$

where $k$ is the number of segmented regions and $L = 256$ is the number of intensity levels. The denominator $k \ln(L)$ is derived from the maximum possible value that Kapur entropy can attain: by Shannon entropy's [40] known maximum, a single segment's entropy is maximized when its pixels are distributed perfectly uniformly across all $L$ intensity levels, giving an upper bound of $\ln(L)$ for that one segment. Summing this bound across all $k$ segments gives $k \ln(L)$ as the maximum achievable total Kapur entropy. Dividing by this bound rescales $H_{kapur}$ into an approximately $[0, 1]$ range, directly comparable to SSIM, without introducing any additional tunable parameter since the bound is derived from $k$ and $L$, both already fixed by the problem itself. The bound is described as approximate rather than exact because segments are constrained to contiguous, non-overlapping intensity

ranges rather than being free to span the full histogram independently, so the true achievable maximum sits somewhat below $k \ln(L)$ in practice; this does not affect the normalization's purpose, however, since only comparable scale, not an exact match to 1.0, is required for $\lambda$ to meaningfully balance the two terms.

The blending weight is fixed at $\lambda = 0.5$ for both grayscale and color images, giving equal importance to entropy and reconstruction quality. This value is not tuned per algorithm. Consistent with the experimental design. For color images, both terms are computed independently across the red, green, and blue channels and are calculated as follows:

$$H^{color}_{mixed}(\vec{X}) = \sum_{c \in \{r,g,b\}} \left[ (1-\lambda)\, H^{n}_{kapur}(\vec{X}_c) + \lambda\, SSIM(I_c, S_c(\vec{X}_c)) \right]$$

where $I_c$ and $S_c$ denote the original and segmented images for channel $c$, respectively, and $\vec{X}_c$ denotes the thresholds belonging to the color channel $c$.
The optimal solution $\vec{X}_{opt}$ will be one that maximizes the following quantity:

$$\vec{X}_{opt} = \begin{cases} \arg\max_{\vec{X}} H_{mixed}(\vec{X}) & \text{if image is gray} \\ \arg\max_{\vec{X}} H^{color}_{mixed}(\vec{X}) & \text{otherwise} \end{cases}$$

Similar to the other two fitness functions, the metaheuristic algorithm's fitness is defined as the negation of $H_{mixed}$ or $H^{color}_{mixed}$.

## 2.2 WOA-ZOA Hybrid

Both Whale Optimization Algorithm and the Zebra Optimization Algorithm are effective population based metaheuristic algorithms that have shown strong performance when solving continuous optimization problems. However, each algorithm has its own weaknesses when applied separately.

WOA heavily utilizes the current global best solution to guide its population of candidate solutions. The two mechanisms of WOA, encircling and the spiral bubble net attack, each update an agent's position relative to $\vec{X}_{opt}$, the best current solution found so far. As a result, WOA becomes highly efficient at exploiting a known promising region but is very susceptible to premature convergence, which is the problem of the population collapsing around a local optimum early in the search. WOA's exploitation-heavy update rules provide a very limited ability to escape the local optimum. This weakness is particularly relevant in the multilevel thresholding context, where the Kapur entropy landscape is multimodal, and the number of local optima grows with $k$.

On the other hand, ZOA employs a foraging phase that drives agents away from the current best solution found so far $\vec{X}_{opt}$ by a random perturbation term, and a defense phase that introduces stochastic escape behavior to the algorithm. On one hand, these employed strategies promote population diversity and exploration of the search space. On the other hand, ZOA's convergence to the global optimum is much slower than WOA's due to its weaker exploitation of known good regions.

The proposed WOA-ZOA hybrid addresses both limitations simultaneously. To the best of the author's knowledge, this specific combination has yet to be explored in the literature. The population's need to cover the threshold search space broadly is achieved via ZOA's exploration mechanism early in the search

and transitioning to WOA's exploration mechanism late in the search to converge toward the best threshold set found. The proposed hybrid is designed to achieve the best of both worlds: the diversity of ZOA and the convergence speed of WOA.

### 2.2.1 Phase-Switching Mechansim

How to govern the transition between the ZOA's exploration phase and WOA's exploitation phase over the course of the optimization is the central design decision in WOA-ZOA. A fixed boundary, such as switching after a fixed number of epochs, would be unflexible and would introduce an additional hyperparameter. Instead, a probabilistic phase switching mechanism is introduced:

$$p(t) = (1 - \frac{t}{T})^{\alpha} \quad (2.1)$$

where $t$ is the current epoch, T is the total number of epochs, $\alpha > 0$ is a parameter that controls the speed of the transition. At every epoch, every agent in the population independently draws a random number $u \sim \mathcal{U}(0, 1)$: if $u < p(t)$, the agent utilizes ZOA exploration. Otherwise, it employs WOA's exploitation.

The behavior of $p(t)$ over the course of the search is illustrated in figure 2.1. At $t = 0$, $p(0) = 1$, every agent performs ZOA's exploration, which is fine as we wish to maximize the population's diversity at the start of the search. At $t = T$, $p(T) = 0$, every agent performs WOA's exploitation, maximizing convergence to the optimal solution. As for the intermediate epochs, $p(t)$ decays smoothly. As a result, a gradual and continuous transition instead of a sudden phase boundary shift occurs. The parameter $\alpha$ controls how quickly this decay actually happens:

- $\alpha < 1$: fast initial decay – the algorithm starts the exploitation strategy of WOA early. This is good for problems where the promising region can be found quickly.
- $\alpha = 1$: linear decay – equal transition speed throughout the course of the search.
- $\alpha > 1$: slow initial decay – ZOA exploration is prolonged before being followed by WOA's exploitation. This is beneficial for complex multimodal problems where a broad search is more important early on.

For the problem of multilevel thresholding, $\alpha = 2$ is used for all experiments, producing a concave decay that extends the ZOA exploration phase beyond the midpoint. This is motivated by the multimodal nature of Kapur entropy at high $k$ values, as mentioned. Since more segmented regions will cause the threshold search space to contain more local optima, early exploration allows us to avoid falling into these local optima and reduces the risk of premature convergence.

### 2.2.2 Position Update Rules

#### ZOA Exploration Phase

When the drawn $u \sim \mathcal{U}(0, 1) < p(t)$ agent $i$ updates its position using ZOA's exploration rule, there are two strategies $S_1$ and $S_2$ that a zebra may take. In

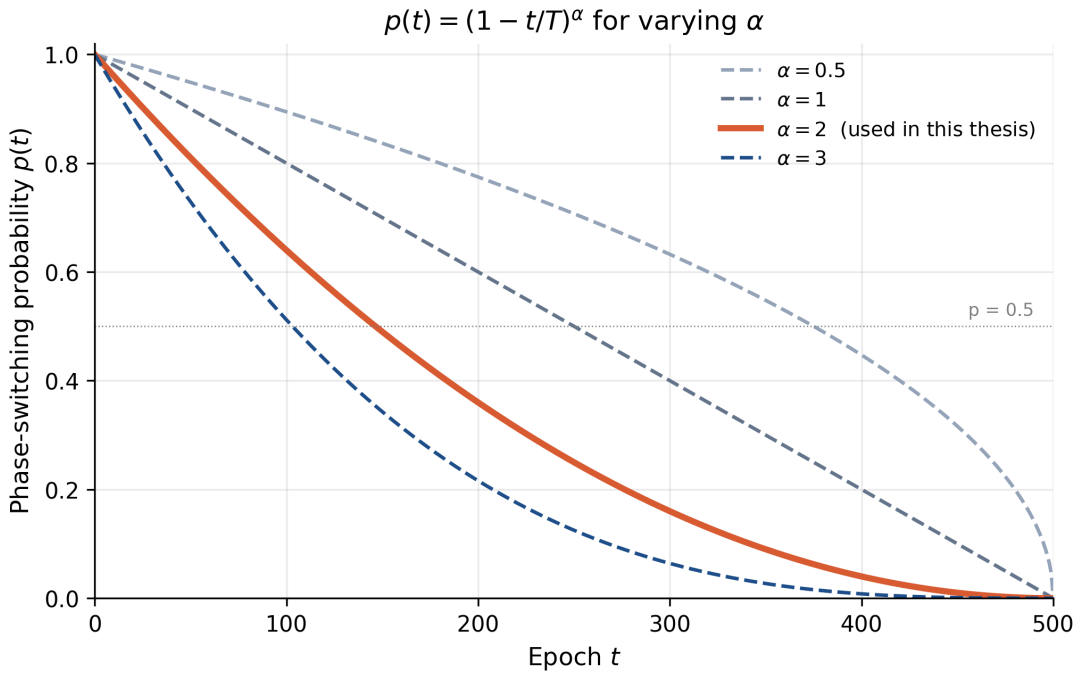



**Figure 2.1** Phase switching probability $p(t) = (1 - t/T)^{\alpha}$ for $\alpha \in \{0.5, 1, 2, 3\}$. Higher $\alpha$ prolongs ZOA exploration before WOA exploitation begins. This thesis uses $\alpha = 2$.

the former, the zebra selects an escape route in response to a predator's attack. While in $S_2$, they scare the small predator away. These movements in the search space are mathematized as follows:

$$x_{i,j}^{(t+1)} = \begin{cases} S_1 : x_{i,j}^t + R.(2r-1)(1 - \frac{t}{T}) & \text{if } P_s \leq 0.5, \\ S_2 : x_{i,j}^t + r.(AZ_j - I.x_{i,j}^t) & \text{otherwise.} \end{cases} \quad (2.11)$$

Where $x_{i,j}^{(t+1)}, x_{i,j}^t$ is the $j$-th dimension of the $i$-th at epoch $t+1$ and $t$. $R$ is a constant of 0.01. $P_s$ is the probability of selecting the two strategies. $AZ_j$ is the $j$-th dimension of the attacked zebra, which is a random zebra. $r$ is a random vector and $I = \text{round}(1 + rand) \in 1, 2$. $T$ is the maximum number of epochs.

### WOA Exploitation Phase

When $u \geq p(t)$, agent $i$ updates its position via WOA's exploitation mechanism. A shrinking scalar is first computed:

$$a = 2 - 2 \cdot \frac{t}{T} \quad (2.12)$$

where $t$ is the current epoch, and $T$ is the total number of epochs. $a$ decreases linearly from 2 to 0 over the course of the search. The coefficient vectors are then computed as follows:

$$\vec{A} = 2a \cdot \vec{r_1} - a \quad (2.13)$$

$$\vec{C} = 2 \cdot \vec{r_2} \quad (2.14)$$

where $\vec{r_1}$,$\vec{r_2} \sim \mathcal{U}(0, 1)$ are random vectors. A random number $q \sim \mathcal{U}(0, 1)$ is drawn to determine which WOA's exploitation strategy is to be used. When $q < 0.5$, we

utilize the Shrinking encircling:

$$\vec{X}_i^{t+1} = \vec{X}_{opt} - \vec{A} \cdot |\vec{C} \cdot \vec{X}_{opt} - \vec{X}_i^t| \quad (2.15)$$

Agent $i$ is pulled toward the global best solution found so far, scaled by $\vec{A}$. When $|\vec{A}| < 1$, the local search is intensified around the best-known solution.

On the other hand, when $q \geq 0.5$ we utilize the Spiral Bubble net:

$$\vec{X}_i^{t+1} = |\vec{X}_{opt} - \vec{X}_i^t| \cdot e^{bl} \cdot \cos(2\pi l) + \vec{X}_{opt} \quad (2.16)$$

where $b = 1$ defines the logarithmic spiral shape, $l \sim \mathcal{U}(-1, 1)$. As a result, the agent approaches $\vec{X}_{opt}$ via a spiral path, providing a different geometry than the direct encircling method. A consequence of this is the prevention of all agents from collapsing onto the same position.

### 2.2.3 Greedy Selection

Following the computation of the new position, the position is clipped to the valid threshold bounds via the `correct_solution` method, and the new agent's fitness is computed. Greedy selection is then used to determine whether the new agent will replace agent $i$ in the population as follows:

$$\vec{X}_i^{t+1} = \begin{cases} \vec{X}_i^{t+1} & \text{if } F(\vec{X}_i^{t+1}) \text{ is better than } F(\vec{X}_i^t) \\ \vec{X}_i^t, & \text{otherwise} \end{cases} \quad (2.17)$$

where $F(\cdot)$ is the fitness function used. This ensures the population never degrades; each agent either improves or remains the same. An overview of the full WOA-ZOA workflow is shown in Fig. 2.2.

### 2.2.4 Complexity Analysis

As for the time complexity of WOA-ZOA per epoch, it is $O(N \cdot d)$, where $N$ is the population size and $d$ is the solution dimension ($k - 1$ for grayscale, $3(k - 1)$ for color). This is consistent with both $WOA$ and $ZOA$, since the phase switching mechanism only adds a constant time probability evaluation per agent and doesn't introduce any additional population level operation. Furthermore, the fitness evaluation function cost per epoch is $O(N \cdot C_f)$, where $C_f$ is the call cost of the chosen objective function $f$. Considering $T$ epochs, the total time complexity would add up to $O(T \cdot N \cdot (d + C_f))$, which is identical to other metaheuristic algorithms, ensuring a fair computational budget.

### 2.2.5 Algorithm

Fig. 2.2 provides an overview of the proposed WOA-ZOA hybrid. The procedure starts by sampling an initial population within the search bounds and evaluating the objective function to determine the best-so-far solution. In each epoch, the method computes the control parameter $a$ and switching probability $p(t)$; then each agent either performs a ZOA-style exploration update or a WOA-style exploitation update (shrinking, encircling or spiral motion). The candidate solution is clipped to the feasible range and accepted

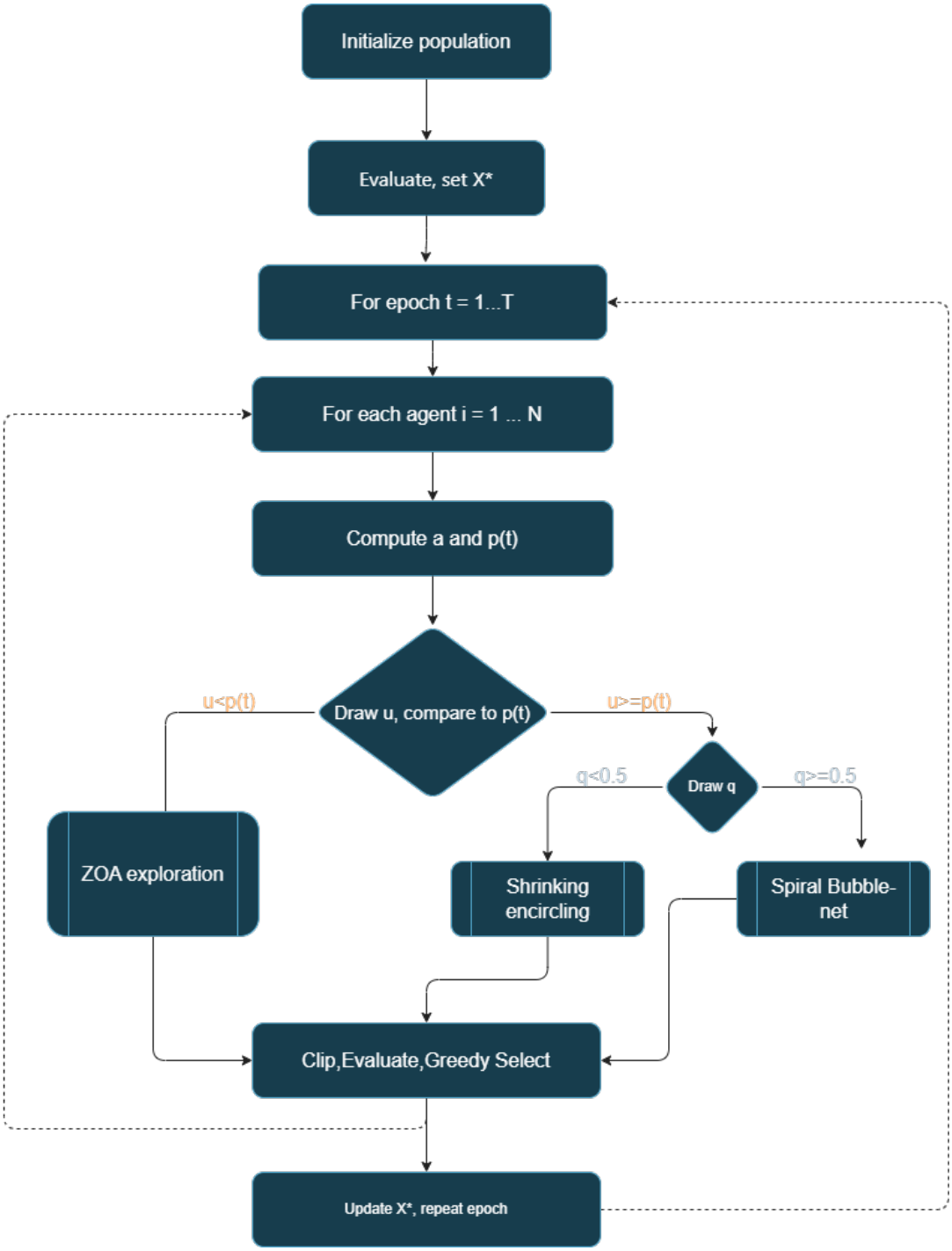



**Figure 2.2** WOA-ZOA hybrid workflow (overview of the algorithm phases and information flow).

only if it improves fitness (greedy selection), while the global best is updated throughout the run. Algorithm 1 gives the corresponding pseudocode.

**Algorithm 1** WOA-ZOA Hybrid Algorithm

**Require:** Objective function $F$, population size $N$, dimensions $d$, bounds $[lb, ub]$, maximum number of epochs $T$, $\alpha$

**Ensure:** Best solution $\vec{X}^*$ and best fitness $F^*$

Initialize population $\vec{X}_i \sim \mathcal{U}(lb, ub)$ for $i = 1, \ldots, N$
Evaluate $F(\vec{X}_i)$ for all $i$; set $\vec{X}^* \leftarrow \arg\min_i F(\vec{X}_i)$
**for** $t = 1, 2, \ldots, T$ **do**
    Compute $a$ via the equation (2.12)
    Compute $p$ via the equation (2.1)
    **for** $i = 1, 2, \ldots, N$ **do**
        Draw $u \sim \mathcal{U}(0, 1)$
        **if** $u < p(t)$ **then**
            $\vec{X}_i^{\text{new}}$ via the equation (2.11)
        **else**
            Draw $\vec{r}_1, \vec{r}_2 \sim \mathcal{U}(0, 1)$
            $\vec{A}$ , $\vec{C}$ via the equation (2.13), (2.14)
            Draw $q \sim \mathcal{U}(0, 1)$
            **if** $q < 0.5$ **then**
                $\vec{X}_i^{\text{new}}$ via the equation (2.15)
            **else**
                Draw $l \sim \mathcal{U}(-1, 1)$
                $\vec{X}_i^{\text{new}}$ via the equation (2.16)
            **end if**
        **end if**
        $\vec{X}_i^{\text{new}} \leftarrow \text{clip}(\vec{X}_i^{\text{new}}, lb, ub)$
        $\vec{X_i^{t+1}}$ Greedy Selection via (2.17)
    **end for**
    Update $\vec{X}^* \leftarrow \arg\min_i F(\vec{X}_i^{t+1})$
**end for**
**return** $\vec{X}^*$, $F(\vec{X}^*)$

# 3 The Experiments

The experimental pipeline consists of selecting the image mode, image, algorithm, and threshold count $k$, formulating the corresponding segmentation problem, and executing the procedure for a fixed number of trials.

## 3.1 Experiment Overview

The experimental pipeline is applied across every configuration. For each combination of algorithms (PSO, WOA, GWO, ZOA, GA, CMA-ES, WOA-ZOA, Otsu, K-means), objective functions (Kapur entropy, Kapur-SSIM, Kapur-Spatial), threshold count $k \in \{2, 4, 6, 8, 10\} - 1$, and 10 trials, an image is loaded from the dataset and segmented using the selected algorithm, guiding the search by the chosen objective function. The resulting threshold set is applied to reconstruct a segmented image, which is then evaluated against the original using the five image quality metrics (MSE, PSNR, SSIM, FSIM, QILV). Each result will be stored as an individual row, and results are aggregated across trials and images to produce summary statistics and convergence plots.

### 3.1.1 Dataset

All experiments were conducted on a dataset constructed from 46 images from the Berkeley Segmentation Data Set (BSDS500) [41] and four classical images: barbara [42], cameraman [42], mandrill [43], and peppers [43].

For the purpose of this thesis, the train, val, and test subsets of images in BSDS500 were merged, and 46 were selected at random from that collection. Additionally, 23 of these were converted to grayscale before use, while the other 23 remained in color. Together with barbara and cameraman in grayscale, and peppers and mandrill in color, this yields 25 color and 25 grayscale images in the dataset.

### 3.1.2 Environment Setup

All experiments were executed on the MetaCentrum computing grid, a Czech national research computing infrastructure. Each experimental batch was allocated 16 CPU cores, 32 GB of memory, and 10 GB of scratch space, with a 98-hour wall-time limit; parallelization across cores was handled via Python's joblib library. The full experimental suite was distributed across multiple independent batch jobs to manage runtime constraints. Since the purpose of these experiments is to demonstrate methods that do not require training data, no training data were used; therefore, no hyperparameter tuning was performed. All algorithms used the default parameters of mealpy – the library [44, 45, 46] that was used, with the exception of WOA-ZOA, for which $\alpha = 2$. All metaheuristic algorithms used 500 epochs with a population size of 50.

### 3.1.3 Metrics

To assess the quality of each segmented image, five image quality metrics are computed by comparing the original image against the reconstructed segmented image.

#### MSE

Mean Squared Error (MSE) measures the average squared pixel-wise difference between the original image $I$ and the reconstructed image $S$ whose pixel values is the region's

mean:

$$MSE(I,S) = \frac{1}{N}\sum_{(x,y)}(I(x,y) - S(x,y))^2$$

where $N$ is the total number of pixels. As the value decreases, the reconstruction more closely matches the original image. When $MSE(I,S) = 0$, the reconstruction is perfect.

### PSNR

PSNR expresses reconstruction quality via a logarithmic decibel scale. It is derived directly from $MSE$ for a given original image $I$ and its segmented image $S$:

$$PSNR(I,S) = 10\log_{10}\left(\frac{255^2}{MSE(I,S)}\right)$$

Higher reconstruction quality is indicated by higher PSNR values.

### FSIM

Instead of raw pixel intensities, FSIM [47] measures similarity based on low-level visual features, in particular phase congruency ($PC$) and gradient magnitude ($GM$). Phase congruency identifies significant features by detecting points at which frequency components align in phase, while gradient magnitude captures local intensity jumps via edge detection operators. The similarity is computed at every pixel as a function of both measures. To emphasize feature-rich regions, it is weighted by phase congruency as follows:

$$FSIM(I,S) = \frac{\sum_{(x,y)} S_L(x,y)\cdot PC_m(x,y)}{\sum_{(x,y)} PC_m(x,y)}$$

where $I$ and $S$ are the original image and the segmented image, respectively. $S_L(x,y)$ is the combined similarity of $PC$ and $GM$ at $(x,y)$. The maximum $PC$ is indicated by $PC_m(x,y)$ between $I$ and $S$ at pixel $(x,y)$. Note that $FSIM \in [0,1]$ where values near 1 indicate greater feature-level similarity.

### SSIM

SSIM [39] measures the similarity between two images (in our case, the original image and its segmented version) by combining luminance, contrast, and structural similarity. Given images $I$ and $S$ representing the original image and its segmented version, respectively, SSIM is defined as:

$$SSIM(I,S) = \frac{(2\mu_I\mu_S + c_1)(2\sigma_{IS} + c_2)}{(\mu_I^2 + \mu_S^2 + c_1)(\sigma_I^2 + \sigma_S^2 + c_2)}$$

where $\mu_I$, $\mu_S$ are the mean intensities, $\sigma_I^2$, $\sigma_S^2$ are the intensity variances, and $\sigma_{IS}$ is the covariance between images $I$ and $S$. $c_1$ and $c_2$ are small constants. The output of $SSIM(I,S) \in [-1,1]$, where 1 indicates identical images.

### QILV

As for QILV [48], it assesses whether the local variance structure of the image is maintained after segmentation. The image is partitioned into patches, and the variance within each pixel is computed for both both original image $I$ and its segmented image

$S$. The variance distributions resulting from it are compared using three terms: mean variance similarity, variance spread similarity, and variance correlation as follows:

$$QILV(I,S) = \frac{2\overline{v}_I\overline{v}_S + c}{\overline{v}_I^2 + \overline{v}_S^2 + c} \cdot \frac{2\sigma_{v_1}\sigma_{v_S} + c}{\sigma_{v_1}^2 + \sigma_{v_S}^2 + c} \cdot \frac{\sigma_{v_1 v_S} + c}{\sigma_{v_1}\sigma_{v_S} + c}$$

where $\overline{v}_I$,$\overline{v}_S$ denote the mean local variances. $\sigma_{v_1}$,$\sigma_{v_S}$ indicate their standard deviations, while $\sigma_{v_1 v_S}$ is their covariance, and $c$ is a small constant. Note that values closer to 1 imply better preservation of local structure.

## 3.2 CMA-ES Numerical Instability

| $k$ | Expected runs | Successful runs | Failed runs | Failure rate |
|---|---|---|---|---|
| 2 | 500 | 255 | 245 | 49.0% |
| 4 | 500 | 251 | 249 | 49.8% |
| 6 | 500 | 265 | 235 | 47.0% |
| 8 | 500 | 476 | 24 | 4.8% |
| 10 | 500 | 500 | 0 | 0.0% |
| **Overall** | 2500 | 1747 | 753 | 30.12% |

**Table 3.1** CMA-ES failure rate by $k$ (Kapur)

| $k$ | Expected runs | Successful runs | Failed runs | Failure rate |
|---|---|---|---|---|
| 2 | 500 | 253 | 247 | 49.4% |
| 4 | 500 | 253 | 247 | 49.4% |
| 6 | 500 | 268 | 232 | 46.4% |
| 8 | 500 | 466 | 34 | 6.8% |
| 10 | 500 | 500 | 0 | 0.0% |
| **Overall** | 2500 | 1740 | 760 | 30.40% |

**Table 3.2** CMA-ES failure rate by $k$ (Kapur-Spatial)

| $k$ | Expected runs | Successful runs | Failed runs | Failure rate |
|---|---|---|---|---|
| 2 | 500 | 254 | 246 | 49.2% |
| 4 | 500 | 256 | 244 | 48.8% |
| 6 | 500 | 261 | 239 | 47.8% |
| 8 | 500 | 456 | 44 | 8.8% |
| 10 | 500 | 500 | 0 | 0.0% |
| **Overall** | 2500 | 1727 | 773 | 30.92% |

**Table 3.3** CMA-ES failure rate by $k$ (Kapur-SSIM)

We observed that CMA-ES occasionally terminates unexpectedly (i.e., it "crashes") for some experimental configurations. Since CMA-ES is designed to be numerically robust, we attribute these failures primarily to the particular software implementation and its default settings rather than to the CMA-ES method itself. Tables 3.1–3.3 report

the observed failure rate of CMA-ES at each tested value of $k$. The failure rate is highest at $k = 2, 4, 6$, where the search-space dimension is still small, and it decreases as $k$ increases from 8 onward, falling to 0.0% by $k = 10$. This trend is consistent with the problem dimension: $d$ grows from 1 (grayscale, $k = 2$) to 9 (grayscale, $k = 10$), or from 3 to 27 for color images.

## 3.3 Performance Comparison

This section evaluates the performance of the algorithms in the experiment. WOA-ZOA, achieving improved segmentation quality at a measurable computational cost. This section quantifies that cost and assesses whether it is proportionate. First, all metaheuristic approaches are approximately 2 to 3 orders of magnitude slower than the deterministic baselines (K-means and Otsu); therefore, this runtime gap should not be attributed specifically to WOA-ZOA. Second, results in which CMA-ES appears to be best for

$$k = 2, 4, 6$$

should be interpreted with caution in light of Section 3.2. Unless otherwise stated, all results are aggregated across 10 trials.

Across the full set of experiments, K-means generally outperforms the other methods and is only surpassed by WOA–ZOA in a small number of cases. However, it is important to note that the K-means segmentation method differs from the thresholding-based methods. This is a consequence of the objective-metric alignment: K-means directly minimizes the within-cluster sum of squared differences, which, for piecewise-constant reconstruction, equals the MSE of the original and the reconstructed image; Otsu maximizes the between-class variance, which is equivalent to minimizing the within-cluster/class variance.In contrast, the metaheuristic methods are broadly competitive with Otsu. Only when using Kapur-SSIM did the metaheuristic algorithms start to become competitive with K-means and Otsu, as evidenced by Tables A.21–A.29. This mirrors the finding at Section 3.4: the choice of objective function dominates the choice of the algorithm.

Focusing on metaheuristics at $k = 10$, WOA–ZOA exhibits the lowest CPU usage under the Kapur entropy objective. Conversely, ZOA is consistently the slowest across all fitness functions. When the objective is changed to Kapur–Spatial or Kapur–SSIM, WOA–ZOA shows a noticeable runtime increase (Figures A.1–A.2), whereas GA is faster than most alternatives. The increase for WOA–ZOA is expected, given its more complex position-update and constraint-handling logic. ZOA's slower runtime, regardless of fitness function, is primarily due to its two-phase procedure and the additional fitness re-evaluations performed after the first phase. This may also be due to the implementation difference between the different components of WOA-ZOA and their corresponding components in its parent algorithms.

In terms of quality under Kapur entropy, WOA-ZOA frequently attains the strongest metrics (bolded) as $k$ increases, with ZOA typically close behind. This behavior is consistent with WOA-ZOA combining ZOA's strong exploratory mechanism (via the second phase) with WOA-style exploitation, as discussed previously. Occasional cases in which GA or PSO perform best, while GWO and WOA lag, may indicate that the latter two methods are overly exploitative for this optimization landscape. Although CMA-ES does not fail at $k = 8$ and $k = 10$, it performs substantially worse, which is likely attributable to hyperparameters that are not tuned for this task.

Under Kapur-Spatial, CMA-ES dominates at $k = 8$ and $k = 10$. One possible explanation is that this objective may induce local optima that trap several algorithms,

while CMA-ES's sampling-based updates enable larger moves that more readily discover high-quality regions of the search space. The Kapur-Spatial results also exhibit noticeably higher variance than the other objectives.

Finally, Kapur-SSIM (which explicitly promotes perceptual similarity) reduces the performance gap between methods: while ZOA and WOA–ZOA remain competitive, simpler algorithms such as GA and PSO improve relative to their Kapur entropy results. This is consistent with the objective, providing a stronger alignment with the evaluation metrics and thereby guiding a broader range of optimizers toward higher-quality solutions.

Detailed performance-comparison tables and figures are provided in Appendix A.1.

## 3.4 Statistical Analysis

This section evaluates whether the observed performance differences between algorithms are statistically significant. Additional plots are provided in Attachment A.2. Tables 3.4–3.6 summarize Friedman tests across algorithms, reported separately for each metric and threshold count $k$. For Kapur entropy, statistically significant differences are detected for all $k$ values for every metric except QILV at $k = 2$. For Kapur-Spatial, differences are significant for all $k$ for FSIM, QILV, and runtime, while MSE/PSNR/SSIM are not significant at $k = 2$ but become significant for $k \geq 4$. For Kapur-SSIM, SSIM, QILV, and runtime are significant at all tested $k$, whereas MSE/PSNR/FSIM are not significant at $k = 2$ but are significant for $k \geq 4$.

Tables 3.7–3.9 report pairwise Wilcoxon signed-rank tests comparing WOA–ZOA against the other algorithms (36 comparisons per $k$). Under Kapur entropy, WOA-ZOA wins the large majority of significant comparisons, particularly for $k \geq 4$. Under Kapur-Spatial, WOA-ZOA exhibits the opposite trend, with substantially more losses than wins across all $k$. Under Kapur-SSIM, results are more balanced: WOA–ZOA loses more often at $k = 2$, but its win count increases with $k$, indicating that its advantage becomes more pronounced as the number of thresholds grows. This is consistent with the performance comparison in Section 3.3, where Kapur-SSIM yields more competitive behavior across metaheuristics overall, and therefore produces a more even split between statistically significant wins and losses for WOA-ZOA.

The A2 attachment provides additional average-rank / critical-difference (CD) diagrams for $k = 10$ (Figs. A.15–A.20), reported per image across six metrics (MSE, PSNR, SSIM, FSIM, QILV, runtime) for the three objectives. It can be observed that PSO is consistently ranked high.

For the Kapur objective (Figs. A.15–A.16), WOA–ZOA achieves the best average ranks for MSE and PSNR and is among the top group for SSIM. For FSIM and QILV, GA and PSO tend to rank slightly better. For runtime, WOA–ZOA ranks best (fastest), while ZOA and CMA-ES are among the slowest.

For the Kapur–Spatial objective (Figs. A.17–A.18), CMA-ES ranks clearly best across the quality metrics (MSE/PSNR/SSIM and also FSIM/QILV), with WOA, WOA–ZOA, GWO, and ZOA generally ranking worse. For runtime, WOA is fastest, while ZOA is slowest.

For the Kapur–SSIM objective (Figs. A.19–A.20), WOA–ZOA ranks best for MSE and PSNR, while GA ranks best for SSIM and FSIM. For QILV, PSO, GA, and ZOA form the better group. For runtime, CMA-ES ranks fastest, and ZOA and WOA–ZOA rank among the slowest.

| Metric | Significant ($k$ values) | Not significant ($k$ values) |
|---|---|---|
| MSE | 2, 4, 6, 8, 10 | – |
| PSNR | 2, 4, 6, 8, 10 | – |
| SSIM | 2, 4, 6, 8, 10 | – |
| FSIM | 2, 4, 6, 8, 10 | – |
| QILV | 4, 6, 8, 10 | 2 |
| runtime_seconds | 2, 4, 6, 8, 10 | – |

**Table 3.4** Friedman test summary, by metric (Kapur)

| Metric | Significant ($k$ values) | Not significant ($k$ values) |
|---|---|---|
| MSE | 4, 6, 8, 10 | 2 |
| PSNR | 4, 6, 8, 10 | 2 |
| SSIM | 4, 6, 8, 10 | 2 |
| FSIM | 2, 4, 6, 8, 10 | – |
| QILV | 2, 4, 6, 8, 10 | – |
| runtime_seconds | 2, 4, 6, 8, 10 | – |

**Table 3.5** Friedman test summary, by metric (Kapur-Spatial)

| Metric | Significant ($k$ values) | Not significant ($k$ values) |
|---|---|---|
| MSE | 4, 6, 8, 10 | 2 |
| PSNR | 4, 6, 8, 10 | 2 |
| SSIM | 2, 4, 6, 8, 10 | – |
| FSIM | 4, 6, 8, 10 | 2 |
| QILV | 2, 4, 6, 8, 10 | – |
| runtime_seconds | 2, 4, 6, 8, 10 | – |

**Table 3.6** Friedman test summary, by metric (Kapur-SSIM)

| $k$ | Significant comparisons | WOA-ZOA wins | WOA-ZOA losses |
|---|---|---|---|
| 2 | 17/36 | 12 | 5 |
| 4 | 29/36 | 28 | 1 |
| 6 | 26/36 | 24 | 2 |
| 8 | 29/36 | 27 | 2 |
| 10 | 29/36 | 27 | 2 |

**Table 3.7** WOA-ZOA pairwise Wilcoxon summary by $k$ (Kapur)

| $k$ | Significant comparisons | WOA-ZOA wins | WOA-ZOA losses |
|---|---|---|---|
| 2 | 25/36 | 4 | 21 |
| 4 | 32/36 | 3 | 29 |
| 6 | 34/36 | 4 | 30 |
| 8 | 33/36 | 4 | 29 |
| 10 | 33/36 | 4 | 29 |

**Table 3.8** WOA-ZOA pairwise Wilcoxon summary by $k$ (Kapur-Spatial)

| $k$ | Significant comparisons | WOA-ZOA wins | WOA-ZOA losses |
|---|---|---|---|
| 2 | 28/36 | 9 | 19 |
| 4 | 29/36 | 17 | 12 |
| 6 | 32/36 | 21 | 11 |
| 8 | 28/36 | 19 | 9 |
| 10 | 29/36 | 21 | 8 |

**Table 3.9** WOA-ZOA pairwise Wilcoxon summary by $k$ (Kapur-SSIM)

In addition to comparing optimization algorithms, we test whether the choice of objective function (Kapur entropy, Kapur–Spatial, and Kapur–SSIM) leads to statistically different outcomes. Table 3.10 reports Friedman tests across objective functions and indicates statistically significant differences for every metric at all tested values of $k$, suggesting that the objective function materially influences the quality metrics.

Table 3.11 summarizes pairwise Wilcoxon signed-rank tests between objective functions across the 30 metric–$k$ combinations. Kapur entropy significantly outperforms Kapur-Spatial in all comparisons (30/30). Kapur-SSIM outperforms Kapur entropy in the majority of significant comparisons (20 wins versus 8, with 28/30 significant). Similarly, Kapur–SSIM dominates Kapur-Spatial (25 wins versus 5, with 30/30 significant). Overall, these results indicate that Kapur-SSIM provides the most consistently favorable trade-off across the evaluated metrics, whereas Kapur-Spatial is the least competitive.

| Metric | Significant ($k$ values) | Not significant ($k$ values) |
|---|---|---|
| MSE | 2, 4, 6, 8, 10 | – |
| PSNR | 2, 4, 6, 8, 10 | – |
| SSIM | 2, 4, 6, 8, 10 | – |
| FSIM | 2, 4, 6, 8, 10 | – |
| QILV | 2, 4, 6, 8, 10 | – |

**Table 3.10** Friedman test summary across objective functions, by metric (all)

| Comparison | Significant | Wins (first) | Wins (second) |
|---|---|---|---|
| Kapur vs Spatial-Kapur | 30/30 | 30 | 0 |
| Kapur vs Kapur-SSIM | 28/30 | 8 | 20 |
| Spatial-Kapur vs Kapur-SSIM | 30/30 | 5 | 25 |

**Table 3.11** Pairwise objective function comparison summary (all)

## 3.5 Convergence Analysis

This section reports the convergence behavior over 500 epochs for the metaheuristic algorithms under each objective function, shown for both grayscale and color images (Figures 3.1–3.6). Each figure includes the best, worst, and mean fitness trajectories aggregated across all images and trials.

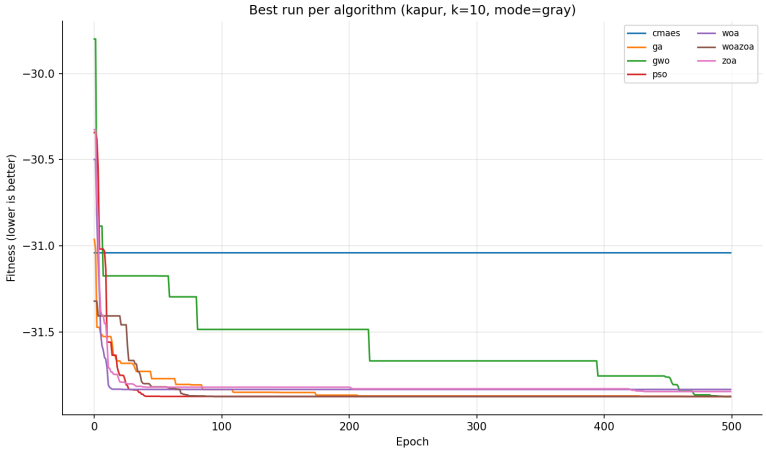
Across all objectives, most methods achieve the majority of their fitness improvement within the first $\approx$ 50–100 epochs, after which progress slows and the curves typically plateau (Figures 3.1–3.6). Algorithms that emphasise exploration tend to exhibit more

step-like decreases and later improvements, whereas more exploitative methods converge rapidly but may stagnate earlier.

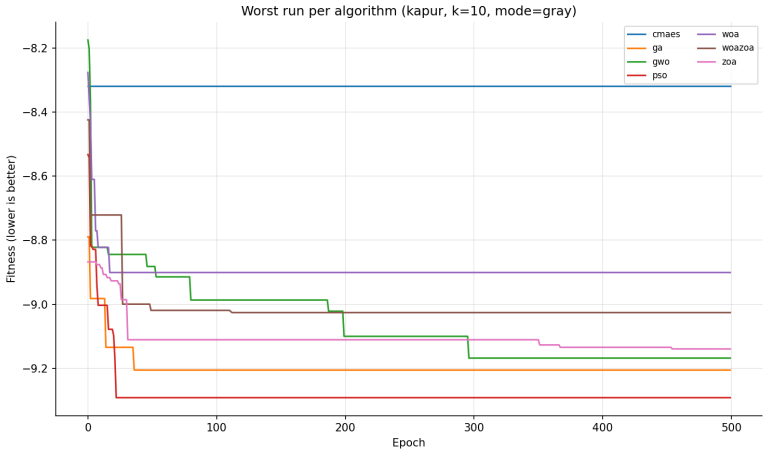
Under Kapur entropy (Figures 3.1 and 3.2), the worst-case and mean trajectories of WOA–ZOA converge at a similar rate to ZOA, while the best-case trajectory converges faster than ZOA. WOA lags behind both methods and, in the mean case, exhibits rapid early convergence followed by minimal subsequent improvement; a similar stagnation pattern is observed for CMA-ES. In contrast, GA and PSO attain the lowest mean fitness values, with GA achieving the best average performance.

Under Kapur–Spatial (Figures 3.3 and 3.4), several algorithms become trapped in poor basins during the worst-case runs and show a limited ability to escape, suggesting that the objective provides a weaker or noisier search signal and increases susceptibility to local optima. Nevertheless, the mean trajectories indicate that WOA–ZOA remains able to continue improving throughout the run, which is consistent with its stronger exploration mechanism; ZOA exhibits similar behavior.

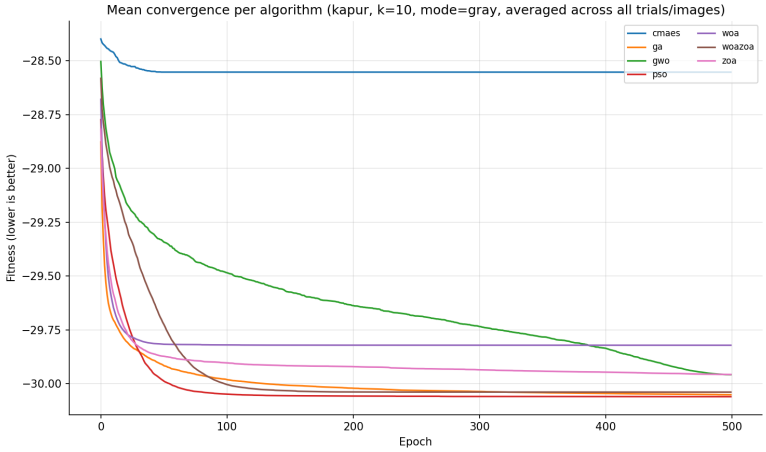
Under Kapur–SSIM (Figures 3.5 and 3.6), the grayscale results show that most algorithms converge to a similar fitness region, with a slightly worse cluster formed by GWO and ZOA, while WOA converges more slowly and CMA-ES again appears to stagnate. This supports the earlier observation that a more targeted objective function reduces performance separation between metaheuristics in the mean case. For color images, GA achieves the lowest mean fitness, marginally outperforming PSO (by approximately 0.005 in fitness), while the remaining algorithms (including WOA–ZOA, ZOA, and GWO) are close behind, differing by roughly 0.08 on average.




**(a)** Best

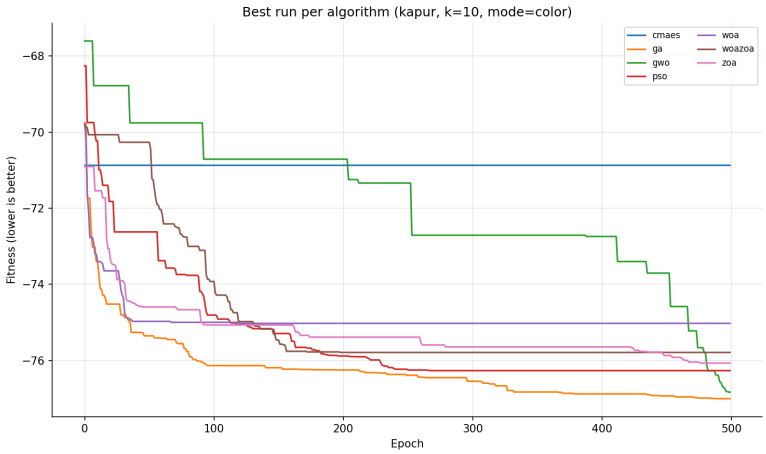



**(b)** Worst

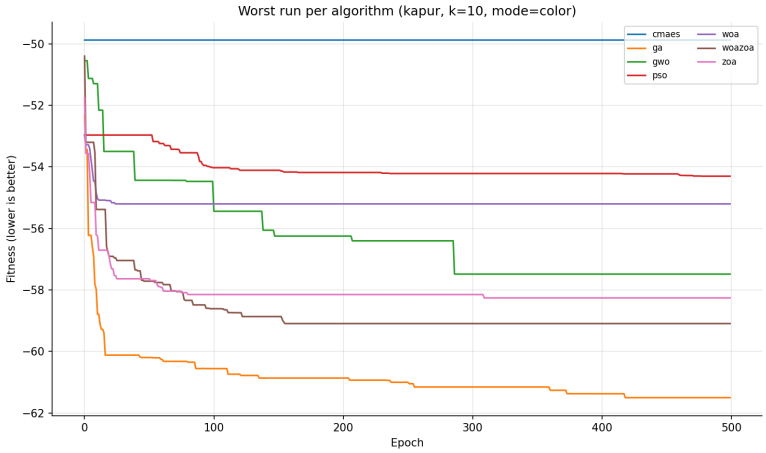



**(c)** Mean

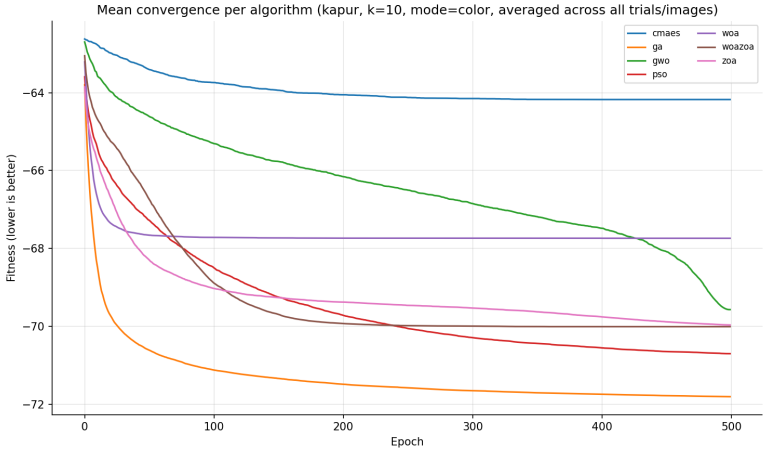
**Figure 3.1** Convergence curves for Kapur objective (grayscale).




**(a)** Best

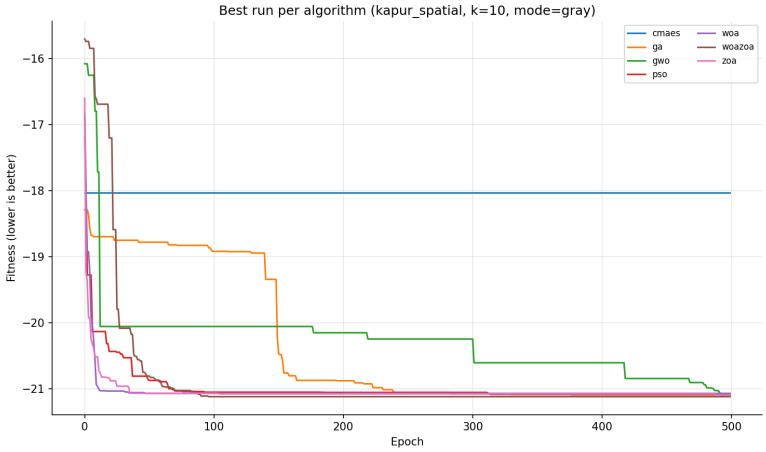



**(b)** Worst

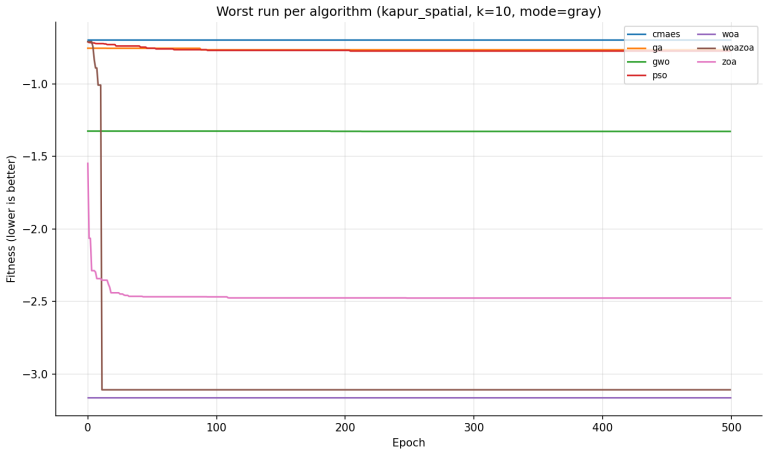



**(c)** Mean

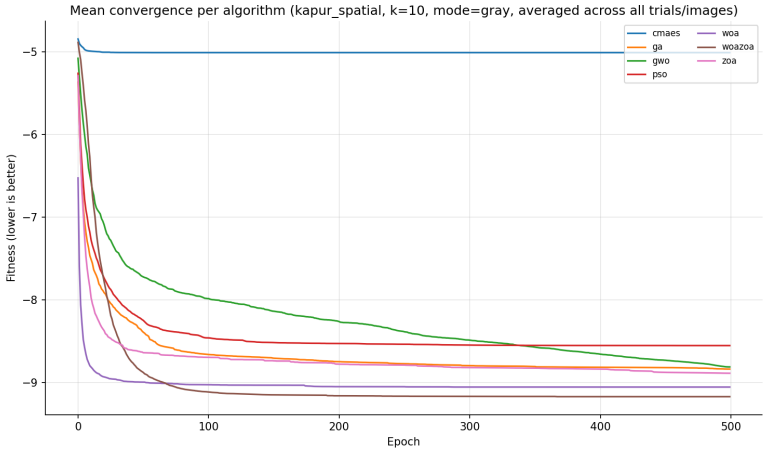
**Figure 3.2** Convergence curves for Kapur objective (color).




**(a)** Best

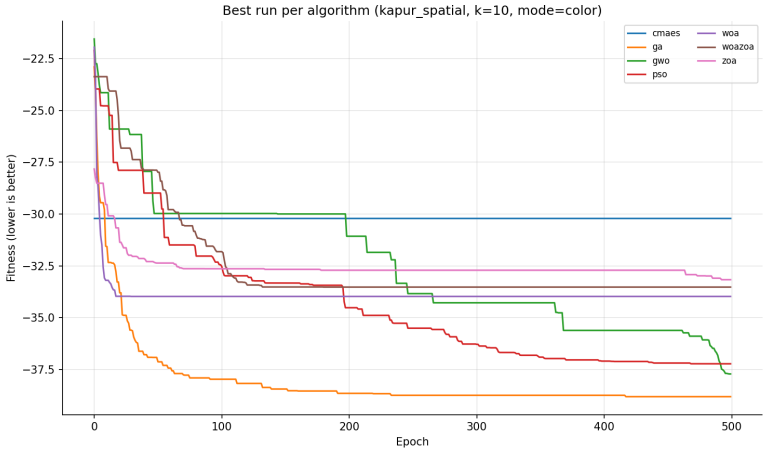



**(b)** Worst

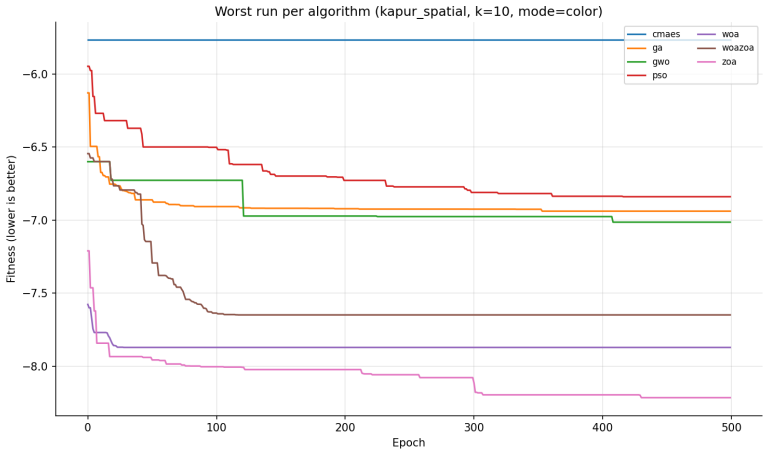



**(c)** Mean

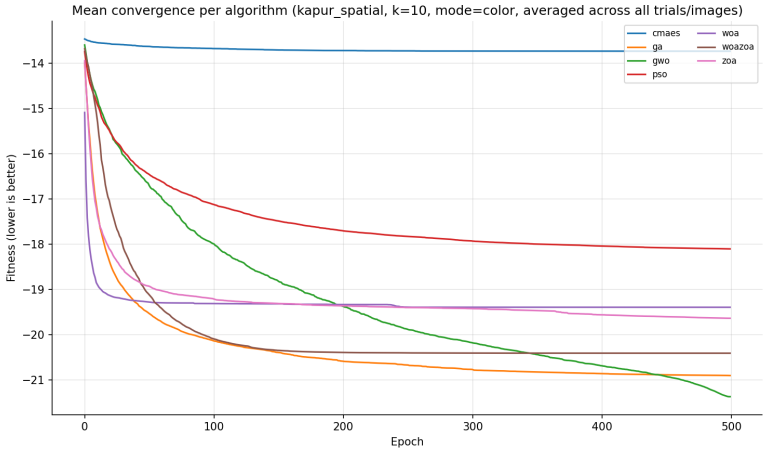
**Figure 3.3** Convergence curves for Kapur-Spatial objective (grayscale).




**(a)** Best

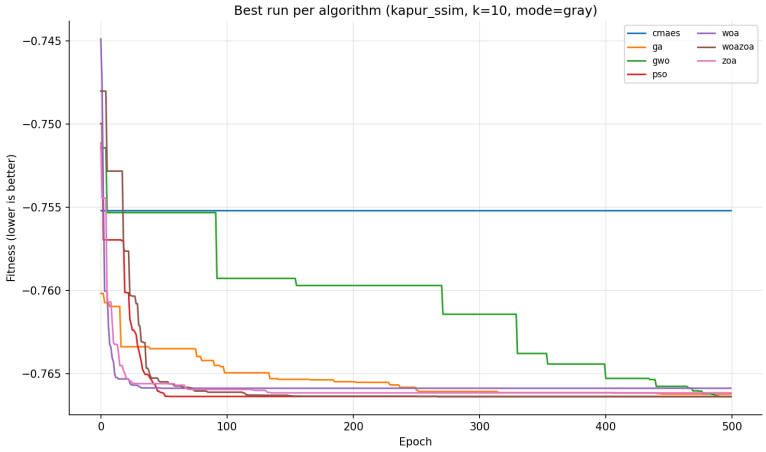



**(b)** Worst

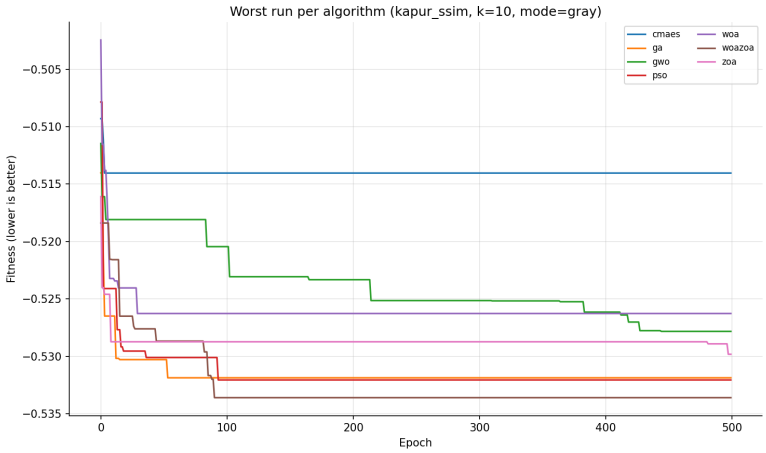



**(c)** Mean

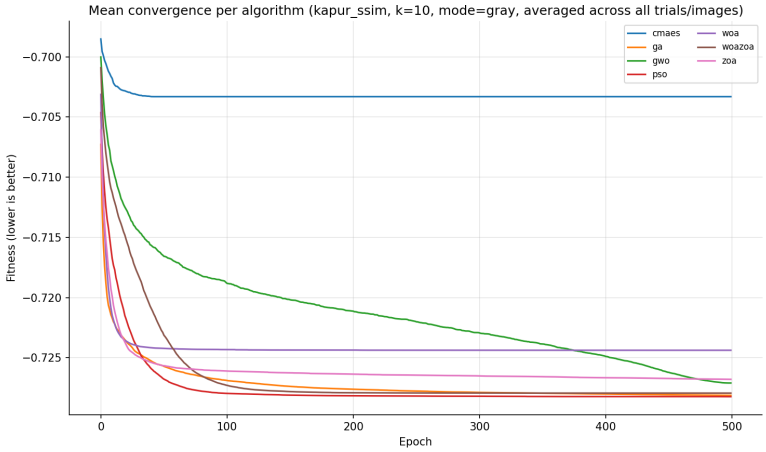
**Figure 3.4** Convergence curves for Kapur-Spatial objective (color).




**(a)** Best

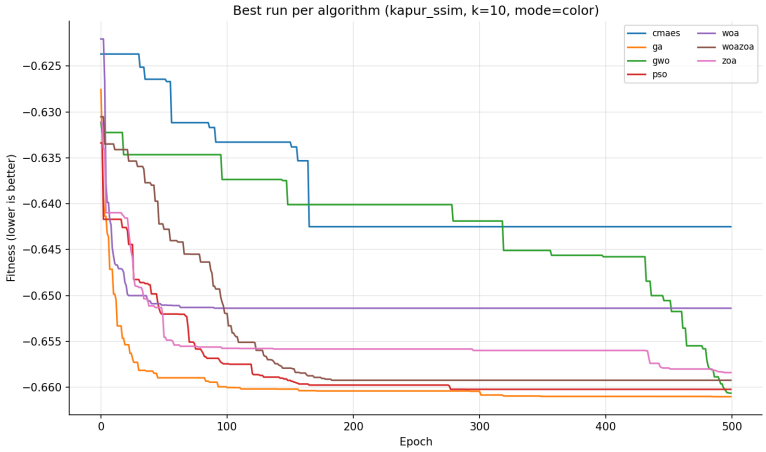



**(b)** Worst

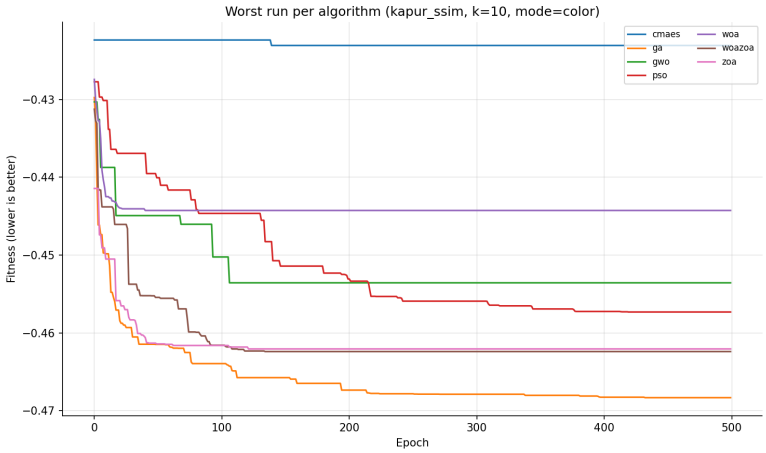



**(c)** Mean

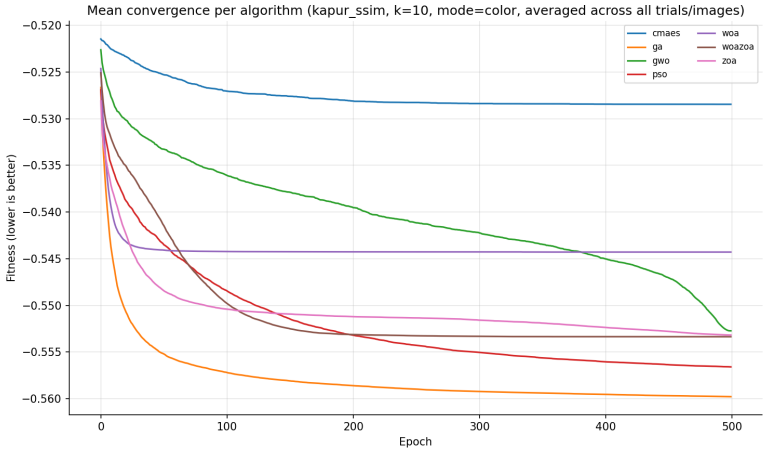
**Figure 3.5** Convergence curves for Kapur-SSIM objective (grayscale).




**(a)** Best




**(b)** Worst




**(c)** Mean

**Figure 3.6** Convergence curves for Kapur-SSIM objective (color).

# Conclusion

This thesis investigated nature-inspired metaheuristics for multilevel-thresholding image segmentation, with the primary aim of comparing their behavior and performance on this task. We proposed a hybrid optimizer, WOA–ZOA, and evaluated it against six established metaheuristics and two classical baselines under three entropy-based objective functions.

Across the evaluated configurations, K-means and Otsu outperform the metaheuristic methods in most cases, which is consistent with their optimization criteria being more closely aligned with within-class variance and MSE-like reconstruction error than the entropy-based thresholding objectives. This gap is particularly pronounced for color images, where per-channel thresholding increases the search-space dimensionality by a factor of three. Within the metaheuristic family, WOA–ZOA consistently improves upon WOA. Nevertheless, ZOA is frequently highly competitive with WOA–ZOA and, in some settings, achieves better results. Consequently, the relative advantage of WOA–ZOA over ZOA is both modality- and objective-dependent: WOA–ZOA matches or exceeds ZOA on grayscale images under Kapur entropy and Kapur–SSIM in most cases, but falls behind on colour images and, most clearly, under the Kapur–Spatial objective. These results indicate that the phase-switching mechanism can interact unfavorably with the spatial smoothness penalty, thereby negating the expected benefits of hybridization.

Among the objective functions, Kapur–SSIM improves upon standard Kapur entropy by incorporating an explicit reconstruction-quality term, whereas Kapur–Spatial is the weakest of the three and, in particular, degrades the hybrid's performance. Across the metaheuristics, the swarm-based methods achieve broadly similar results, with GA remaining competitive throughout. Finally, CMA-ES exhibits occasional numerical failures in our experiments, which we attribute primarily to the particular software implementation and its default settings rather than to the CMA-ES method itself.

Future work should focus on clarifying why the Kapur–Spatial objective degrades the hybrid by diagnosing and, if necessary, rescaling the spatial penalty $M(\vec{X})$. The fixed phase-switching parameter $\alpha = 2$ could be replaced by an adaptive schedule since the effective exploration length appears to vary by objective function and by $k$. For color images, evaluating joint (rather than per-channel) thresholding may reduce the dimensionality increase and help close the gap to classical baselines. Finally, it would be useful to extend the benchmark to learning-based segmentation methods and explore neuroevolution approaches (e.g., NEAT) for this problem.

## Use of Generative AI Tools

Generative AI tools were used to improve wording and grammar and to assist with the placement and formatting of tables.

# Bibliography


1. Kabiraj, Anwesh; Pal, Debojyoti; Ganguly, Debayan; Chatterjee, Kingshuk; Roy, Sudipta. Number plate recognition from enhanced super-resolution using generative adversarial network. *Multimedia Tools and Applications*. 2022, vol. 82, pp. 1–17. Available from doi: 10.1007/s11042-022-14018-0.
2. Jin, Bo; Cruz, Leandro; Gonçalves, Nuno. Deep Facial Diagnosis: Deep Transfer Learning From Face Recognition to Facial Diagnosis. *IEEE Access*. 2020, vol. 8, pp. 1–1. Available from doi: 10.1109/ACCESS.2020.3005687.
3. Zhao, Mengyang; Liu, Quan; Jha, Aadarsh; Deng, Ruining; Yao, Tianyuan; Mahadevan-Jansen, Anita; Tyska, Matthew J.; Millis, Bryan A.; Huo, Yuankai. *VoxelEmbed: 3D Instance Segmentation and Tracking with Voxel Embedding based Deep Learning*. 2021. Available from arXiv: 2106.11480 [cs.CV].
4. Minaee, Shervin; Boykov, Yuri; Porikli, Fatih; Plaza, Antonio; Kehtarnavaz, Nasser; Terzopoulos, Demetri. *Image Segmentation Using Deep Learning: A Survey*. 2020. Available from arXiv: 2001.05566 [cs.CV].
5. Bonabeau, Eric; Dorigo, Marco; Theraulaz, Guy. Swarm Intelligence : From Natural to Artificial Systems / E. Bonabeau, M. Dorigo, G. Theraulaz. 2001.
6. Goldberg, David E. *Genetic Algorithms in Search, Optimization and Machine Learning*. 1st. USA: Addison-Wesley Longman Publishing Co., Inc., 1989. isbn 0201157675.
7. Beyer, Hans-Georg; Schwefel, Hans-Paul. Evolution strategies - A comprehensive introduction. *Natural Computing*. 1999, vol. 1, pp. 3–52. Available from doi: 10.1023/A:1015059928466.
8. Jyothi, Singaraju; Kumbham, Bhargavi. A Survey on Threshold Based Segmentation Technique in Image Processing. *26. K. Bhargavi, S. Jyothi*. 2014, vol. 3.
9. Lloyd, Stuart. Least squares quantization in PCM. *IEEE transactions on information theory*. 1982, vol. 28, no. 2, pp. 129–137.
10. MacQueen, J. Some methods for classification and analysis of multivariate observations. In: 1967. Available also from: https://api.semanticscholar.org/CorpusID:6278891.
11. Ziou, Djemel; Tabbone, Salvatore. Edge detection techniques-an overview. *Pattern Recognition and Image Analysis: Advances in Mathematical Theory and Applications*. 1998, vol. 8, no. 4, pp. 537–559.
12. Adams, Rolf; Bischof, Leanne. Seeded region growing. *IEEE Transactions on pattern analysis and machine intelligence*. 1994, vol. 16, no. 6, pp. 641–647.
13. Mirjalili, Seyedali; Lewis, Andrew. The Whale Optimization Algorithm. *Advances in Engineering Software*. 2016, vol. 95, pp. 51–67. issn 0965-9978. Available from doi: https://doi.org/10.1016/j.advengsoft.2016.01.008.
14. Trojovská, Eva; Dehghani, Mohammad; Trojovský, Pavel. Zebra Optimization Algorithm: A New Bio-Inspired Optimization Algorithm for Solving Optimization Algorithm. *IEEE Access*. 2022, vol. 10, pp. 49445–49473. Available from doi: 10.1109/ACCESS.2022.3172789.
15. Otsu, Nobuyuki. A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics*. 1979, vol. 9, no. 1, pp. 62–66. Available from doi: 10.1109/TSMC.1979.4310076.

16. Morse, Bryan; Barrett, William. A recursive Otsu thresholding method for scanned document binarization. In: 2011, pp. 307–314. Available from DOI: 10.1109/WACV.2011.5711519.

17. Reddi, S. S.; Rudin, S. F.; Keshavan, H. R. An optimal multiple threshold scheme for image segmentation. *IEEE Transactions on Systems, Man, and Cybernetics*. 1984, vol. SMC-14, no. 4, pp. 661–665. Available from DOI: 10.1109/TSMC.1984.6313341.

18. Xu, Junqin; Zhang, Jihui. Exploration-exploitation tradeoffs in metaheuristics: Survey and analysis. In: *Proceedings of the 33rd Chinese control conference*. IEEE, 2014, pp. 8633–8638.

19. Kotthoff, Lars. Algorithm selection for combinatorial search problems: A survey. In: *Data mining and constraint programming: Foundations of a cross-disciplinary approach*. Springer, 2016, pp. 149–190.

20. Wolpert, David H; Macready, William G. No free lunch theorems for optimization. *IEEE transactions on evolutionary computation*. 1997, vol. 1, no. 1, pp. 67–82.

21. Kennedy, J.; Eberhart, R. Particle swarm optimization. In: *Proceedings of ICNN'95 - International Conference on Neural Networks*. 1995, vol. 4, 1942–1948 vol.4. Available from DOI: 10.1109/ICNN.1995.488968.

22. Pilát, Martin. *Swarms and Colonies* [Lecture notes, Nature-Inspired Algorithms course]. 2026. Available also from: https://martinpilat.com/en/nature-inspired-algorithms/swarms-colonies.

23. Mirjalili, Seyedali; Mirjalili, Seyed; Lewis, Andrew. Grey Wolf Optimizer. *Advances in Engineering Software*. 2014, vol. 69, pp. 46–61. Available from DOI: 10.1016/j.advengsoft.2013.12.007.

24. Goldbogen, Jeremy A; Friedlaender, Ari S; Calambokidis, John; McKenna, Megan F; Simon, Malene; Nowacek, Douglas P. Integrative approaches to the study of baleen whale diving behavior, feeding performance, and foraging ecology. *BioScience*. 2013, vol. 63, no. 2, pp. 90–100.

25. Hansen, Nikolaus; Müller, Sibylle D.; Koumoutsakos, Petros. Reducing the Time Complexity of the Derandomized Evolution Strategy with Covariance Matrix Adaptation (CMA-ES). *Evolutionary Computation*. 2003, vol. 11, no. 1, pp. 1–18. Available from DOI: 10.1162/106365603321828970.

26. Chouksey, Mausam; Jha, Rajib Kumar; Sharma, Rajat. A fast technique for image segmentation based on two meta-heuristic algorithms. *Multimedia tools and applications*. 2020, vol. 79, no. 27, pp. 19075–19127.

27. Mirjalili, Seyedali; Mirjalili, Seyed Mohammad; Hatamlou, Abdolreza. Multiverse optimizer: a nature-inspired algorithm for global optimization. *Neural computing and applications*. 2016, vol. 27, no. 2, pp. 495–513.

28. Mirjalili, Seyedali. The ant lion optimizer. *Advances in engineering software*. 2015, vol. 83, pp. 80–98.

29. Abualigah, Laith; Almotairi, Khaled H; Elaziz, Mohamed Abd. Multilevel thresholding image segmentation using meta-heuristic optimization algorithms: Comparative analysis, open challenges and new trends. *Applied Intelligence*. 2023, vol. 53, no. 10, pp. 11654–11704.

30. Seyyedabbasi, Amir. A hybrid multi-strategy optimization metaheuristic algorithm for multi-level thresholding color image segmentation. *Applied Sciences*. 2025, vol. 15, no. 13, p. 7255.

31. Seyyedabbasi, Amir; Kiani, Farzad. Sand Cat swarm optimization: A nature-inspired algorithm to solve global optimization problems. *Engineering with computers*. 2023, vol. 39, no. 4, pp. 2627–2651.

32. Mohammed, Hardi; Rashid, Tarik. A novel hybrid GWO with WOA for global numerical optimization and solving pressure vessel design. *Neural Computing and Applications*. 2020, vol. 32, no. 18, pp. 14701–14718.

33. Mafarja, Majdi M; Mirjalili, Seyedali. Hybrid whale optimization algorithm with simulated annealing for feature selection. *Neurocomputing*. 2017, vol. 260, pp. 302–312.

34. Trivedi, Indrajit N; Jangir, Pradeep; Kumar, Arvind; Jangir, Narottam; Totlani, Rahul. A novel hybrid PSO–WOA algorithm for global numerical functions optimization. In: *Advances in Computer and Computational Sciences: Proceedings of ICCCCS 2016, Volume 2*. Springer, 2017, pp. 53–60.

35. Ali, Ayad. A Hybrid Grey Wolf Optimizer–Zebra Optimization Algorithm for Solving Optimization Problems. *Jambura Journal of Mathematics*. 2026, vol. 8, no. 1, pp. 7–25.

36. Ravinder, Bathini; Haritha, D. JZOA: an effective hybrid optimization algorithm for energy-aware VM allocation in green cloud computing. *Engineering Research Express*. 2026, vol. 8, no. 8, p. 085213.

37. Li, Jinming; Lei, Bo. Two-dimensional Kapur Entropy Multilevel Thresholding Algorithm Based on Dynamic Programming and Spatial Contextual Information. In: *2023 19th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*. IEEE, 2023, pp. 1–8.

38. Kapur, Jagat Narain; Sahoo, Prasanna K; Wong, Andrew KC. A new method for gray-level picture thresholding using the entropy of the histogram. *Computer vision, graphics, and image processing*. 1985, vol. 29, no. 3, pp. 273–285.

39. Wang, Zhou; Bovik, Alan C; Sheikh, Hamid R; Simoncelli, Eero P. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*. 2004, vol. 13, no. 4, pp. 600–612.

40. Shannon, C. E. A mathematical theory of communication. *The Bell System Technical Journal*. 1948, vol. 27, no. 3, pp. 379–423. Available from doi: 10.1002/j.1538-7305.1948.tb01338.x.

41. Arbelaez, Pablo; Maire, Michael; Fowlkes, Charless; Malik, Jitendra. Contour Detection and Hierarchical Image Segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.* 2011, vol. 33, no. 5, pp. 898–916. issn 0162-8828. Available from doi: 10.1109/TPAMI.2010.161.

42. Computer Vision Group, University of Granada. *Computer Vision Image Database* [https://decsai.ugr.es/cvg/dbimagenes/]. [N.d.]. Accessed 2026.

43. USC Signal and Image Processing Institute. *The USC-SIPI Image Database* [http://sipi.usc.edu/database/]. Accessed 2026. University of Southern California.

44. Van Thieu, Nguyen; Mirjalili, Seyedali. MEALPY: An open-source library for latest meta-heuristic algorithms in Python. *Journal of Systems Architecture*. 2023. Available from doi: 10.1016/j.sysarc.2023.102871.

45. Van Thieu, Nguyen; Barma, Surajit Deb; Van Lam, To; Kisi, Ozgur; Mahesha, Amai. Groundwater level modeling using Augmented Artificial Ecosystem Optimization. *Journal of Hydrology*. 2023, vol. 617, p. 129034. Available from doi: https://doi.org/10.1016/j.jhydrol.2022.129034.

46. Ahmed, Ali Najah; Van Lam, To; Hung, Nguyen Duy; Van Thieu, Nguyen; Kisi, Ozgur; El-Shafie, Ahmed. A comprehensive comparison of recent developed meta-heuristic algorithms for streamflow time series forecasting problem. *Applied Soft Computing*. 2021, vol. 105, p. 107282. Available from doi: 10.1016/j.asoc.2021.107282.

47. Zhang, Lin; Zhang, Lei; Mou, Xuanqin; Zhang, David. FSIM: A feature similarity index for image quality assessment. *IEEE transactions on Image Processing*. 2011, vol. 20, no. 8, pp. 2378–2386.

48. Aja-Fernandez, Santiago; Estepar, Raul San Jose; Alberola-Lopez, Carlos; Westin, Carl-Fredrik. Image quality assessment based on local variance. In: *2006 international conference of the ieee engineering in medicine and biology society*. IEEE, 2006, pp. 4815–4818.

# List of Figures

# List of Tables

# List of Abbreviations

**GA** Genetic Algorithm

**CMA-ES** Covariance Matrix Adaptation Evolution Strategy

**HC** Hill Climbing

**SA** Simulated Annealing

**ZOA** Zebra Optimization Algorithm

**PSO** Particle Swarm Optimization

**WOA** Whale Optimization Algorithm

**GWO** Gray Wolf Optimizer

**WOA-ZOA** Whale Optimization Algorithm - Zebra Optimization Algorithm

**ROI** Region of Interest

**MSE** Mean Squared Error

**QILV** Quality Index based on Local Variance

**PSNR** Peak Signal to Noise Ratio

**FSIM** Feature Similarity Index Measure

**SSIM** Structural Similarity Index Measure

**BSDS500** Berkeley Segmentation Data Set

**MVO** Multi-Verse Optimizer

**ALO** Ant Lion Optimizer

**SCSO** Sand Cat Swarm Optimization

**Spatial Kapur** Kapur-Spatial

**PC** Phase Congruency

**GM** Gradient Magnitude

# A Attachments

## A.1 Performance Comparison: Tables and Figures

This attachment contains the supplementary tables and figures referenced in Section 3.3.

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 990.998 ± 635.130 | 18.749 ± 2.141 | 0.580 ± 0.111 | 0.609 ± 0.062 | 0.556 ± 0.192 |
| GA | 991.051 ± 634.813 | 18.749 ± 2.140 | 0.580 ± 0.111 | 0.609 ± 0.062 | 0.556 ± 0.192 |
| GWO | 990.904 ± 634.812 | 18.749 ± 2.140 | 0.580 ± 0.111 | 0.609 ± 0.062 | 0.556 ± 0.192 |
| K-means | **724.945** ± 290.801 | **19.974** ± 2.180 | 0.624 ± 0.101 | **0.628** ± 0.063 | **0.583** ± 0.175 |
| Otsu | 725.898 ± 290.743 | 19.968 ± 2.181 | **0.625** ± 0.101 | **0.628** ± 0.063 | 0.576 ± 0.172 |
| PSO | 990.904 ± 634.812 | 18.749 ± 2.140 | 0.580 ± 0.111 | 0.609 ± 0.062 | 0.556 ± 0.192 |
| WOA | 990.904 ± 634.812 | 18.749 ± 2.140 | 0.580 ± 0.111 | 0.609 ± 0.062 | 0.556 ± 0.192 |
| WOA-ZOA | 990.904 ± 634.812 | 18.749 ± 2.140 | 0.580 ± 0.111 | 0.609 ± 0.062 | 0.556 ± 0.192 |
| ZOA | 990.904 ± 634.812 | 18.749 ± 2.140 | 0.580 ± 0.111 | 0.609 ± 0.062 | 0.556 ± 0.192 |

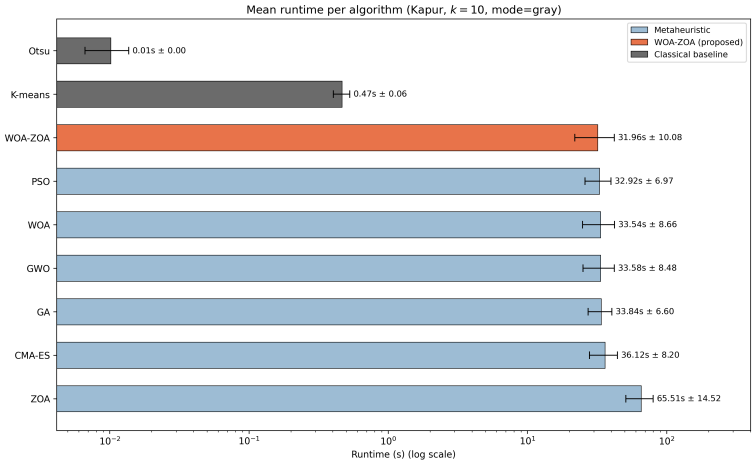
**Table A.1** Mean ± std per algorithm (Kapur, $k = 2$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 714.496 ± 340.824 | 20.302 ± 2.711 | **0.633** ± 0.157 | **0.706** ± 0.065 | **0.723** ± 0.053 |
| GA | 994.348 ± 719.272 | 18.885 ± 2.423 | 0.582 ± 0.119 | 0.688 ± 0.071 | 0.647 ± 0.209 |
| GWO | 1024.198 ± 723.322 | 18.773 ± 2.490 | 0.583 ± 0.119 | 0.688 ± 0.072 | 0.639 ± 0.208 |
| K-means | **664.627** ± 298.632 | **20.437** ± 2.290 | 0.627 ± 0.113 | 0.695 ± 0.070 | 0.685 ± 0.175 |
| Otsu | 664.802 ± 298.618 | 20.435 ± 2.289 | 0.628 ± 0.113 | 0.696 ± 0.070 | 0.682 ± 0.176 |
| PSO | 1015.391 ± 725.269 | 18.820 ± 2.496 | 0.584 ± 0.120 | 0.689 ± 0.072 | 0.639 ± 0.213 |
| WOA | 1017.176 ± 732.420 | 18.832 ± 2.534 | 0.584 ± 0.121 | 0.690 ± 0.072 | 0.641 ± 0.211 |
| WOA-ZOA | 1016.900 ± 728.062 | 18.833 ± 2.542 | 0.584 ± 0.120 | 0.689 ± 0.073 | 0.641 ± 0.208 |
| ZOA | 1014.278 ± 724.313 | 18.834 ± 2.521 | 0.584 ± 0.120 | 0.689 ± 0.073 | 0.641 ± 0.210 |

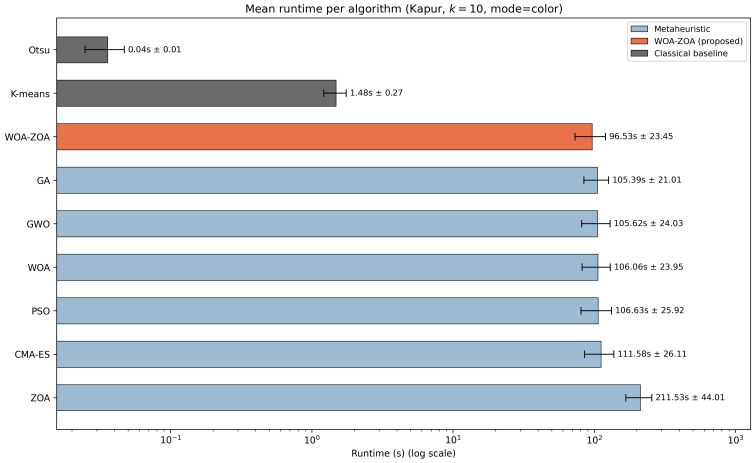
**Table A.2** Mean ± std per algorithm (Kapur, $k = 2$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 365.037 ± 0.000 | 22.507 ± 0.000 | 0.722 ± 0.000 | 0.742 ± 0.000 | 0.888 ± 0.000 |
| GA | 243.874 ± 76.141 | 24.480 ± 1.407 | 0.762 ± 0.065 | 0.766 ± 0.046 | 0.912 ± 0.047 |
| GWO | 242.801 ± 76.996 | 24.507 ± 1.433 | 0.763 ± 0.064 | 0.768 ± 0.045 | 0.915 ± 0.040 |
| K-means | **180.097** ± 63.097 | **25.954** ± 2.067 | **0.795** ± 0.048 | **0.794** ± 0.032 | **0.918** ± 0.036 |
| Otsu | 262.585 ± 122.561 | 24.539 ± 2.566 | 0.784 ± 0.054 | 0.778 ± 0.045 | 0.822 ± 0.150 |
| PSO | 243.731 ± 76.519 | 24.488 ± 1.427 | 0.762 ± 0.065 | 0.766 ± 0.046 | 0.915 ± 0.039 |
| WOA | 244.956 ± 75.800 | 24.459 ± 1.408 | 0.759 ± 0.068 | 0.764 ± 0.047 | 0.915 ± 0.040 |
| WOA-ZOA | 241.160 ± 76.811 | 24.537 ± 1.432 | 0.764 ± 0.062 | 0.769 ± 0.043 | 0.916 ± 0.037 |
| ZOA | 242.815 ± 76.772 | 24.505 ± 1.426 | 0.763 ± 0.063 | 0.768 ± 0.044 | 0.915 ± 0.040 |

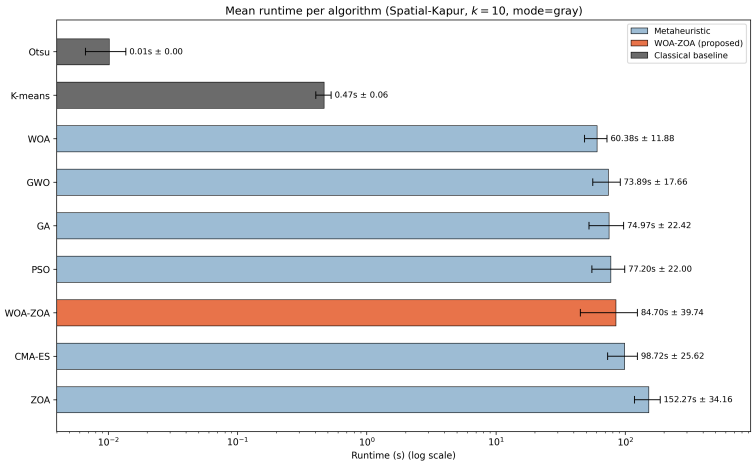
**Table A.3** Mean ± std per algorithm (Kapur, $k = 4$, mode=gray)




**(a)** Kapur, $k = 10$, gray

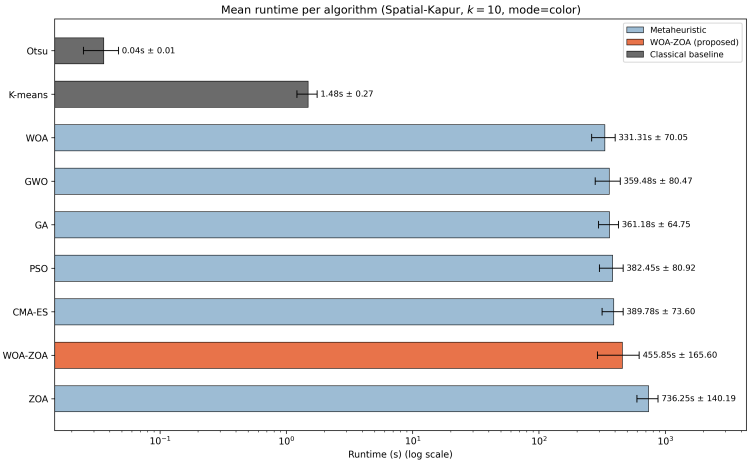



**(b)** Kapur, $k = 10$, color

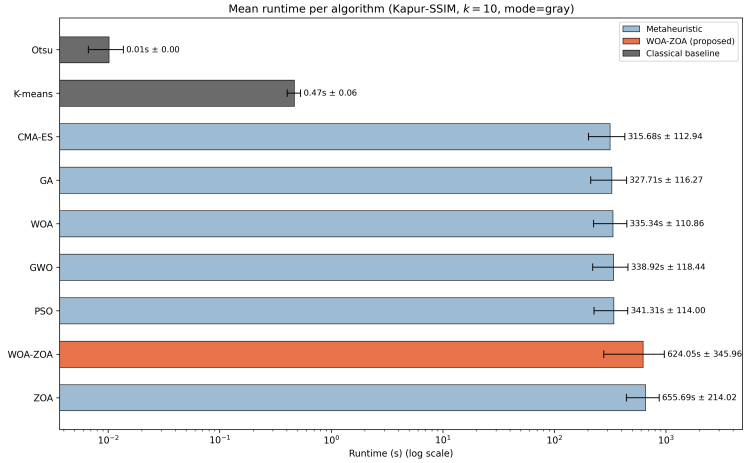



**(c)** Kapur-Spatial, $k = 10$, gray

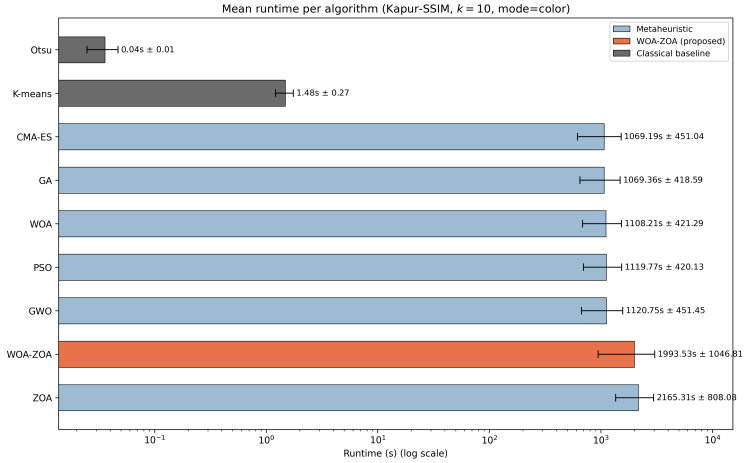
**Figure A.1** Runtime comparison bar charts (with classical baselines).




**(a)** Kapur-Spatial, $k = 10$, color

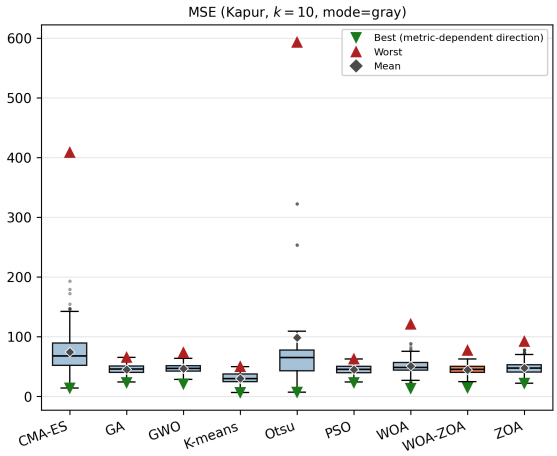



**(b)** Kapur-SSIM, $k = 10$, gray

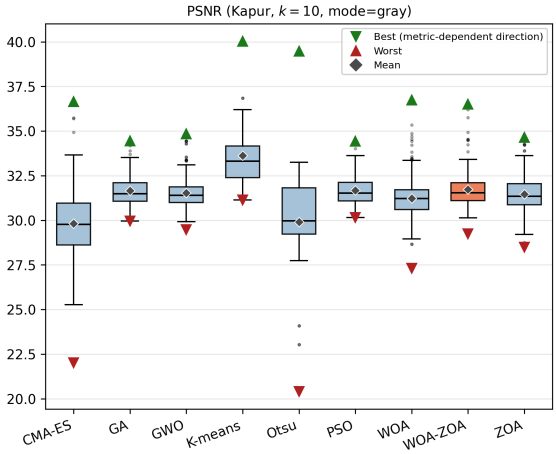



**(c)** Kapur-SSIM, $k = 10$, color

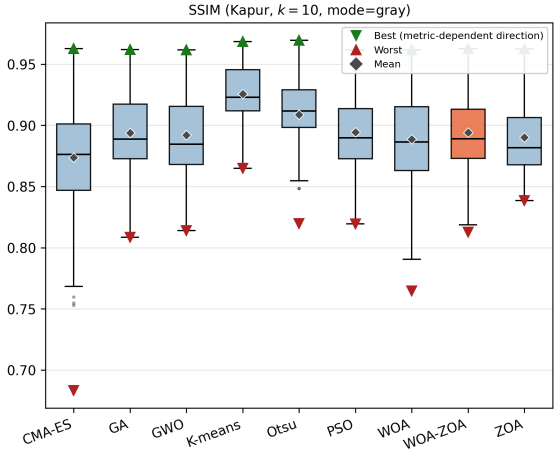
**Figure A.2** Runtime comparison bar charts (with classical baselines).




**(a)** MSE (gray)

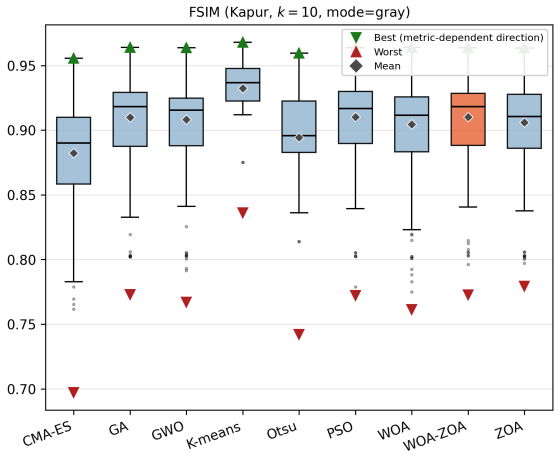



**(b)** PSNR (gray)

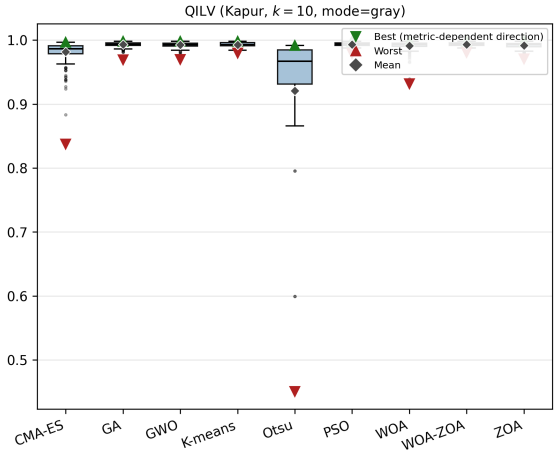



**(c)** SSIM (gray)

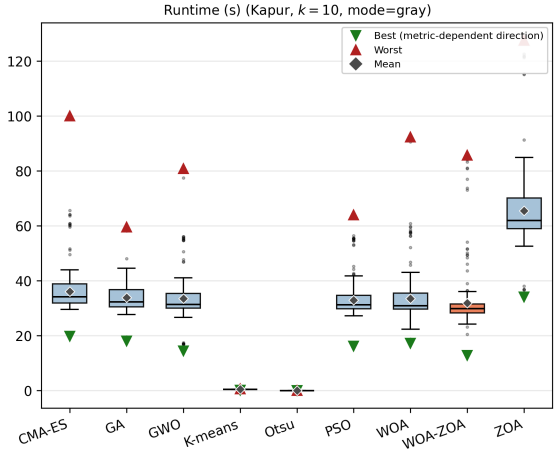
**Figure A.3** Boxplots for Kapur objective, $k = 10$ (grayscale).




**(a)** FSIM (gray)

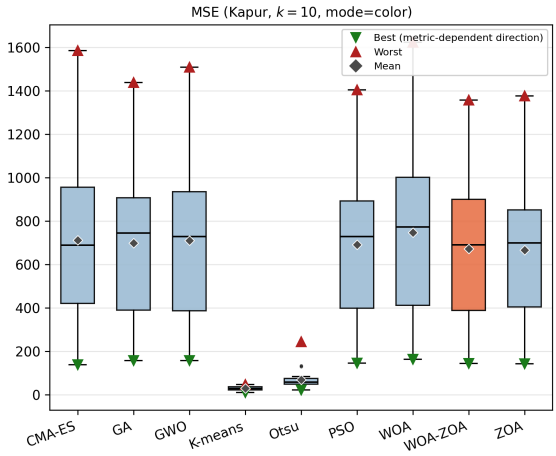



**(b)** QILV (gray)

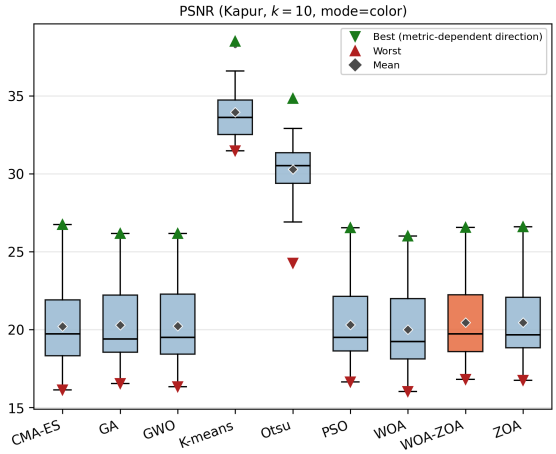



**(c)** Runtime (s, gray)

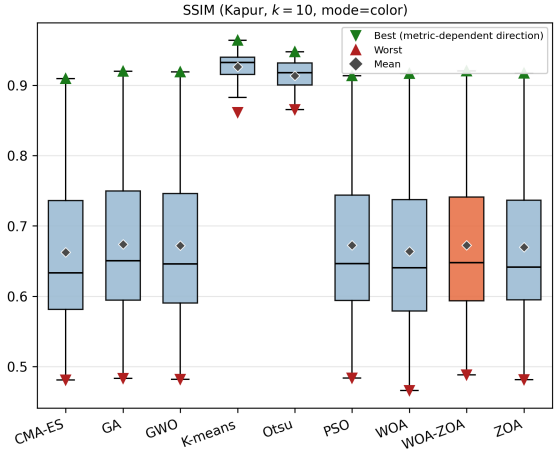
**Figure A.4** Boxplots for Kapur objective, $k = 10$ (grayscale).




**(a)** MSE (color)

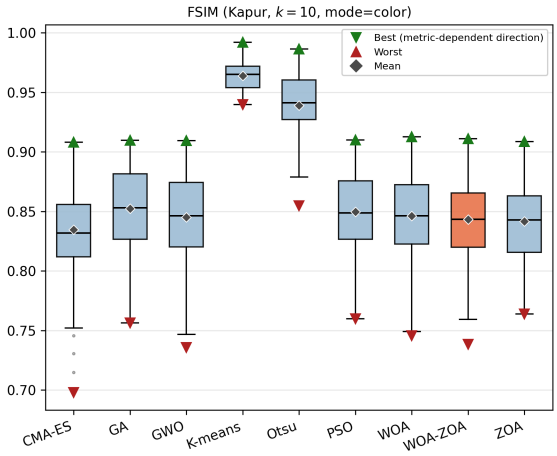



**(b)** PSNR (color)

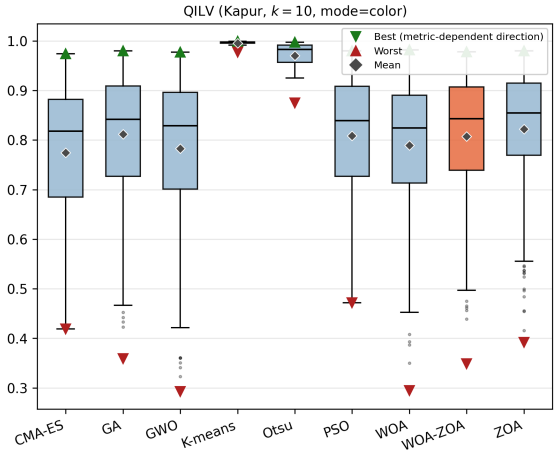



**(c)** SSIM (color)

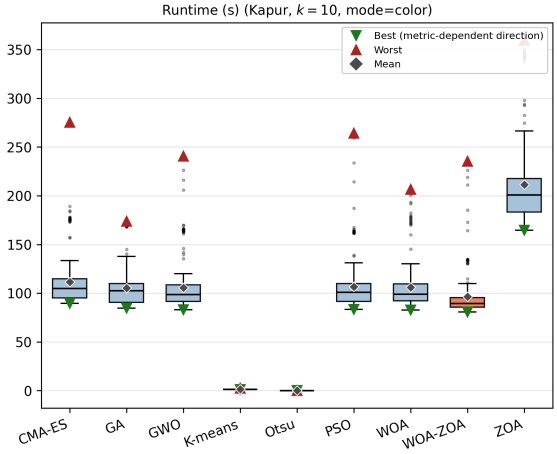
**Figure A.5** Boxplots for Kapur objective, $k = 10$ (color).




**(a)** FSIM (color)

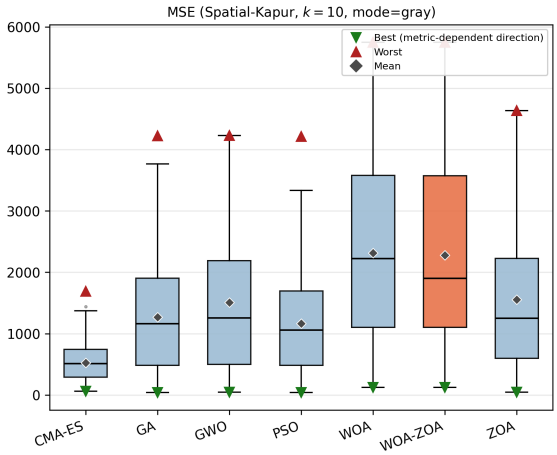



**(b)** QILV (color)

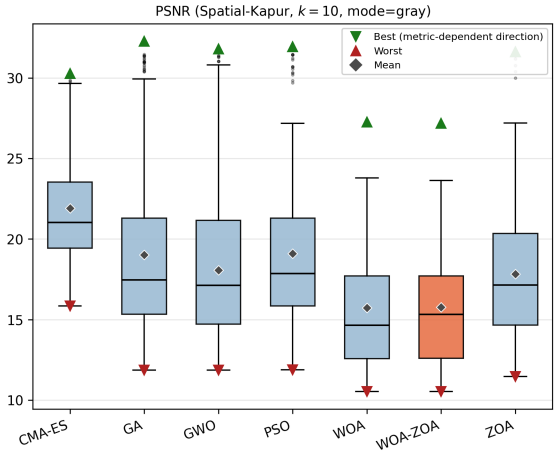



**(c)** Runtime (s, color)

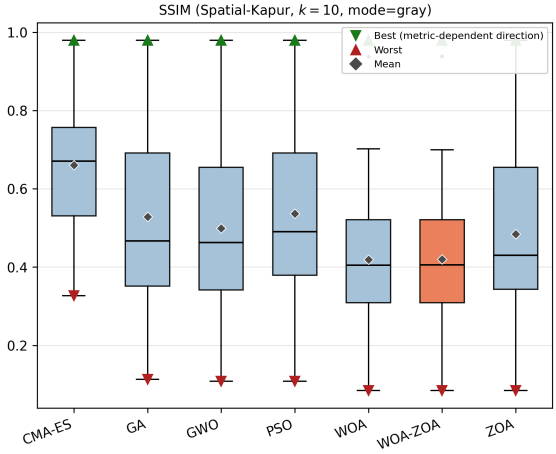
**Figure A.6** Boxplots for Kapur objective, $k = 10$ (color).




**(a)** MSE (gray)

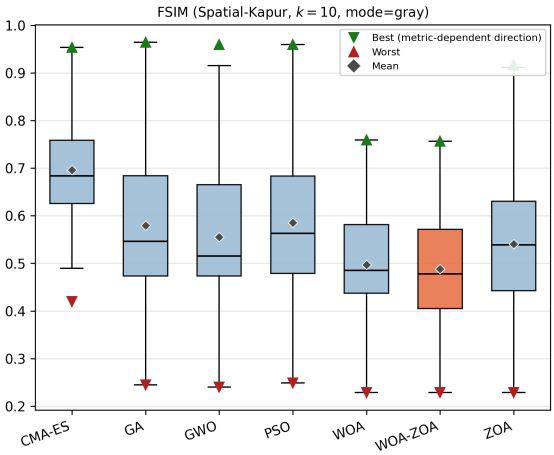



**(b)** PSNR (gray)

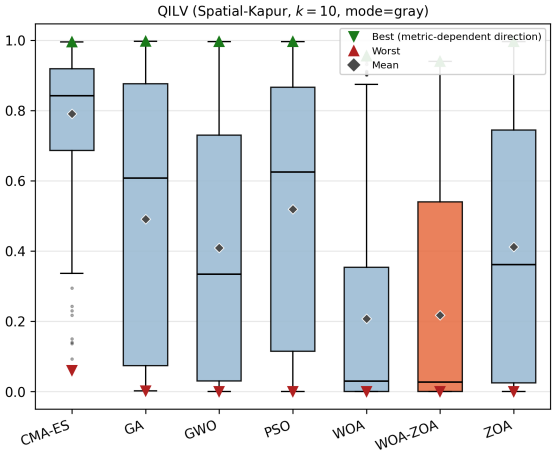



**(c)** SSIM (gray)

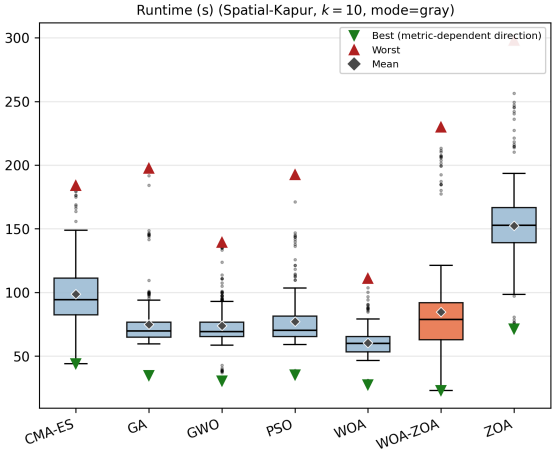
**Figure A.7** Boxplots for Kapur-Spatial objective, $k = 10$ (grayscale).




**(a)** FSIM (gray)

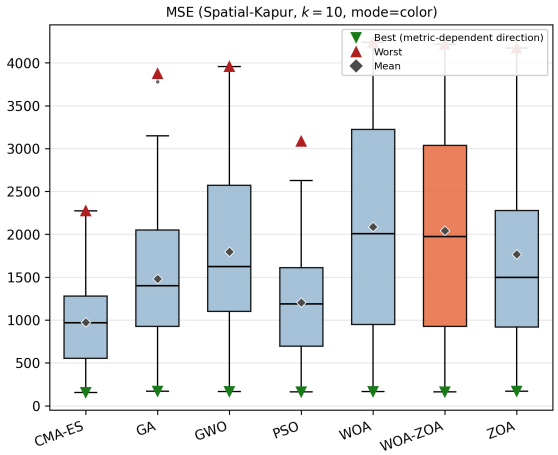



**(b)** QILV (gray)

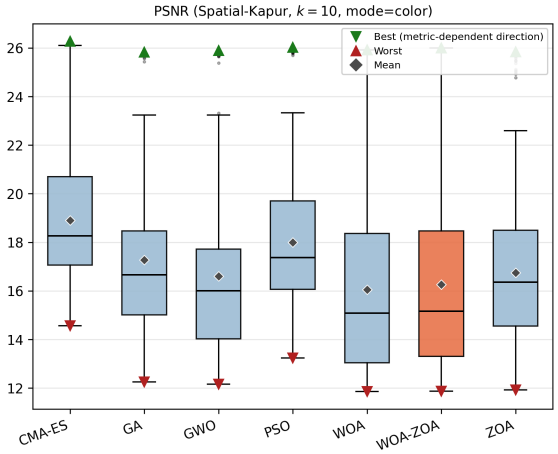



**(c)** Runtime (s, gray)

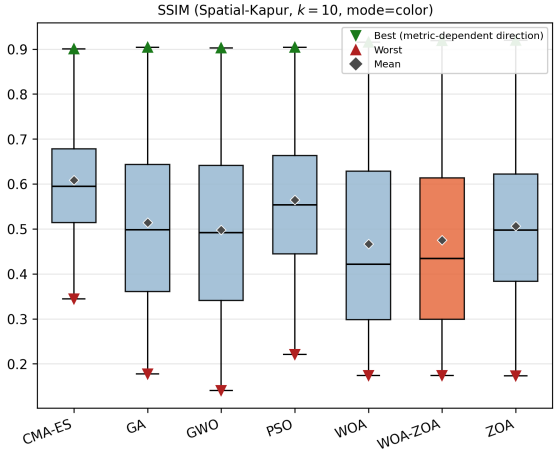
**Figure A.8** Boxplots for Kapur-Spatial objective, $k = 10$ (grayscale).




**(a)** MSE (color)

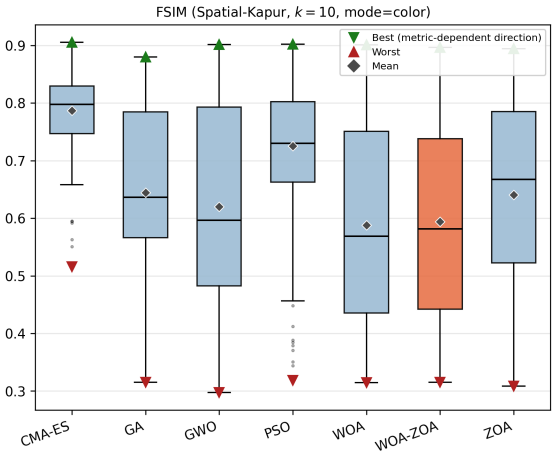



**(b)** PSNR (color)

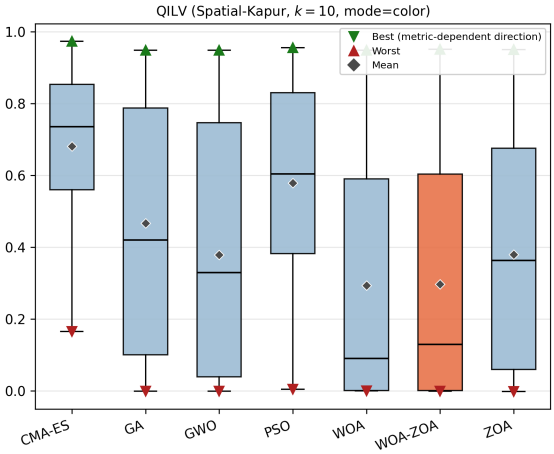



**(c)** SSIM (color)

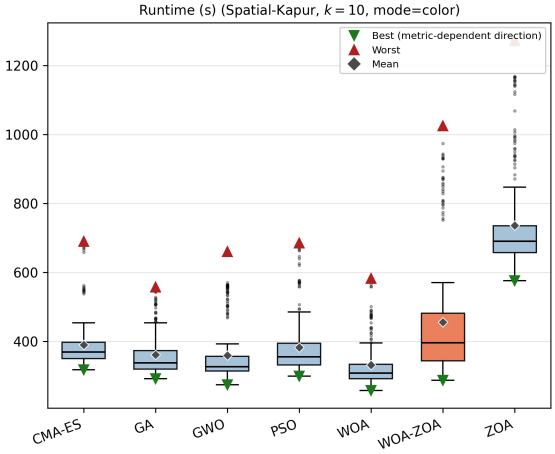
**Figure A.9** Boxplots for Kapur-Spatial objective, $k = 10$ (color).




**(a)** FSIM (color)

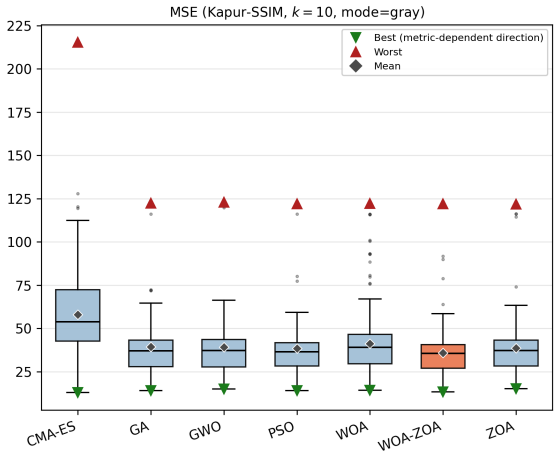



**(b)** QILV (color)

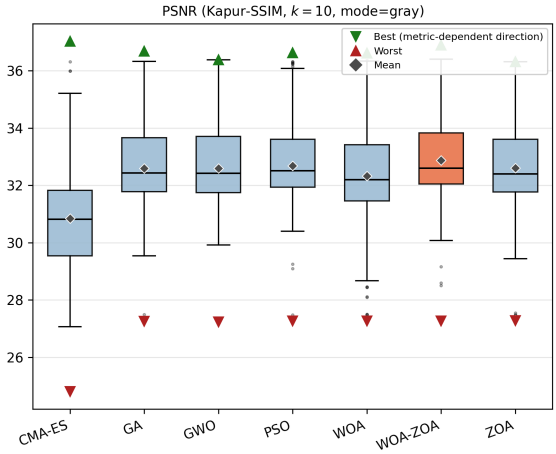



**(c)** Runtime (s, color)

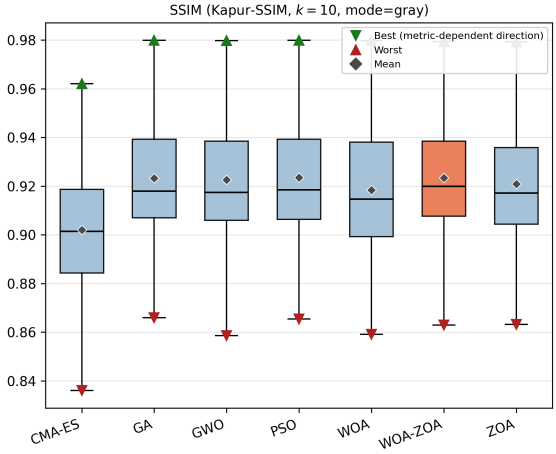
**Figure A.10** Boxplots for Kapur-Spatial objective, $k = 10$ (color).




**(a)** MSE (gray)

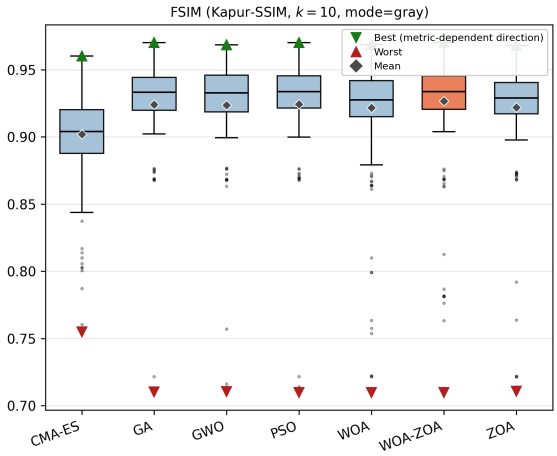



**(b)** PSNR (gray)

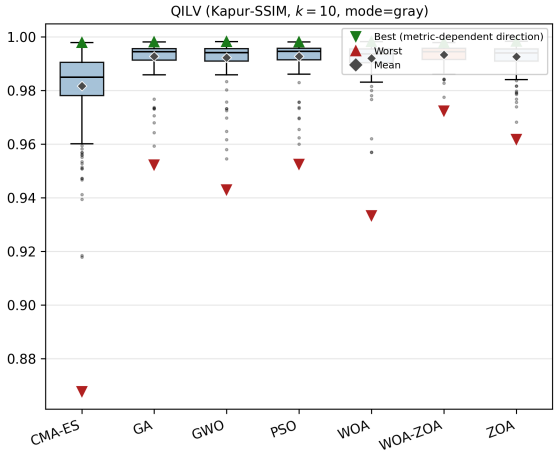



**(c)** SSIM (gray)

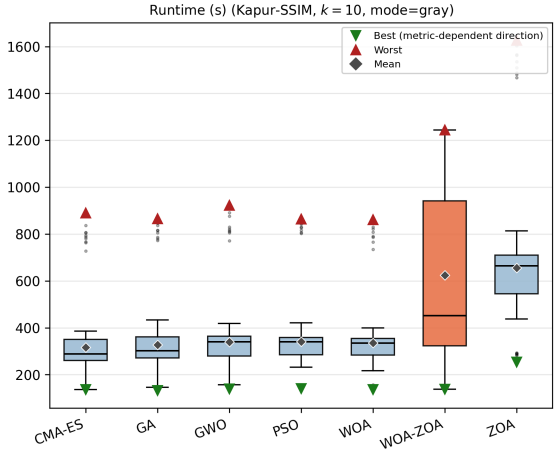
**Figure A.11** Boxplots for Kapur-SSIM objective, $k = 10$ (grayscale).




**(a)** FSIM (gray)

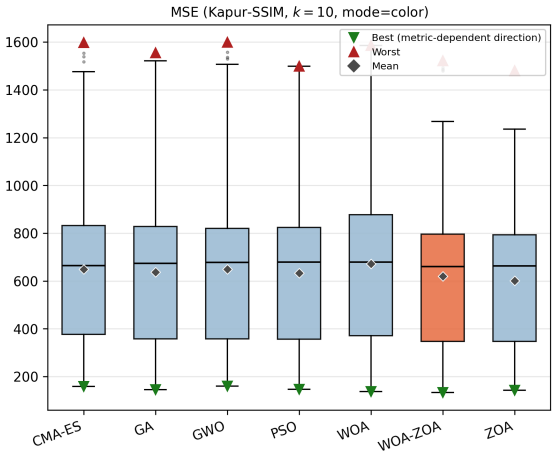



**(b)** QILV (gray)

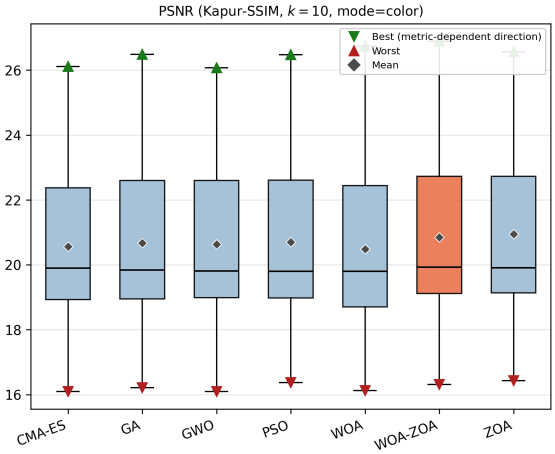



**(c)** Runtime (s, gray)

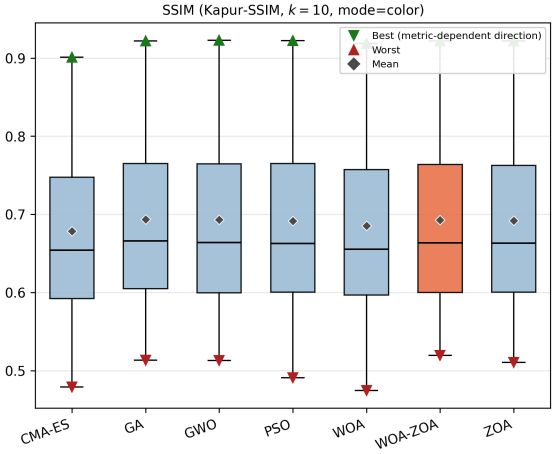
**Figure A.12** Boxplots for Kapur-SSIM objective, $k = 10$ (grayscale).




**(a)** MSE (color)

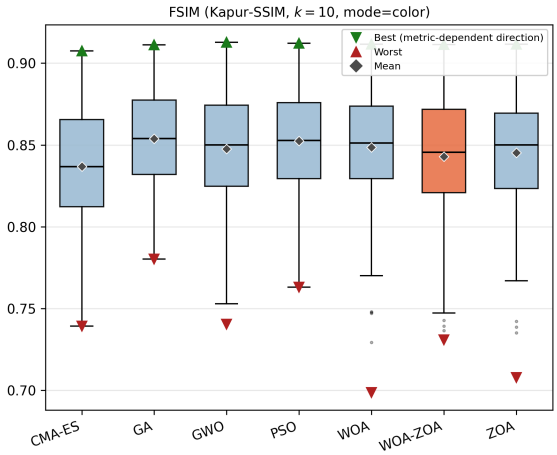



**(b)** PSNR (color)

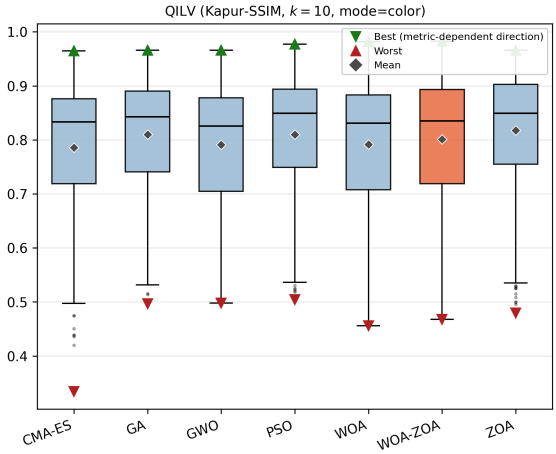



**(c)** SSIM (color)

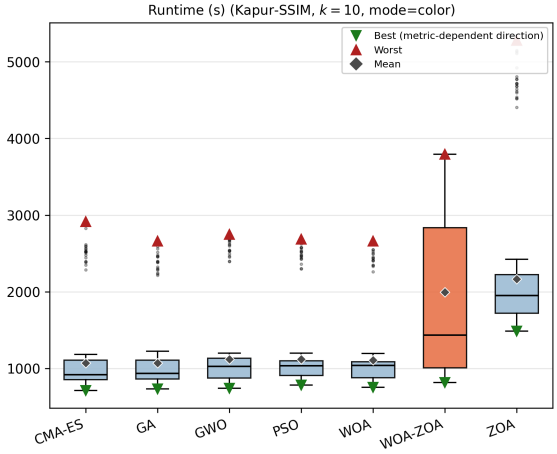
**Figure A.13** Boxplots for Kapur-SSIM objective, $k = 10$ (color).




**(a)** FSIM (color)




**(b)** QILV (color)




**(c)** Runtime (s, color)

**Figure A.14** Boxplots for Kapur-SSIM objective, $k = 10$ (color).

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 686.994 ± 323.010 | 20.334 ± 2.375 | 0.638 ± 0.106 | 0.793 ± 0.046 | 0.769 ± 0.146 |
| GA | 673.077 ± 314.412 | 20.429 ± 2.398 | 0.653 ± 0.105 | 0.817 ± 0.035 | 0.798 ± 0.138 |
| GWO | 658.309 ± 307.321 | 20.529 ± 2.410 | 0.656 ± 0.103 | 0.819 ± 0.036 | 0.802 ± 0.131 |
| K-means | **166.314** ± 64.040 | **26.342** ± 2.068 | **0.798** ± 0.057 | **0.854** ± 0.041 | **0.948** ± 0.033 |
| Otsu | 272.305 ± 126.084 | 24.367 ± 2.430 | 0.786 ± 0.059 | 0.837 ± 0.054 | 0.835 ± 0.150 |
| PSO | 663.909 ± 311.938 | 20.492 ± 2.409 | 0.655 ± 0.103 | 0.819 ± 0.034 | 0.803 ± 0.131 |
| WOA | 692.813 ± 334.777 | 20.326 ± 2.433 | 0.650 ± 0.105 | 0.814 ± 0.039 | 0.793 ± 0.139 |
| WOA-ZOA | 648.893 ± 300.869 | 20.587 ± 2.406 | 0.656 ± 0.102 | 0.818 ± 0.036 | 0.803 ± 0.131 |
| ZOA | 658.478 ± 294.918 | 20.491 ± 2.351 | 0.650 ± 0.102 | 0.813 ± 0.037 | 0.810 ± 0.125 |

**Table A.4** Mean ± std per algorithm (Kapur, $k = 4$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 143.591 ± 31.157 | 26.667 ± 0.977 | 0.832 ± 0.056 | 0.836 ± 0.046 | 0.953 ± 0.031 |
| GA | 118.790 ± 24.618 | 27.491 ± 1.014 | 0.827 ± 0.051 | 0.836 ± 0.045 | 0.971 ± 0.015 |
| GWO | 118.439 ± 20.999 | 27.480 ± 0.910 | 0.828 ± 0.051 | 0.837 ± 0.044 | 0.972 ± 0.014 |
| K-means | **82.254** ± 27.600 | **29.319** ± 1.932 | **0.866** ± 0.035 | **0.869** ± 0.031 | 0.970 ± 0.020 |
| Otsu | 180.055 ± 123.179 | 26.523 ± 3.018 | 0.847 ± 0.036 | 0.832 ± 0.045 | 0.859 ± 0.151 |
| PSO | 118.529 ± 22.995 | 27.494 ± 0.998 | 0.827 ± 0.051 | 0.837 ± 0.044 | 0.972 ± 0.015 |
| WOA | 117.939 ± 26.750 | 27.550 ± 1.155 | 0.827 ± 0.051 | 0.837 ± 0.043 | 0.971 ± 0.017 |
| WOA-ZOA | 115.577 ± 23.920 | 27.625 ± 1.120 | 0.827 ± 0.050 | 0.838 ± 0.042 | **0.973** ± 0.014 |
| ZOA | 117.947 ± 24.687 | 27.535 ± 1.103 | 0.826 ± 0.051 | 0.836 ± 0.043 | 0.972 ± 0.014 |

**Table A.5** Mean ± std per algorithm (Kapur, $k = 6$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 707.885 ± 341.325 | 20.221 ± 2.399 | 0.650 ± 0.105 | 0.817 ± 0.042 | 0.771 ± 0.145 |
| GA | 681.993 ± 321.497 | 20.387 ± 2.434 | 0.666 ± 0.102 | 0.840 ± 0.034 | 0.803 ± 0.134 |
| GWO | 685.251 ± 330.560 | 20.392 ± 2.483 | 0.664 ± 0.102 | 0.836 ± 0.037 | 0.798 ± 0.137 |
| K-means | **75.404** ± 27.746 | **29.744** ± 1.989 | **0.866** ± 0.040 | **0.919** ± 0.026 | **0.983** ± 0.013 |
| Otsu | 144.104 ± 88.708 | 27.164 ± 2.237 | 0.854 ± 0.037 | 0.891 ± 0.038 | 0.902 ± 0.122 |
| PSO | 681.446 ± 319.990 | 20.375 ± 2.390 | 0.665 ± 0.100 | 0.839 ± 0.033 | 0.801 ± 0.131 |
| WOA | 714.627 ± 339.610 | 20.180 ± 2.411 | 0.659 ± 0.105 | 0.832 ± 0.038 | 0.783 ± 0.155 |
| WOA-ZOA | 665.730 ± 310.493 | 20.488 ± 2.440 | 0.665 ± 0.101 | 0.835 ± 0.035 | 0.804 ± 0.133 |
| ZOA | 655.158 ± 298.138 | 20.543 ± 2.421 | 0.662 ± 0.101 | 0.832 ± 0.034 | 0.825 ± 0.116 |

**Table A.6** Mean ± std per algorithm (Kapur, $k = 6$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 100.180 ± 44.640 | 28.493 ± 1.798 | 0.848 ± 0.051 | 0.859 ± 0.043 | 0.974 ± 0.021 |
| GA | 68.062 ± 12.830 | 29.892 ± 0.927 | 0.868 ± 0.039 | 0.883 ± 0.038 | 0.988 ± 0.006 |
| GWO | 67.386 ± 12.742 | 29.936 ± 0.931 | 0.868 ± 0.039 | 0.882 ± 0.038 | 0.988 ± 0.007 |
| K-means | **46.607** ± 15.424 | **31.775** ± 1.895 | **0.902** ± 0.029 | **0.908** ± 0.029 | **0.989** ± 0.006 |
| Otsu | 140.921 ± 131.691 | 28.082 ± 3.431 | 0.883 ± 0.033 | 0.869 ± 0.045 | 0.876 ± 0.158 |
| PSO | 66.723 ± 12.325 | 29.977 ± 0.925 | 0.868 ± 0.039 | 0.883 ± 0.039 | 0.988 ± 0.006 |
| WOA | 71.514 ± 20.100 | 29.755 ± 1.236 | 0.866 ± 0.042 | 0.880 ± 0.041 | 0.986 ± 0.010 |
| WOA-ZOA | 66.989 ± 12.976 | 29.965 ± 0.949 | 0.868 ± 0.040 | 0.883 ± 0.039 | 0.988 ± 0.006 |
| ZOA | 69.531 ± 15.240 | 29.827 ± 1.051 | 0.865 ± 0.039 | 0.880 ± 0.038 | 0.987 ± 0.006 |

**Table A.7** Mean ± std per algorithm (Kapur, $k = 8$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 649.541 ± 313.842 | 20.564 ± 2.299 | 0.679 ± 0.101 | 0.837 ± 0.038 | 0.786 ± 0.137 |
| GA | 636.653 ± 300.368 | 20.670 ± 2.388 | 0.693 ± 0.102 | 0.854 ± 0.031 | 0.810 ± 0.119 |
| GWO | 648.859 ± 328.242 | 20.630 ± 2.438 | 0.693 ± 0.102 | 0.848 ± 0.034 | 0.791 ± 0.127 |
| K-means | **28.405** ± 9.935 | **33.938** ± 1.851 | **0.927** ± 0.023 | **0.964** ± 0.015 | **0.995** ± 0.005 |
| Otsu | 69.224 ± 43.610 | 30.290 ± 2.062 | 0.914 ± 0.024 | 0.939 ± 0.030 | 0.970 ± 0.029 |
| PSO | 632.814 ± 295.418 | 20.704 ± 2.421 | 0.691 ± 0.103 | 0.853 ± 0.030 | 0.810 ± 0.121 |
| WOA | 672.160 ± 336.278 | 20.486 ± 2.478 | 0.685 ± 0.104 | 0.849 ± 0.036 | 0.792 ± 0.125 |
| WOA-ZOA | 619.958 ± 317.022 | 20.847 ± 2.476 | 0.693 ± 0.102 | 0.843 ± 0.038 | 0.801 ± 0.125 |
| ZOA | 600.897 ± 287.790 | 20.949 ± 2.448 | 0.692 ± 0.101 | 0.845 ± 0.035 | 0.818 ± 0.117 |

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 710.505 ± 341.279 | 20.202 ± 2.384 | 0.658 ± 0.104 | 0.829 ± 0.040 | 0.778 ± 0.139 |
| GA | 696.011 ± 329.744 | 20.307 ± 2.457 | 0.670 ± 0.103 | 0.848 ± 0.033 | 0.807 ± 0.130 |
| GWO | 706.134 ± 344.737 | 20.275 ± 2.507 | 0.668 ± 0.104 | 0.841 ± 0.037 | 0.790 ± 0.142 |
| K-means | **43.272** ± 15.397 | **32.128** ± 1.911 | **0.904** ± 0.029 | **0.948** ± 0.019 | **0.991** ± 0.008 |
| Otsu | 94.273 ± 47.728 | 28.884 ± 2.064 | 0.890 ± 0.028 | 0.919 ± 0.036 | 0.959 ± 0.035 |
| PSO | 691.264 ± 322.914 | 20.324 ± 2.431 | 0.669 ± 0.102 | 0.845 ± 0.032 | 0.806 ± 0.129 |
| WOA | 733.891 ± 353.493 | 20.088 ± 2.472 | 0.663 ± 0.107 | 0.839 ± 0.038 | 0.781 ± 0.151 |
| WOA-ZOA | 673.866 ± 317.894 | 20.452 ± 2.478 | 0.670 ± 0.102 | 0.842 ± 0.033 | 0.814 ± 0.127 |
| ZOA | 663.704 ± 297.740 | 20.472 ± 2.396 | 0.668 ± 0.101 | 0.840 ± 0.031 | 0.823 ± 0.119 |

**Table A.8** Mean ± std per algorithm (Kapur, $k = 8$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 74.454 ± 36.281 | 29.809 ± 1.846 | 0.874 ± 0.045 | 0.882 ± 0.042 | 0.982 ± 0.017 |
| GA | 45.178 ± 8.359 | 31.664 ± 0.870 | 0.894 ± 0.034 | 0.910 ± 0.038 | **0.993** ± 0.004 |
| GWO | 46.643 ± 8.778 | 31.529 ± 0.898 | 0.892 ± 0.035 | 0.908 ± 0.037 | 0.992 ± 0.004 |
| K-means | **30.342** ± 9.957 | **33.630** ± 1.857 | **0.926** ± 0.025 | **0.932** ± 0.027 | **0.993** ± 0.004 |
| Otsu | 98.385 ± 120.779 | 29.904 ± 3.544 | 0.909 ± 0.034 | 0.894 ± 0.045 | 0.921 ± 0.126 |
| PSO | 44.852 ± 7.797 | 31.685 ± 0.812 | 0.894 ± 0.034 | 0.910 ± 0.039 | **0.993** ± 0.003 |
| WOA | 50.858 ± 14.637 | 31.235 ± 1.218 | 0.889 ± 0.038 | 0.905 ± 0.039 | 0.991 ± 0.008 |
| WOA-ZOA | 44.684 ± 8.535 | 31.724 ± 0.960 | 0.894 ± 0.034 | 0.910 ± 0.037 | **0.993** ± 0.003 |
| ZOA | 47.704 ± 10.774 | 31.458 ± 1.003 | 0.890 ± 0.033 | 0.906 ± 0.036 | 0.992 ± 0.004 |

**Table A.9** Mean ± std per algorithm (Kapur, $k = 10$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 711.565 ± 344.353 | 20.206 ± 2.408 | 0.663 ± 0.104 | 0.834 ± 0.038 | 0.775 ± 0.144 |
| GA | 698.383 ± 330.091 | 20.294 ± 2.461 | 0.674 ± 0.103 | 0.852 ± 0.034 | 0.812 ± 0.126 |
| GWO | 710.430 ± 343.386 | 20.238 ± 2.489 | 0.672 ± 0.104 | 0.845 ± 0.037 | 0.783 ± 0.150 |
| K-means | **28.405** ± 9.935 | **33.938** ± 1.851 | **0.927** ± 0.023 | **0.964** ± 0.015 | **0.995** ± 0.005 |
| Otsu | 69.224 ± 43.610 | 30.290 ± 2.062 | 0.914 ± 0.024 | 0.939 ± 0.030 | 0.970 ± 0.029 |
| PSO | 691.648 ± 322.601 | 20.311 ± 2.399 | 0.673 ± 0.103 | 0.850 ± 0.033 | 0.809 ± 0.122 |
| WOA | 747.395 ± 355.653 | 20.011 ± 2.492 | 0.664 ± 0.107 | 0.846 ± 0.036 | 0.789 ± 0.141 |
| WOA-ZOA | 672.457 ± 314.553 | 20.455 ± 2.466 | 0.672 ± 0.102 | 0.843 ± 0.034 | 0.807 ± 0.129 |
| ZOA | 665.324 ± 295.469 | 20.453 ± 2.384 | 0.670 ± 0.101 | 0.842 ± 0.032 | 0.822 ± 0.122 |

**Table A.10** Mean ± std per algorithm (Kapur, $k = 10$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 2605.316 ± 1337.909 | 14.744 ± 2.907 | 0.396 ± 0.178 | 0.482 ± 0.112 | 0.014 ± 0.039 |
| GA | 2627.389 ± 1352.660 | 14.706 ± 2.901 | 0.394 ± 0.180 | 0.486 ± 0.116 | 0.010 ± 0.023 |
| GWO | 2627.640 ± 1352.860 | 14.705 ± 2.901 | 0.394 ± 0.180 | 0.486 ± 0.117 | 0.010 ± 0.023 |
| K-means | **724.945** ± 290.801 | **19.974** ± 2.180 | 0.624 ± 0.101 | **0.628** ± 0.063 | **0.583** ± 0.175 |
| Otsu | 725.898 ± 290.743 | 19.968 ± 2.181 | **0.625** ± 0.101 | **0.628** ± 0.063 | 0.576 ± 0.172 |
| PSO | 2627.350 ± 1352.548 | 14.706 ± 2.901 | 0.394 ± 0.180 | 0.486 ± 0.116 | 0.010 ± 0.023 |
| WOA | 2627.792 ± 1353.010 | 14.705 ± 2.901 | 0.394 ± 0.180 | 0.485 ± 0.117 | 0.010 ± 0.023 |
| WOA-ZOA | 2627.527 ± 1352.733 | 14.706 ± 2.901 | 0.394 ± 0.180 | 0.486 ± 0.117 | 0.010 ± 0.023 |
| ZOA | 2627.748 ± 1352.964 | 14.705 ± 2.901 | 0.394 ± 0.180 | 0.485 ± 0.117 | 0.010 ± 0.023 |

**Table A.11** Mean ± std per algorithm (Kapur-Spatial, $k = 2$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 1144.869 ± 980.058 | 19.403 ± 4.158 | 0.530 ± 0.253 | 0.578 ± 0.154 | 0.527 ± 0.382 |
| GA | 2670.917 ± 1334.878 | 14.722 ± 3.142 | 0.408 ± 0.199 | 0.506 ± 0.141 | 0.064 ± 0.192 |
| GWO | 2838.392 ± 1647.069 | 14.506 ± 3.152 | 0.404 ± 0.195 | 0.506 ± 0.143 | 0.047 ± 0.166 |
| K-means | **664.627** ± 298.632 | **20.437** ± 2.290 | 0.627 ± 0.113 | 0.695 ± 0.070 | **0.685** ± 0.175 |
| Otsu | 664.802 ± 298.618 | 20.435 ± 2.289 | **0.628** ± 0.113 | **0.696** ± 0.070 | 0.682 ± 0.176 |
| PSO | 2659.771 ± 1331.065 | 14.739 ± 3.138 | 0.408 ± 0.199 | 0.505 ± 0.141 | 0.071 ± 0.197 |
| WOA | 3300.933 ± 2386.957 | 13.931 ± 3.044 | 0.390 ± 0.185 | 0.504 ± 0.144 | 0.026 ± 0.093 |
| WOA-ZOA | 3317.425 ± 2378.144 | 13.878 ± 2.978 | 0.390 ± 0.184 | 0.508 ± 0.145 | 0.010 ± 0.036 |
| ZOA | 3314.941 ± 2378.636 | 13.882 ± 2.979 | 0.389 ± 0.184 | 0.507 ± 0.145 | 0.010 ± 0.036 |

**Table A.12** Mean ± std per algorithm (Kapur-Spatial, $k = 2$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 2171.083 ± 737.759 | 15.014 ± 1.473 | 0.362 ± 0.137 | 0.459 ± 0.056 | 0.390 ± 0.229 |
| GA | 2116.505 ± 1319.881 | 15.857 ± 3.139 | 0.430 ± 0.181 | 0.505 ± 0.116 | 0.234 ± 0.297 |
| GWO | 2436.237 ± 1445.269 | 15.292 ± 3.314 | 0.407 ± 0.188 | 0.490 ± 0.120 | 0.132 ± 0.250 |
| K-means | **180.097** ± 63.097 | **25.954** ± 2.067 | **0.795** ± 0.048 | **0.794** ± 0.032 | **0.918** ± 0.036 |
| Otsu | 262.585 ± 122.561 | 24.539 ± 2.566 | 0.784 ± 0.054 | 0.778 ± 0.045 | 0.822 ± 0.150 |
| PSO | 2207.531 ± 1397.237 | 15.728 ± 3.240 | 0.424 ± 0.186 | 0.499 ± 0.121 | 0.221 ± 0.290 |
| WOA | 2584.028 ± 1376.356 | 14.860 ± 3.060 | 0.398 ± 0.180 | 0.487 ± 0.119 | 0.053 ± 0.142 |
| WOA-ZOA | 2598.861 ± 1370.658 | 14.811 ± 3.016 | 0.396 ± 0.179 | 0.483 ± 0.120 | 0.035 ± 0.087 |
| ZOA | 2593.069 ± 1366.925 | 14.821 ± 3.018 | 0.397 ± 0.179 | 0.487 ± 0.119 | 0.040 ± 0.100 |

**Table A.13** Mean ± std per algorithm (Kapur-Spatial, $k = 4$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 1497.339 ± 769.336 | 17.162 ± 2.936 | 0.511 ± 0.164 | 0.656 ± 0.102 | 0.464 ± 0.290 |
| GA | 2180.043 ± 1220.534 | 15.802 ± 3.522 | 0.444 ± 0.202 | 0.540 ± 0.145 | 0.243 ± 0.318 |
| GWO | 2462.633 ± 1386.367 | 15.425 ± 3.864 | 0.424 ± 0.208 | 0.522 ± 0.147 | 0.192 ± 0.323 |
| K-means | **166.314** ± 64.040 | **26.342** ± 2.068 | **0.798** ± 0.057 | **0.854** ± 0.041 | **0.948** ± 0.033 |
| Otsu | 272.305 ± 126.084 | 24.367 ± 2.430 | 0.786 ± 0.059 | 0.837 ± 0.054 | 0.835 ± 0.150 |
| PSO | 2077.402 ± 1129.704 | 15.949 ± 3.407 | 0.450 ± 0.199 | 0.545 ± 0.143 | 0.267 ± 0.317 |
| WOA | 2557.539 ± 1374.990 | 15.198 ± 3.782 | 0.419 ± 0.207 | 0.511 ± 0.144 | 0.154 ± 0.299 |
| WOA-ZOA | 2537.097 ± 1408.073 | 15.341 ± 3.993 | 0.420 ± 0.210 | 0.514 ± 0.146 | 0.170 ± 0.319 |
| ZOA | 2386.466 ± 1364.577 | 15.590 ± 3.909 | 0.431 ± 0.210 | 0.521 ± 0.153 | 0.204 ± 0.330 |

**Table A.14** Mean ± std per algorithm (Kapur-Spatial, $k = 4$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 1223.055 ± 673.797 | 17.876 ± 2.371 | 0.527 ± 0.106 | 0.598 ± 0.074 | 0.654 ± 0.227 |
| GA | 1795.899 ± 1240.021 | 16.709 ± 3.305 | 0.458 ± 0.193 | 0.526 ± 0.124 | 0.328 ± 0.338 |
| GWO | 2274.880 ± 1437.408 | 15.656 ± 3.355 | 0.419 ± 0.185 | 0.498 ± 0.118 | 0.190 ± 0.290 |
| K-means | **82.254** ± 27.600 | **29.319** ± 1.932 | **0.866** ± 0.035 | **0.869** ± 0.031 | **0.970** ± 0.020 |
| Otsu | 180.055 ± 123.179 | 26.523 ± 3.018 | 0.847 ± 0.036 | 0.832 ± 0.045 | 0.859 ± 0.151 |
| PSO | 1809.588 ± 1273.449 | 16.704 ± 3.332 | 0.456 ± 0.195 | 0.524 ± 0.125 | 0.329 ± 0.340 |
| WOA | 2486.248 ± 1463.302 | 15.249 ± 3.430 | 0.407 ± 0.189 | 0.491 ± 0.121 | 0.119 ± 0.249 |
| WOA-ZOA | 2467.920 ± 1471.740 | 15.290 ± 3.427 | 0.407 ± 0.189 | 0.485 ± 0.121 | 0.125 ± 0.256 |
| ZOA | 2161.172 ± 1442.533 | 15.929 ± 3.385 | 0.427 ± 0.191 | 0.499 ± 0.124 | 0.242 ± 0.312 |

**Table A.15** Mean ± std per algorithm (Kapur-Spatial, $k = 6$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 1128.865 ± 542.377 | 18.258 ± 2.636 | 0.565 ± 0.142 | 0.730 ± 0.087 | 0.611 ± 0.244 |
| GA | 1841.506 ± 1033.099 | 16.452 ± 3.310 | 0.475 ± 0.196 | 0.581 ± 0.141 | 0.355 ± 0.330 |
| GWO | 2143.698 ± 1203.982 | 15.909 ± 3.581 | 0.452 ± 0.209 | 0.548 ± 0.160 | 0.254 ± 0.332 |
| K-means | **75.404** ± 27.746 | **29.744** ± 1.989 | **0.866** ± 0.040 | **0.919** ± 0.026 | **0.983** ± 0.013 |
| Otsu | 144.104 ± 88.708 | 27.164 ± 2.237 | 0.854 ± 0.037 | 0.891 ± 0.038 | 0.902 ± 0.122 |
| PSO | 1698.089 ± 936.488 | 16.734 ± 3.157 | 0.492 ± 0.190 | 0.611 ± 0.137 | 0.420 ± 0.329 |
| WOA | 2436.864 ± 1339.546 | 15.402 ± 3.744 | 0.429 ± 0.204 | 0.531 ± 0.150 | 0.179 ± 0.302 |
| WOA-ZOA | 2373.819 ± 1362.742 | 15.645 ± 3.963 | 0.433 ± 0.209 | 0.533 ± 0.154 | 0.216 ± 0.342 |
| ZOA | 2054.368 ± 1200.266 | 16.194 ± 3.756 | 0.461 ± 0.205 | 0.567 ± 0.158 | 0.302 ± 0.348 |

**Table A.16** Mean ± std per algorithm (Kapur-Spatial, $k = 6$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 730.277 ± 367.781 | 20.189 ± 2.734 | 0.600 ± 0.144 | 0.648 ± 0.101 | 0.691 ± 0.237 |
| GA | 1487.268 ± 1019.203 | 17.774 ± 3.983 | 0.492 ± 0.208 | 0.550 ± 0.142 | 0.404 ± 0.357 |
| GWO | 1772.983 ± 1251.712 | 17.052 ± 3.990 | 0.463 ± 0.209 | 0.527 ± 0.142 | 0.324 ± 0.358 |
| K-means | **46.607** ± 15.424 | **31.775** ± 1.895 | **0.902** ± 0.029 | **0.908** ± 0.029 | **0.989** ± 0.006 |
| Otsu | 140.921 ± 131.691 | 28.082 ± 3.431 | 0.883 ± 0.033 | 0.869 ± 0.045 | 0.876 ± 0.158 |
| PSO | 1487.797 ± 1012.975 | 17.757 ± 3.946 | 0.492 ± 0.207 | 0.549 ± 0.140 | 0.412 ± 0.359 |
| WOA | 2402.137 ± 1485.843 | 15.486 ± 3.575 | 0.412 ± 0.190 | 0.493 ± 0.122 | 0.162 ± 0.286 |
| WOA-ZOA | 2321.963 ± 1492.763 | 15.615 ± 3.438 | 0.416 ± 0.187 | 0.484 ± 0.123 | 0.190 ± 0.297 |
| ZOA | 1725.708 ± 1243.686 | 17.148 ± 3.913 | 0.465 ± 0.214 | 0.527 ± 0.148 | 0.351 ± 0.360 |

**Table A.17** Mean ± std per algorithm (Kapur-Spatial, $k = 8$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 1026.058 ± 482.811 | 18.640 ± 2.547 | 0.588 ± 0.132 | 0.761 ± 0.077 | 0.651 ± 0.224 |
| GA | 1613.747 ± 894.527 | 16.948 ± 3.131 | 0.495 ± 0.195 | 0.613 ± 0.145 | 0.416 ± 0.336 |
| GWO | 1933.010 ± 1073.463 | 16.293 ± 3.438 | 0.476 ± 0.204 | 0.588 ± 0.161 | 0.327 ± 0.343 |
| K-means | **43.272** ± 15.397 | **32.128** ± 1.911 | **0.904** ± 0.029 | **0.948** ± 0.019 | **0.991** ± 0.008 |
| Otsu | 94.273 ± 47.728 | 28.884 ± 2.064 | 0.890 ± 0.028 | 0.919 ± 0.036 | 0.959 ± 0.035 |
| PSO | 1427.236 ± 763.592 | 17.412 ± 2.994 | 0.529 ± 0.175 | 0.674 ± 0.121 | 0.493 ± 0.314 |
| WOA | 2300.109 ± 1311.154 | 15.663 ± 3.697 | 0.443 ± 0.205 | 0.552 ± 0.159 | 0.223 ± 0.329 |
| WOA-ZOA | 2177.723 ± 1297.537 | 16.004 ± 3.873 | 0.455 ± 0.210 | 0.563 ± 0.163 | 0.275 ± 0.358 |
| ZOA | 1854.717 ± 1132.583 | 16.606 ± 3.635 | 0.489 ± 0.195 | 0.612 ± 0.156 | 0.356 ± 0.341 |

**Table A.18** Mean ± std per algorithm (Kapur-Spatial, $k = 8$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 526.049 ± 307.176 | 21.912 ± 3.281 | 0.661 ± 0.154 | 0.696 ± 0.112 | 0.791 ± 0.189 |
| GA | 1270.498 ± 959.830 | 19.023 ± 4.978 | 0.528 ± 0.228 | 0.579 ± 0.167 | 0.491 ± 0.372 |
| GWO | 1507.161 ± 1107.461 | 18.064 ± 4.652 | 0.499 ± 0.221 | 0.555 ± 0.157 | 0.409 ± 0.375 |
| K-means | **30.342** ± 9.957 | **33.630** ± 1.857 | **0.926** ± 0.025 | **0.932** ± 0.027 | **0.993** ± 0.004 |
| Otsu | 98.385 ± 120.779 | 29.904 ± 3.544 | 0.909 ± 0.034 | 0.894 ± 0.045 | 0.921 ± 0.126 |
| PSO | 1164.527 ± 839.725 | 19.091 ± 4.466 | 0.537 ± 0.215 | 0.585 ± 0.156 | 0.519 ± 0.358 |
| WOA | 2315.867 ± 1503.434 | 15.729 ± 3.699 | 0.419 ± 0.191 | 0.497 ± 0.124 | 0.207 ± 0.314 |
| WOA-ZOA | 2276.873 ± 1510.383 | 15.778 ± 3.579 | 0.420 ± 0.189 | 0.488 ± 0.123 | 0.218 ± 0.316 |
| ZOA | 1552.772 ± 1165.382 | 17.824 ± 4.378 | 0.485 ± 0.222 | 0.541 ± 0.157 | 0.412 ± 0.378 |

**Table A.19** Mean ± std per algorithm (Kapur-Spatial, $k = 10$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 972.315 ± 469.269 | 18.900 ± 2.587 | 0.609 ± 0.125 | 0.787 ± 0.069 | 0.681 ± 0.215 |
| GA | 1480.425 ± 810.252 | 17.274 ± 3.030 | 0.514 ± 0.192 | 0.644 ± 0.145 | 0.466 ± 0.336 |
| GWO | 1797.621 ± 1045.072 | 16.606 ± 3.376 | 0.498 ± 0.202 | 0.620 ± 0.167 | 0.378 ± 0.344 |
| K-means | **28.405** ± 9.935 | **33.938** ± 1.851 | **0.927** ± 0.023 | **0.964** ± 0.015 | **0.995** ± 0.005 |
| Otsu | 69.224 ± 43.610 | 30.290 ± 2.062 | 0.914 ± 0.024 | 0.939 ± 0.030 | 0.970 ± 0.029 |
| PSO | 1206.209 ± 584.292 | 17.998 ± 2.713 | 0.565 ± 0.158 | 0.725 ± 0.105 | 0.578 ± 0.277 |
| WOA | 2085.648 ± 1255.834 | 16.054 ± 3.528 | 0.467 ± 0.208 | 0.588 ± 0.178 | 0.293 ± 0.342 |
| WOA-ZOA | 2041.237 ± 1249.401 | 16.255 ± 3.764 | 0.475 ± 0.205 | 0.594 ± 0.170 | 0.297 ± 0.343 |
| ZOA | 1767.672 ± 1095.299 | 16.754 ± 3.487 | 0.506 ± 0.187 | 0.640 ± 0.153 | 0.380 ± 0.330 |

**Table A.20** Mean ± std per algorithm (Kapur-Spatial, $k = 10$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 798.299 ± 316.385 | 19.464 ± 1.806 | **0.641** ± 0.087 | **0.637** ± 0.057 | 0.539 ± 0.184 |
| GA | 798.177 ± 315.915 | 19.464 ± 1.803 | **0.641** ± 0.087 | **0.637** ± 0.057 | 0.540 ± 0.184 |
| GWO | 798.302 ± 316.087 | 19.464 ± 1.805 | **0.641** ± 0.087 | **0.637** ± 0.057 | 0.539 ± 0.184 |
| K-means | **724.945** ± 290.801 | **19.974** ± 2.180 | 0.624 ± 0.101 | 0.628 ± 0.063 | **0.583** ± 0.175 |
| Otsu | 725.898 ± 290.743 | 19.968 ± 2.181 | 0.625 ± 0.101 | 0.628 ± 0.063 | 0.576 ± 0.172 |
| PSO | 798.302 ± 316.087 | 19.464 ± 1.805 | **0.641** ± 0.087 | **0.637** ± 0.057 | 0.539 ± 0.184 |
| WOA | 798.302 ± 316.087 | 19.464 ± 1.805 | **0.641** ± 0.087 | **0.637** ± 0.057 | 0.539 ± 0.184 |
| WOA-ZOA | 798.302 ± 316.087 | 19.464 ± 1.805 | **0.641** ± 0.087 | **0.637** ± 0.057 | 0.539 ± 0.184 |
| ZOA | 798.302 ± 316.087 | 19.464 ± 1.805 | **0.641** ± 0.087 | **0.637** ± 0.057 | 0.539 ± 0.184 |

**Table A.21** Mean ± std per algorithm (Kapur-SSIM, $k = 2$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 1069.814 ± 553.609 | 18.351 ± 2.048 | 0.574 ± 0.047 | 0.690 ± 0.056 | 0.617 ± 0.193 |
| GA | 820.482 ± 398.454 | 19.529 ± 2.248 | 0.634 ± 0.111 | 0.713 ± 0.065 | 0.620 ± 0.186 |
| GWO | 829.311 ± 404.322 | 19.492 ± 2.272 | **0.635** ± 0.110 | 0.713 ± 0.065 | 0.612 ± 0.189 |
| K-means | **664.627** ± 298.632 | **20.437** ± 2.290 | 0.627 ± 0.113 | 0.695 ± 0.070 | **0.685** ± 0.175 |
| Otsu | 664.802 ± 298.618 | 20.435 ± 2.289 | 0.628 ± 0.113 | 0.696 ± 0.070 | 0.682 ± 0.176 |
| PSO | 825.003 ± 403.428 | 19.514 ± 2.265 | **0.635** ± 0.110 | 0.713 ± 0.064 | 0.614 ± 0.188 |
| WOA | 830.897 ± 406.998 | 19.485 ± 2.268 | 0.634 ± 0.110 | **0.714** ± 0.064 | 0.614 ± 0.188 |
| WOA-ZOA | 828.721 ± 405.056 | 19.498 ± 2.276 | **0.635** ± 0.110 | 0.713 ± 0.065 | 0.612 ± 0.189 |
| ZOA | 820.430 ± 400.304 | 19.534 ± 2.257 | **0.635** ± 0.110 | **0.714** ± 0.064 | 0.618 ± 0.186 |

**Table A.22** Mean ± std per algorithm (Kapur-SSIM, $k = 2$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 223.883 ± 77.750 | 24.853 ± 1.338 | 0.781 ± 0.018 | 0.790 ± 0.032 | 0.884 ± 0.030 |
| GA | 209.213 ± 76.303 | 25.207 ± 1.598 | **0.810** ± 0.049 | **0.798** ± 0.037 | 0.910 ± 0.039 |
| GWO | 213.457 ± 77.432 | 25.125 ± 1.619 | **0.810** ± 0.048 | **0.798** ± 0.036 | 0.908 ± 0.041 |
| K-means | **180.097** ± 63.097 | **25.954** ± 2.067 | 0.795 ± 0.048 | 0.794 ± 0.032 | **0.918** ± 0.036 |
| Otsu | 262.585 ± 122.561 | 24.539 ± 2.566 | 0.784 ± 0.054 | 0.778 ± 0.045 | 0.822 ± 0.150 |
| PSO | 212.321 ± 75.297 | 25.124 ± 1.533 | 0.809 ± 0.049 | **0.798** ± 0.037 | 0.909 ± 0.039 |
| WOA | 215.174 ± 77.147 | 25.081 ± 1.587 | 0.809 ± 0.048 | 0.797 ± 0.037 | 0.908 ± 0.044 |
| WOA-ZOA | 212.345 ± 77.239 | 25.147 ± 1.615 | **0.810** ± 0.048 | **0.798** ± 0.036 | 0.909 ± 0.041 |
| ZOA | 211.864 ± 77.432 | 25.160 ± 1.624 | **0.810** ± 0.048 | **0.798** ± 0.037 | 0.908 ± 0.040 |

**Table A.23** Mean ± std per algorithm (Kapur-SSIM, $k = 4$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 645.081 ± 315.858 | 20.607 ± 2.333 | 0.661 ± 0.103 | 0.801 ± 0.044 | 0.764 ± 0.148 |
| GA | 649.097 ± 340.714 | 20.652 ± 2.464 | 0.676 ± 0.103 | 0.820 ± 0.033 | 0.773 ± 0.142 |
| GWO | 644.199 ± 340.687 | 20.679 ± 2.433 | 0.679 ± 0.103 | 0.817 ± 0.033 | 0.769 ± 0.143 |
| K-means | **166.314** ± 64.040 | **26.342** ± 2.068 | **0.798** ± 0.057 | **0.854** ± 0.041 | **0.948** ± 0.033 |
| Otsu | 272.305 ± 126.084 | 24.367 ± 2.430 | 0.786 ± 0.059 | 0.837 ± 0.054 | 0.835 ± 0.150 |
| PSO | 644.710 ± 341.018 | 20.680 ± 2.445 | 0.678 ± 0.103 | 0.820 ± 0.033 | 0.775 ± 0.143 |
| WOA | 662.297 ± 349.811 | 20.570 ± 2.473 | 0.675 ± 0.103 | 0.816 ± 0.034 | 0.766 ± 0.142 |
| WOA-ZOA | 637.930 ± 337.072 | 20.719 ± 2.433 | 0.679 ± 0.102 | 0.817 ± 0.033 | 0.770 ± 0.140 |
| ZOA | 629.218 ± 332.647 | 20.788 ± 2.459 | 0.676 ± 0.102 | 0.817 ± 0.032 | 0.788 ± 0.134 |

**Table A.24** Mean ± std per algorithm (Kapur-SSIM, $k = 4$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 100.627 ± 50.008 | 28.724 ± 2.416 | 0.867 ± 0.042 | 0.844 ± 0.044 | 0.947 ± 0.032 |
| GA | 97.900 ± 39.670 | 28.518 ± 1.591 | 0.871 ± 0.037 | 0.865 ± 0.041 | **0.971** ± 0.015 |
| GWO | 96.560 ± 33.215 | 28.545 ± 1.542 | **0.873** ± 0.036 | 0.867 ± 0.041 | 0.969 ± 0.018 |
| K-means | **82.254** ± 27.600 | **29.319** ± 1.932 | 0.866 ± 0.035 | **0.869** ± 0.031 | 0.970 ± 0.020 |
| Otsu | 180.055 ± 123.179 | 26.523 ± 3.018 | 0.847 ± 0.036 | 0.832 ± 0.045 | 0.859 ± 0.151 |
| PSO | 97.193 ± 34.507 | 28.525 ± 1.561 | 0.872 ± 0.036 | 0.866 ± 0.042 | 0.970 ± 0.016 |
| WOA | 99.172 ± 39.717 | 28.464 ± 1.593 | 0.870 ± 0.037 | 0.865 ± 0.042 | 0.970 ± 0.016 |
| WOA-ZOA | 97.833 ± 39.409 | 28.528 ± 1.613 | **0.873** ± 0.036 | 0.867 ± 0.041 | 0.969 ± 0.019 |
| ZOA | 96.411 ± 34.041 | 28.550 ± 1.527 | 0.871 ± 0.036 | 0.865 ± 0.041 | 0.970 ± 0.016 |

**Table A.25** Mean ± std per algorithm (Kapur-SSIM, $k = 6$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 642.225 ± 308.701 | 20.626 ± 2.345 | 0.670 ± 0.102 | 0.821 ± 0.040 | 0.775 ± 0.141 |
| GA | 642.712 ± 331.590 | 20.693 ± 2.479 | 0.686 ± 0.101 | 0.840 ± 0.033 | 0.791 ± 0.131 |
| GWO | 642.174 ± 337.785 | 20.707 ± 2.485 | 0.686 ± 0.101 | 0.834 ± 0.036 | 0.785 ± 0.132 |
| K-means | **75.404** ± 27.746 | **29.744** ± 1.989 | **0.866** ± 0.040 | **0.919** ± 0.026 | **0.983** ± 0.013 |
| Otsu | 144.104 ± 88.708 | 27.164 ± 2.237 | 0.854 ± 0.037 | 0.891 ± 0.038 | 0.902 ± 0.122 |
| PSO | 645.955 ± 330.014 | 20.663 ± 2.465 | 0.685 ± 0.102 | 0.841 ± 0.032 | 0.796 ± 0.129 |
| WOA | 674.545 ± 336.934 | 20.435 ± 2.369 | 0.680 ± 0.103 | 0.836 ± 0.034 | 0.786 ± 0.129 |
| WOA-ZOA | 629.516 ± 334.982 | 20.811 ± 2.524 | 0.687 ± 0.101 | 0.834 ± 0.035 | 0.789 ± 0.129 |
| ZOA | 620.589 ± 322.979 | 20.857 ± 2.503 | 0.685 ± 0.100 | 0.834 ± 0.032 | 0.803 ± 0.125 |

**Table A.26** Mean ± std per algorithm (Kapur-SSIM, $k = 6$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 84.273 ± 38.188 | 29.253 ± 1.813 | 0.883 ± 0.032 | 0.877 ± 0.040 | 0.974 ± 0.015 |
| GA | 57.715 ± 22.847 | 30.836 ± 1.667 | 0.903 ± 0.030 | 0.902 ± 0.046 | 0.987 ± 0.007 |
| GWO | 57.886 ± 22.499 | 30.821 ± 1.673 | **0.904** ± 0.029 | 0.902 ± 0.046 | 0.986 ± 0.007 |
| K-means | **46.607** ± 15.424 | **31.775** ± 1.895 | 0.902 ± 0.029 | **0.908** ± 0.029 | **0.989** ± 0.006 |
| Otsu | 140.921 ± 131.691 | 28.082 ± 3.431 | 0.883 ± 0.033 | 0.869 ± 0.045 | 0.876 ± 0.158 |
| PSO | 57.890 ± 22.028 | 30.802 ± 1.611 | 0.903 ± 0.030 | 0.901 ± 0.046 | 0.987 ± 0.007 |
| WOA | 60.851 ± 24.183 | 30.592 ± 1.608 | 0.900 ± 0.031 | 0.900 ± 0.044 | 0.986 ± 0.009 |
| WOA-ZOA | 56.513 ± 22.060 | 30.911 ± 1.615 | **0.904** ± 0.029 | 0.903 ± 0.044 | 0.987 ± 0.007 |
| ZOA | 58.163 ± 21.663 | 30.770 ± 1.584 | 0.901 ± 0.030 | 0.900 ± 0.045 | 0.987 ± 0.006 |

**Table A.27** Mean ± std per algorithm (Kapur-SSIM, $k = 8$, mode=gray)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 659.658 ± 321.817 | 20.529 ± 2.391 | 0.675 ± 0.102 | 0.832 ± 0.038 | 0.782 ± 0.138 |
| GA | 641.043 ± 320.629 | 20.685 ± 2.458 | 0.690 ± 0.102 | 0.849 ± 0.032 | 0.802 ± 0.124 |
| GWO | 641.269 ± 335.551 | 20.713 ± 2.488 | 0.690 ± 0.102 | 0.842 ± 0.038 | 0.792 ± 0.127 |
| K-means | **43.272** ± 15.397 | **32.128** ± 1.911 | **0.904** ± 0.029 | **0.948** ± 0.019 | **0.991** ± 0.008 |
| Otsu | 94.273 ± 47.728 | 28.884 ± 2.064 | 0.890 ± 0.028 | 0.919 ± 0.036 | 0.959 ± 0.035 |
| PSO | 639.910 ± 311.710 | 20.665 ± 2.410 | 0.689 ± 0.102 | 0.850 ± 0.031 | 0.806 ± 0.123 |
| WOA | 658.693 ± 321.473 | 20.544 ± 2.411 | 0.684 ± 0.103 | 0.845 ± 0.035 | 0.798 ± 0.125 |
| WOA-ZOA | 612.708 ± 312.013 | 20.899 ± 2.490 | 0.690 ± 0.101 | 0.840 ± 0.037 | 0.800 ± 0.122 |
| ZOA | 600.720 ± 288.797 | 20.951 ± 2.448 | 0.689 ± 0.100 | 0.842 ± 0.033 | 0.817 ± 0.117 |

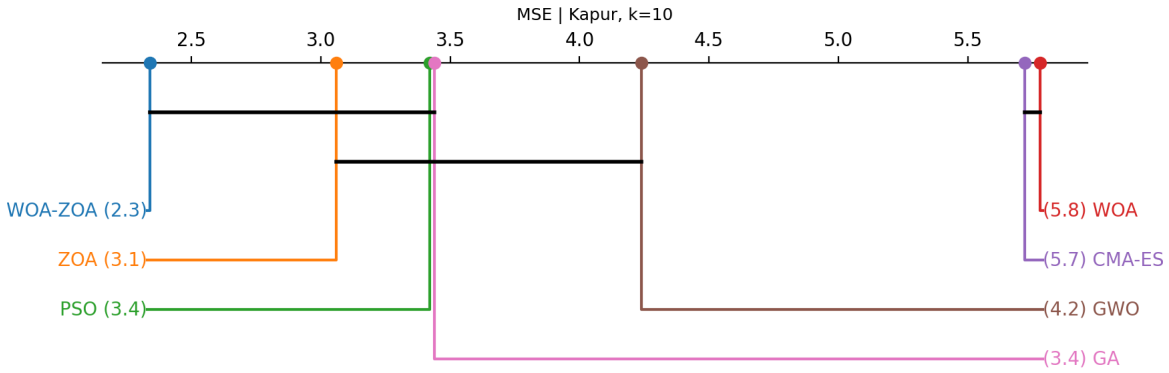
**Table A.28** Mean ± std per algorithm (Kapur-SSIM, $k = 8$, mode=color)

| Algorithm | MSE | PSNR | SSIM | FSIM | QILV |
|---|---|---|---|---|---|
| CMA-ES | 58.053 ± 23.667 | 30.848 ± 1.809 | 0.902 ± 0.028 | 0.902 ± 0.033 | 0.982 ± 0.015 |
| GA | 39.260 ± 19.975 | 32.589 ± 1.770 | 0.923 ± 0.025 | 0.924 ± 0.048 | **0.993** ± 0.006 |
| GWO | 39.189 ± 19.377 | 32.588 ± 1.769 | 0.923 ± 0.025 | 0.923 ± 0.047 | 0.992 ± 0.007 |
| K-means | **30.342** ± 9.957 | **33.630** ± 1.857 | **0.926** ± 0.025 | **0.932** ± 0.027 | **0.993** ± 0.004 |
| Otsu | 98.385 ± 120.779 | 29.904 ± 3.544 | 0.909 ± 0.034 | 0.894 ± 0.045 | 0.921 ± 0.126 |
| PSO | 38.372 ± 19.769 | 32.686 ± 1.757 | 0.923 ± 0.026 | 0.924 ± 0.048 | **0.993** ± 0.006 |
| WOA | 41.155 ± 17.806 | 32.325 ± 1.687 | 0.918 ± 0.026 | 0.922 ± 0.041 | 0.992 ± 0.006 |
| WOA-ZOA | 35.737 ± 13.951 | 32.875 ± 1.531 | 0.923 ± 0.024 | 0.927 ± 0.039 | **0.993** ± 0.003 |
| ZOA | 38.550 ± 17.803 | 32.601 ± 1.620 | 0.921 ± 0.025 | 0.922 ± 0.045 | **0.993** ± 0.005 |

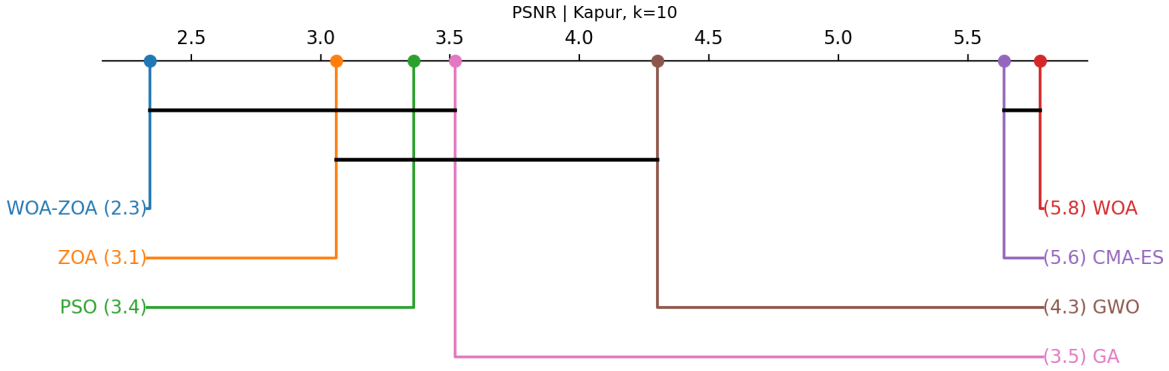
**Table A.29** Mean ± std per algorithm (Kapur-SSIM, $k = 10$, mode=gray)

## A.2 Additional performance plots (CD)

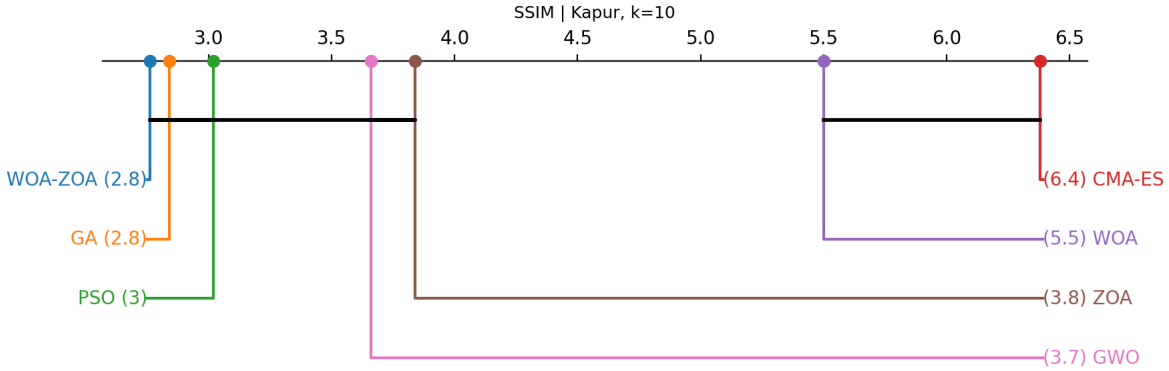
This attachment collects additional plots (stored in the `cd/` folder) referenced in Section 3.4.




**(a)** MSE (Kapur, $k = 10$)

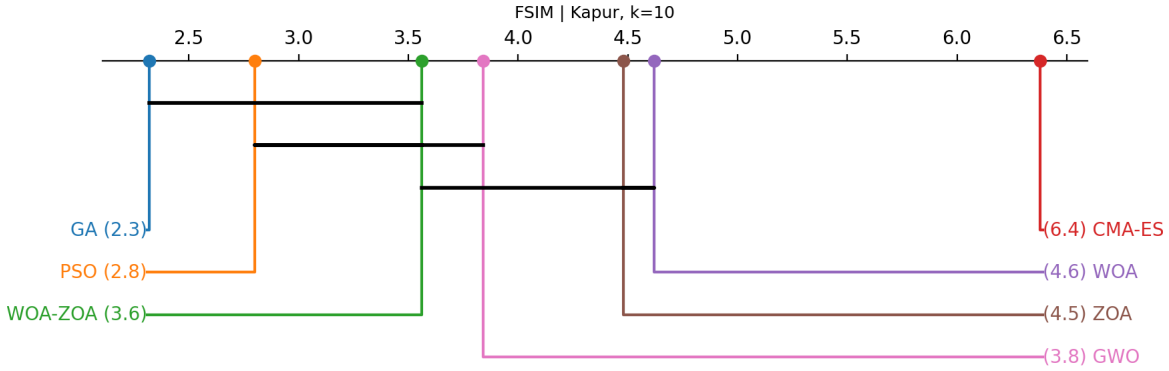



**(b)** PSNR (Kapur, $k = 10$)

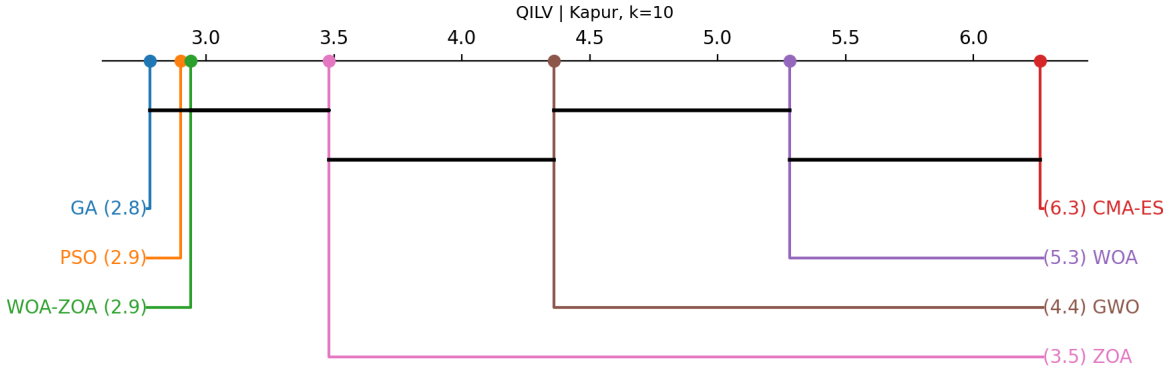



**(c)** SSIM (Kapur, $k = 10$)

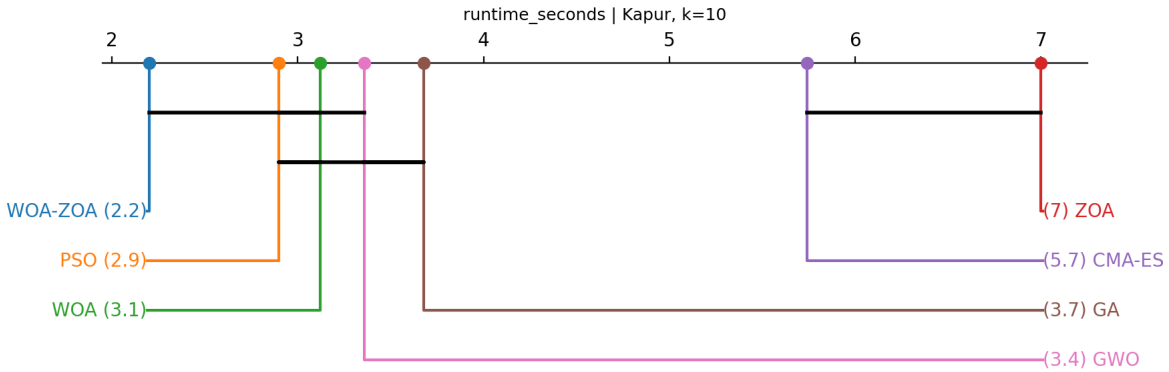
**Figure A.15** Additional plots for Kapur objective ($k = 10$).




**(a)** FSIM (Kapur, $k = 10$)

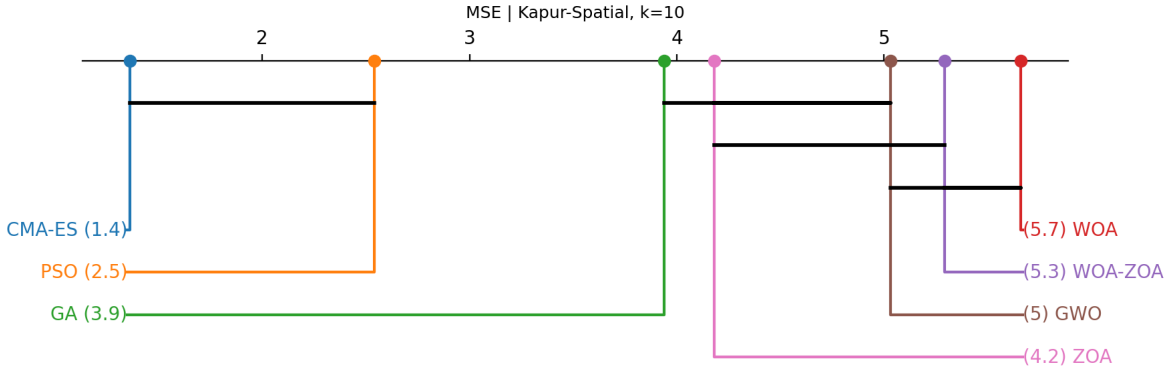



**(b)** QILV (Kapur, $k = 10$)

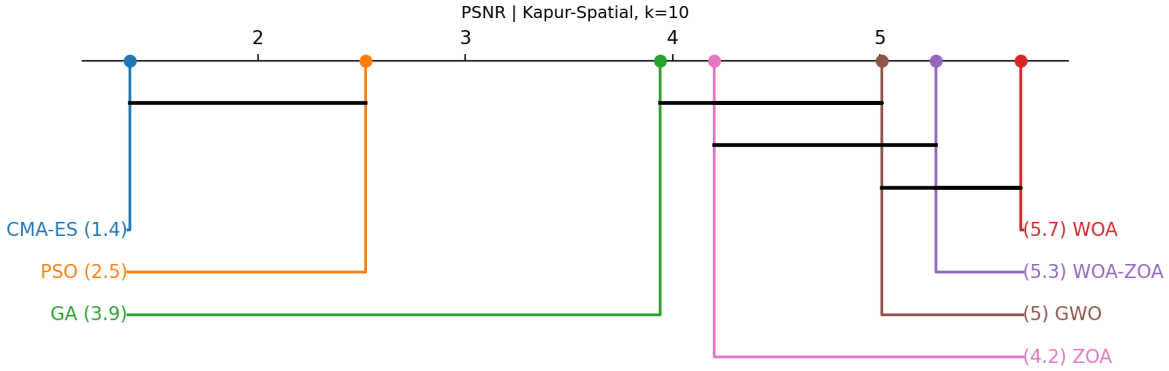



**(c)** Runtime (s, Kapur, $k = 10$)

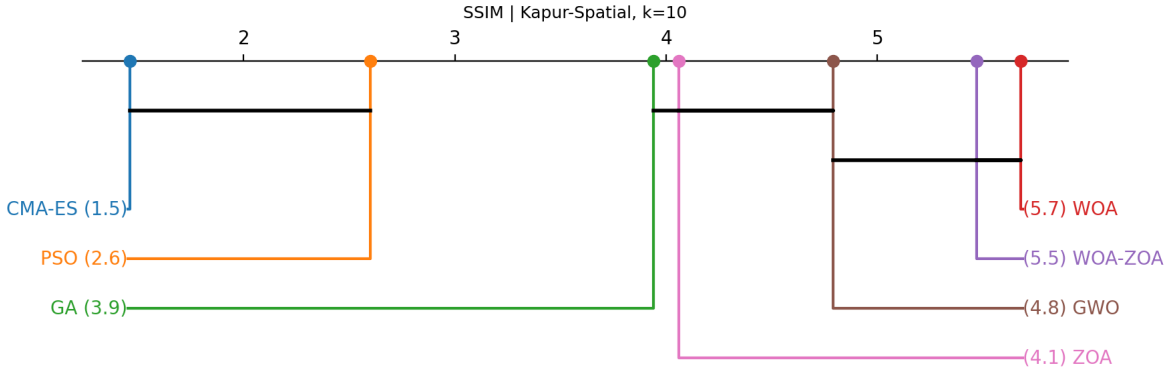
**Figure A.16** Additional plots for Kapur objective ($k = 10$).




**(a)** MSE (Kapur-Spatial, $k = 10$)

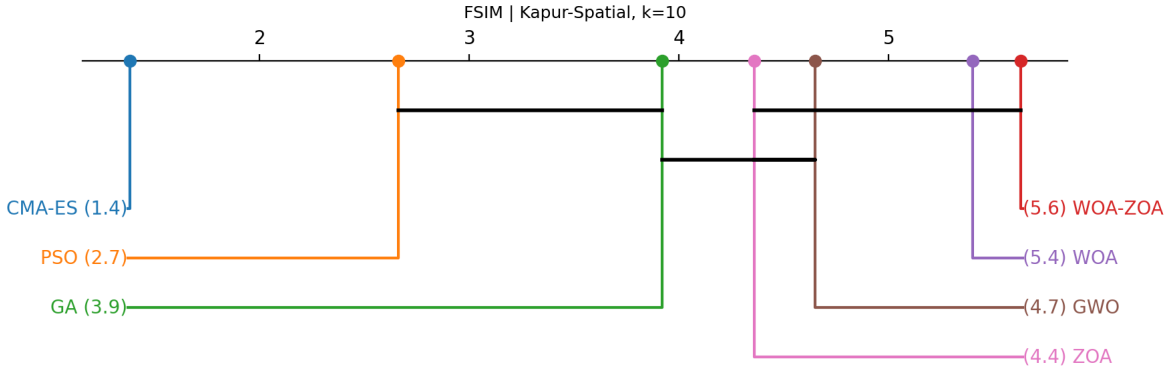



**(b)** PSNR (Kapur-Spatial, $k = 10$)

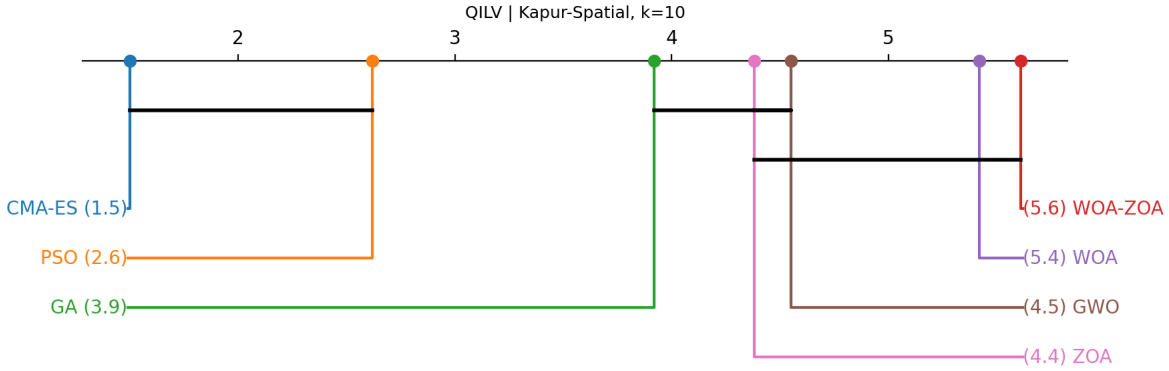



**(c)** SSIM (Kapur-Spatial, $k = 10$)

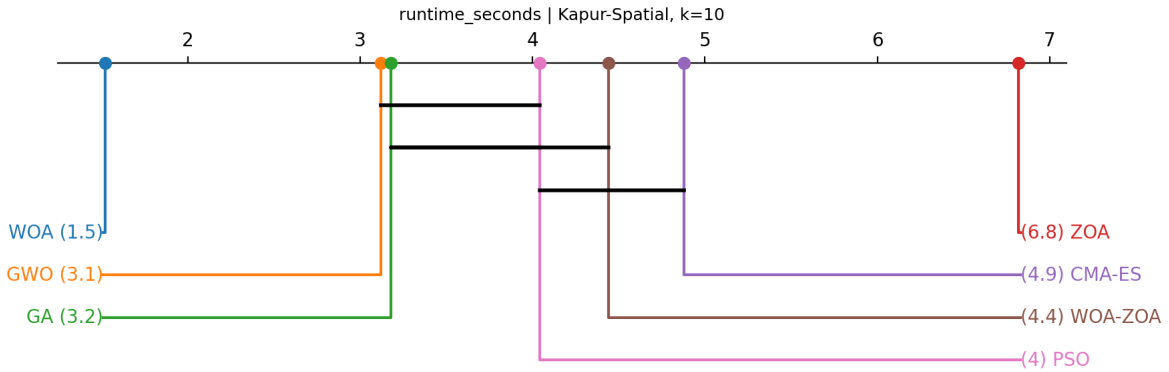
**Figure A.17** Additional plots for Kapur-Spatial objective ($k = 10$).




**(a)** FSIM (Kapur-Spatial, $k = 10$)

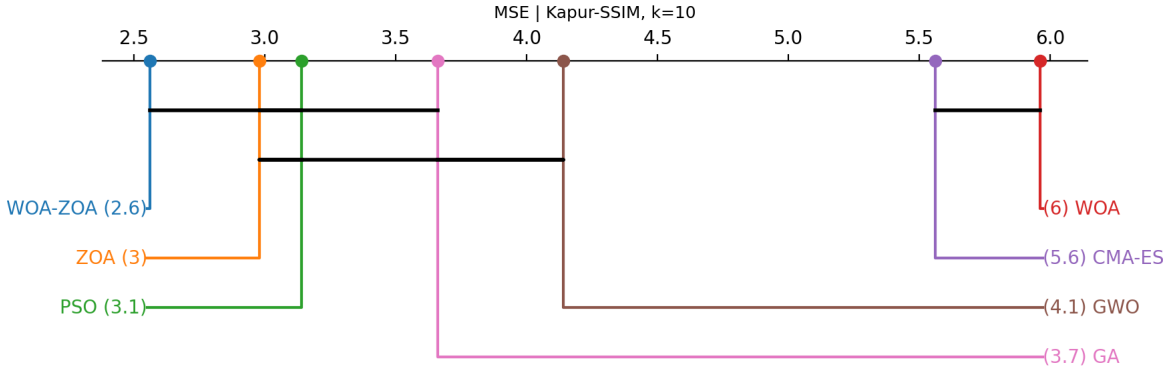



**(b)** QILV (Kapur-Spatial, $k = 10$)

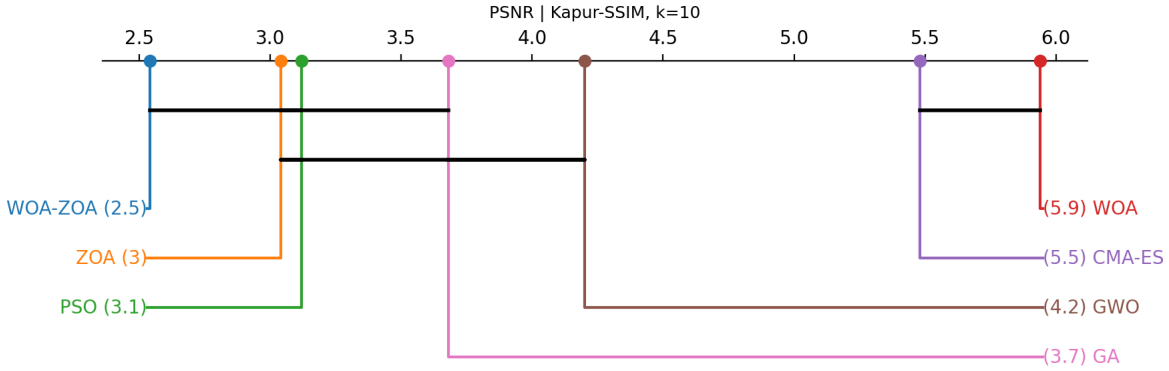



**(c)** Runtime (s, Kapur-Spatial, $k = 10$)

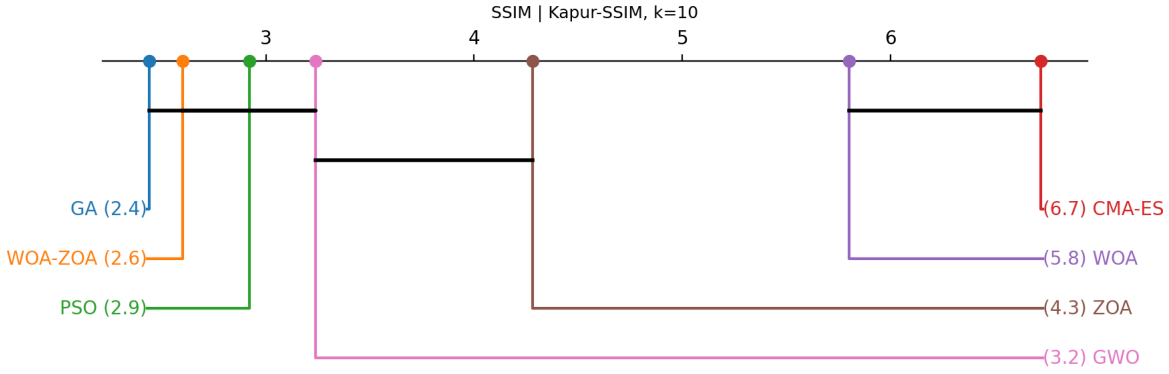
**Figure A.18** Additional plots for Kapur-Spatial objective ($k = 10$).




**(a)** MSE (Kapur-SSIM, $k = 10$)

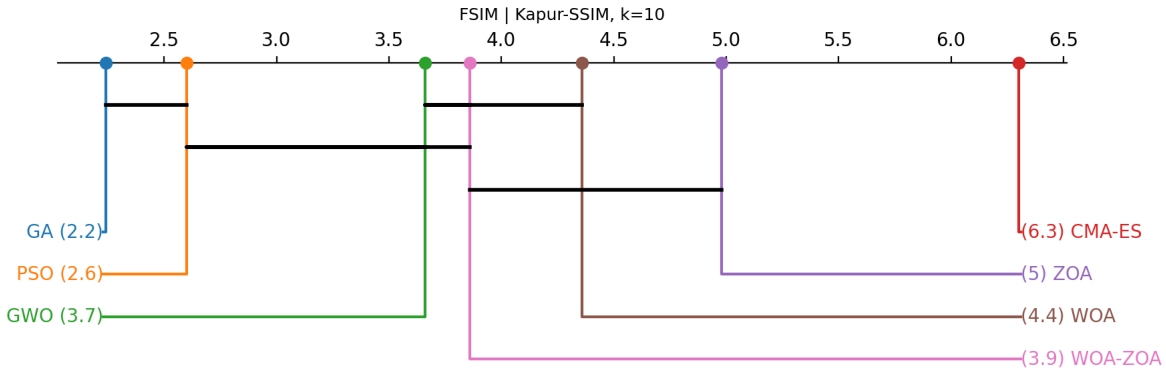



**(b)** PSNR (Kapur-SSIM, $k = 10$)

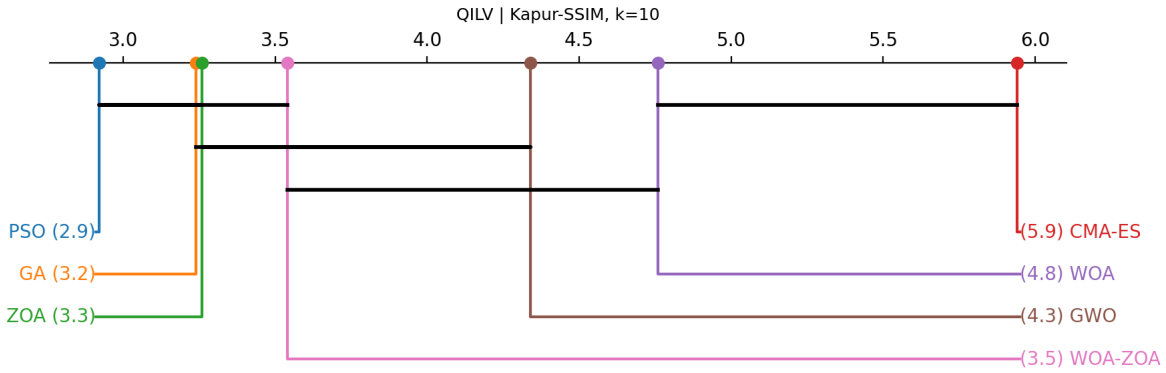



**(c)** SSIM (Kapur-SSIM, $k = 10$)

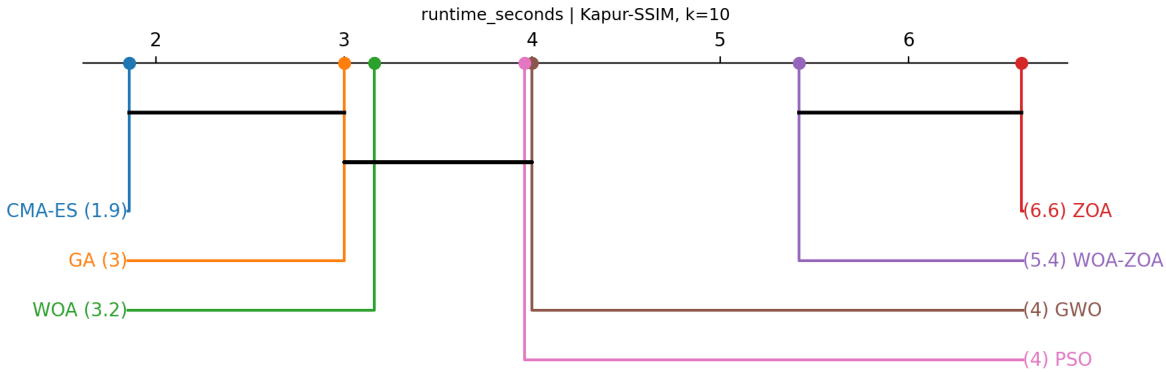
**Figure A.19** Additional plots for Kapur-SSIM objective ($k = 10$).




**(a)** FSIM (Kapur-SSIM, $k = 10$)




**(b)** QILV (Kapur-SSIM, $k = 10$)




**(c)** Runtime (s, Kapur-SSIM, $k = 10$)

**Figure A.20** Additional plots for Kapur-SSIM objective ($k = 10$).